Programming Concepts using C

PRAVESH – Student Guide Subject: Programming Concepts using C
V1.0

Chapter 1: Introduction to C-programming 3
Chapter 2: Control Structures 23
Chapter 3: Iteration 36
Chapter 4: Preprocessor 45
Chapter 5: Function 53
Chapter 6: Data types and Storage Classes 64
Chapter 7: Arrays 73
Chapter 8: String 83
Chapter 9: Pointers 96
Chapter 10: Structures 110
Chapter 11: Files 129
Chapter 12: Linked List 144
Chapter 13: Miscellaneous 157

Programming Concepts using C

Chapter 1: Introduction to C-programming

Objectives

- Discuss the history of C
- Recognize where the 'C' language stands among all other languages.
- Describe the grammar of 'C'
- Understand basic data types.
- Define constants, variables, keywords.
- Use operands, Operators and Expressions.
- Describe how type conversion takes place.
- List the order of precedence and Associativity.
- Described the function used for inputting and outputting.
- Write simple 'C' programs

Introduction to Programming Concepts

Program

A program is a list of instructions, together with any fixed information required to carry out those instructions. This applies to computers, of course, but also to any other subject that involves fixed instructions.

Some machines are designed with one or a few fixed programs already built in. For example, a washing machine allows you to set the desired water level and temperature, pick one of perhaps four possible wash cycles, and start the action going. Of course, you also have to add detergent and the clothes to be washed, but the actions of the washing machine itself are already completely defined and cannot be changed. This washing machine, then, is dedicated to its task and cannot be reprogrammed to perform any other tasks.

Some more modern machines are partly dedicated. For example, a microwave oven today is dedicated to the task of heating whatever is placed inside it. However, it is up to the user to specify, or program, the length of time, the power level, possibly a continuation at a different power level, and perhaps a temperature probe. In a similar fashion, modern ovens can be set to simply bake or to broil, or they can be set up to delay for a preselected time, cook the food that is in them for a preselected time, and then turn off or else drop down to a minimum "keep warm" temperature.

A computer, unlike washing machines, microwave ovens is a general purpose device. It is not pre-programmed to take any specific action other than to initialize itself when you turn it on. Once you've turned it on and have your command prompt or graphic desktop on the screen, you must tell the computer what to do -- it won't do anything else by itself.

Although it is always necessary to tell the computer what to do, there are many ways available to do this. Some methods are long and involved; others are quite simple and direct. This group of pages will provide an introduction to programs and programming methods.

Since a computer program consists of a series of instructions for the computer to carry out, plus any necessary fixed data required to perform those instructions, the process of programming consists of defining those instructions and that data. Thus, in order to design a program for a computer, you must determine three basic elements:

1. The instructions that must be performed.
2. The order in which those instructions are to be performed.
3. The fixed data required to perform the instructions.

At this stage, you're not even working with a computer, except possibly as a word processor so you can write your work to a disk file instead of onto a piece of paper. Either way, however, all of the early stages of computer programming are essentially paper processes.

The reason for this is that you must determine, clearly and in detail, exactly what you want the computer to do before you start telling the computer to do it. Remember that the computer will carry out the instructions you give it, exactly as given. It can't tell what you wanted; it can only do what you said.

Before your new program ever gets anywhere close to a computer, there are several steps you must take. These steps are:

Define the problem.

Before you can tell a computer or even another person what you want to accomplish, you must know it yourself. You must be able to clearly state what the computer is to acomplish or produce as the end result of the activities it will carry out.

Define the solution.

Now that you know what information the computer is to produce as its final output, you need to look at what information you have available and what information you still need, that will apply to that output. You also need to define the equations, logical procedures, or other methods you will need to use to manipulate the raw input data into becoming the final desired output.

Map the solution.

The third paper step in programming is to lay out the solution in its proper sequence. Remember that the order in which actions are taken is just as important as the actions themselves. You need to organize the solution procedure into its proper sequence, taking choices and alternatives into account.

Early programming languages were designed for specific kinds of tasks. Modern languages are more general-purpose. In any case, each language has its own characteristics, vocabulary, and syntax. While this page will not by any means cover all of the available programming languages, we will take a look at a number of the better-known languages.

FORTRAN

One of the earliest computer languages, FORTRAN (an acronym for FORmula TRANslator) was designed to handle mathematical operations, originally on mainframe computers. FORTRAN was unable to handle text manipulations of any sort, and could just barely place quoted text in its printed output.

Assembly Language

Assembly language is a symbolic representation of the absolute machine code of a particular processor. Therefore, each processor has its own specific assembly language, although a family of processors, such as the Intel 80x86 series, may share some or all of its assembly code.

C

First came as an experimental language called A, which was improved, corrected, and expanded until it was called B. This language in turn was improved, upgraded, and debugged and was finally called C. The C language has turned out to be quite versatile and amazingly powerful. The C language is amazingly simple, and is nevertheless capable of great things. The Unix operating system, which has been adapted to a wide range of platforms and is gaining in popularity, is written in C.

C++

When the concepts of objects and object-oriented programming were being developed, the standard C language didn't have the built-in structures to handle them. However, C was (and is) still highly useful and well worth keeping around, so a sort of extended C language was developed. This language was essentially "C and then some", or C-plus (C+). As the concepts of object-oriented programming continued to develop, C+ had to be upgraded, and became C++

These languages requires representation of Words, Numbers, and Values with Variables

---

Variables are an important concept in computer programming. A variable is a letter or name that can store a value. When you create computer programs, you can use variables to store numbers, such as the height of a building, or words, such as a person's name. Simply put, you can use variables to represent any kind of information your program needs.

You might ask, "Why use a variable when I could just use the information instead?" As the name implies, variables can change the value that they represent as the program is running. For example, you might write a program to track the number of jelly beans you have in a jar on your desk. Because candy is meant to be eaten, the number of jelly beans in the jar is likely to change over time. Rather than rewriting your program every time you get a sugar craving, you can represent the number of jelly beans with a variable that can change over time.

Storing Information in Variables
There are three steps to using a variable:

Declare the variable. Tell the program the name and kind of variable you want to use.
Assign the variable. Give the variable a value to hold.
Use the variable. Retrieve the value held in the variable and use it in your program.

Programming concepts using C

Introduction to C

Welcome to the world of 'C', a programming language which is simple to learn and highly effective. As you go on exploring 'C' and get experience, you will find it very interesting to work with.

When we learn any new language, we first study the grammar. Similarly here we will see grammar of 'C' and all the basic building blocks, which will help us to write powerful programs. We begin with history of 'C' which tells us how 'C' evolved.

History of 'C'

The history of 'C' ranges back to the 1960's, when a number of computer languages were being used for various purposes. COBOL was being used for commercial applications, FORTRAN for engineering & scientific application and so on.

- An international committee was set to develop a language, which can program all types of application instead of using different languages for specific purpose. This committee developed a language called 60. But ALGOL 60 was too abstract and general.

- To reduce this abstractness and generality, a new language called Combined programming Language (CPL) was developed in 1963 at Cambridge University. But CPL was hard to learn and difficult to implement, since it was extensive with many features.

- Then, Martin Richards developed Basic Combined Programming Language (BCPL) in 1967 at Cambridge University, to extract good features of CPL.But BCPL was less powerful and too specific.

- While BCPL was developed, at around the same time a language called 'B' was written by Ken Thompson in 1970 at AT & T's Bell Labs, for the first Unix system on DEC PDP-7, as simplification of CPL. But 'B' language also was very specific.

- At last in 1972, Dennis Ritchie designed 'C' by taking 'B' and BCPL as the base. It was designed for and implemented on the Unix operating system on DEC PDP-11. Thus it is

closely related with Unix operating system. BCPL 'and 'B' were typeless languages while 'C' provides a number of data types.

Where 'C' stands?

- The languages like BASIC, FORTRAN, COBOL, and PASCAL are called high level languages. These languages are application specific and are used to develop applications related to particular field. They have been designed to give better programming efficiency.
- Assembly language and Machine language are called low-level languages. These are machine-oriented languages and are designed to give better machine efficiency.
- 'C'language provides facilities of both high level programming as well as low-level programming. Hence it can be called as middle level language.
- It is not specialized to any application as different types of application can be developed using 'C'.
- It is not hardware or system dependent. Hence portable programs can be written with'C'.
- Compliers and operating system can be written using 'C'. Hence it can be called as 'systems programming language'.
- In 1988 American National Standards Institute (ANSI) established a standard definition of 'C' known as ANCI standard or 'ANSI C'. Most of the features of ANSI C are supported by modern compilers.

Grammar of 'C'

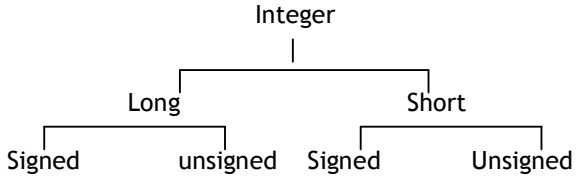
- Data types.
- Constants.
- Variables.
- Keywords.
- Operands, Operators and Keywords.

Data Types

There are basically 4 data types in 'C'.

- char - It is used to define characters. They occupy one byte.
- int - It is used to define integer numbers .They occupy 2 bytes.
- float - It is used to define floating point numbers (single precision). They occupy 4 bytes.
- Double - It is used for double precision floating point. They occupy 8 bytes.

Additional qualifiers or modifiers can be applied to these basic types to get additional ranges like,



Character
|
Signed Unsigned

Double
|
Long double

Constants

- Integer constants.
- Floating point constants.
- Character constants.

A constant can be defined as a quantity that does not change. There are 3 different types of constants. Integer, floating point and character constants.

Integer Constants

Integer constant is an integer quantity containing sequence of digits. It has no decimal point. It can be either positive or negative. Blanks and commas are not allowed within integer constants. The value of the constant must lie within the range of integer values.
A decimal integer constant can consist of any combination of digits taken from 0 through 9.
E. g. 0 5280 9999
An octal integer constant can consist of any combination of digits from 0 through 7. The first digit must be 0.
E. g. 0 01 0743
A hexadecimal integer constant must begin with 0x or 0x, followed by any combination of digit from 0 through 9 and a through f.
E. g. 0x1 oxab7f

Certain suffixes can be given to constants, which identify them for the type

Type	Suffix
long	l or L
unsigned long	Ul or UL

E. g.
5000U unsigned (decimal)
0x5000U unsigned (hex)
1234UL unsigned long (decimal)
0777U unsigned (octal)

Floating Point Constants

Floating point constant contains a decimal point or an exponent. It could be either negative or positive. No commas or blanks are allowed within a real constant
E. g. +3.2e-5, -0.2e-5, -32.76

Type	Suffix
float	f or F
long double	l or L

Character Constants

A character constant is a character enclosed in single quotation marks. A character constant can contain only one character.
E. g. 'A' '3' '$'
Each character has a corresponding ASCII value (which is the numeric value of character in machine's character set.) as given below.

Character	ASCII
'A'	65
'B'	66

Variables

- Variable declaration

A variable can be defined as a quantity, which may vary during program execution.

The name given to the memory location which stores the quantity is called **variable name**.
There are certain rules, which must be followed while naming a variable. A variable name can be any combination of alphabets, digits or underscores. The length of variable name can range from 1 to 8. The first character of the variable name must be an alphabet. No commas or blanks are allowed within a variable name and no special symbol other than underscore can be used.
'C' is a case sensitive language i. e. variable name 'NAME' and 'name' are different variables.

Variable Declaration

Declaration is the place where the nature of the variable (i. e. type of variable) is stated but no storage is allocated. Definition refers to place where the variable is created or stored.

Any variable must be declared before we use it. The variable is declared as follows

```
Storage - class    data - type    variable - name;
```

Let us keep the storage-class part aside right now. We will consider it later.

```
Data - type    variable - name;
```

E.g.

```
char  basal;
float cd;
```

Meaningful names must be given to variables. Two or more variables of the same type can declare separating them with comma.
E. g.

```
int  m,  m_hra;
```

Thus declaration specifies a type and contains a list of one or more variables of that type.

Keywords

There are certain reserved words in 'C', which are called keywords. The compiler already knows meaning of keywords. Following are few standard keywords

auto	break	case	extern	float	far
sizeof	static	struct			

They are all written with small case letters. Keywords cannot be used as variable name as it will alter their meaning.

Operands, operators and Expression

- Arithmetic operators
- Unary operators
- Assignment operators.

Operands and operators are both together used to form expressions.
Data items that operators act upon are called operands. Operands can be constants or variables. When all the operands in an expression are constant it is called a constant expression. Most operators require two operands to operate upon. They are called binary operators.
Some operators require single operands; they are called unary operators. We will see how arithmetic operators; unary operators and assignment operators are used.
Logical (Result = operand1 operator operand2) * / Expression */

Arithmetic Operators

There are five arithmetic operators in 'C'

Operator	Purpose	Remarks
+	Addition	
-	Subtraction	
*	Multiplication	
/	Division	The denominator must be nonzero
%	Remainder (after integer division)	The denominator must be nonzero. Both operands should be integers. It truncates any fractional part.

The operands must be numeric values i. e. they can be integer quantities, floating point or character quantities (each character represents an integer value). When both operands of division operator (/) are integers, result is an integer. It is called integer division. Integer division result in truncation towards zero. If both operands are floating point or if one is floating point and one is integer, result is floating point.
There is no operator for exponentiation in 'C'.
Arithmetic instruction can be formed as

```
/* variable declaration */
int  ad;
float k = 4.0, alpha = 10.0, beta = 8.0, /* variable
```

```
initialized during declaration */
float delta;
delta = alpha *beta / k + 3.2 * 2 / 5 ; /*   arithmetic instruction */
```

The expression on the right of '=' is evaluated and assigned to variable on the left hand side of '='. As it is seen initialization can be done during declaration.

Unary Operators

- Minus operator.
- Increment and decrement operator.
- Sizeof operator.
- Unary operators require a single operand.

Minus Operator

Minus sign precedes a numerical constant, a variable or an expression. It is used for negation. It is different from arithmetic subtraction operator.

E. g. -1 -3.0

Increment and Decrement Operator

These are used for incrementing and decrementing variables. Increment operator '++' adds 1 to its operand, while decrement operator '_' subtracts 1 from its operand. The operator may be **postfixed** or **prefixed**. Postfixing and prefixing makes a difference when the operators are used in expression.
E. g.

```
j++ ;
++j ;
```

Both are same when used as simple statements.
If we have

```
j = 3 ;
X = j+ + ;
```

Here j is first assigned to x and then increment by 1. Hence X = 3, J = 4;

```
X = + + j ;
```

Here j is first incremented and then assigned to x. Hence x = 4, j = 4;

Sizeof Operator

It returns the size of its operand in bytes.

```
int i , j ;
j = sizeof (i) ;
```

will give j = 2. (Since integer occupies 2 bytes and i is an integer).

Assignment Operator

When we write an expression

```
x = a + b ;
```

Here '=' is the assignment operator. In this case expression on the right of '=' is evaluated and value is assigned to the variable on the left.

There is one more type of assignment operator. If we have an expression where variable on the left is repeated immediately on the right, it can be written as:

```
i  = i + 2 ;
```

```
i  + = 2 ;
```

The operator '+' is called the assignment operator. Most binary operators have corresponding assignment operators. General form of assignment expression being

```
expr1 op = expr2 ;
```

Instead

```
expr1 = (expr1) op (expr2) ;
```

Where expr1 and expr2 are expressions and op can be one of the operators.

```
+        -        *        /        %        <<        >>        &        ^        |
```

There are more five operators known as bitwise operators which would be taken up in miscellaneous chapter.
This type of assignment operators make long expression easy to work with.
E. g.

```
yyval [yypv [p3 + p4] + yypv [p1 + p2] ] + = 2; is easier than
yyval [yypv [p3 + p4] + yypv [p1 + p2] ] = yyval [yypv [p3 + p4] + yypv [p1 + p2] ] + 2 ;
```

Type Conversion

An arithmetic operation between two integers gives an integer result. Operation between two real numbers gives a real result. Let us turn our attention to cases when the operands in an expression are of different types. The general rule is that if a binary operator has operands of different types, then the 'lower' type is promoted to the 'higher' type, before the operation proceeds.
If either operand is long double convert other to long double and result is long double.
If one operand is integer and other float, integer is converted to float and result is float and so on.

E. g	Result	
5 / 2. 0	= 5.0 / 2.0	= 2.5
2.0 / 5	= 2.0 /5.0	= 0.4
2 / 5	= 2 / 5	= 0
5 / 2	= 5 / 2	= 2
5.0 / 2.0	= 5.0 / 2.0	= 2.5

Further if type of expression on the right of '=' and type of variable on the left differ, then the value of expression is promoted or demoted depending upon the type of variable on left side of '=',
E. g.

```
int  j ;
float b ;
j  = 3.5 ;
b = 30 ;
```

Here though 3.5 is a real number. When it is assigned to j (which is an integer), it is converted to an integer i. e. j becomes 3. Conversion to an integer truncates the decimal part. Also 30 assigned to b convert it to floating point value 30.000. Since b is a float variable.
In the following expression.

```
/ * variable declaration * /
float a, c ;
int s ;
s = a * b = c * / 100 + 32 / 4 - 3 * 1 ; / * Expression * /
```

Here since the expression on the right is a mixed type, whenever an operator has different types of operands, they are converted according to above given rule and then expression solved. Whatever will be the resulting type of expression on right it is finally truncated to integer value and assigned to s. Since s is of integer type.

Precedence and Associativity

If we have an expression

```
J = 2 * 3 / 4 + 4 / 4 + 8 -2 + 5 / 8 ;
```

You may wonder which operation will be carried out first. In order to solve the dilemma, 'C' provides precedence i. e. the order of evaluation. The following table gives the precedence and associatively of operators.

Operator Precedence Chart		
Operator Type	Operator	Associativity
Primary Expression Operators	() [] . -> expr++ expr--	left-to-right
Unary Operators	* & + - ! ~ ++expr --expr (typecast) sizeof()	right-to-left
Binary Operators	* / %	left-to-right
	+ -	
	>> <<	
	< > <= >=	
	== !=	
	&	

	^	
	|	
	&&	
	||	
Ternary Operator	?:	right-to-left
Assignment Operators	= += -= *= /= %= >>= <<= &= ^= |=	right-to-left
Comma	,	left-to-right

The precedence decreases from top to bottom i. e. operations with higher precedence (top) are carried out before operations with lower precedence. Using parenthesis can alter the natural order.
E.g. In the table

*	/	%	occur above	+	and	-

Thus multiplication, division and remainder operation in an expression will be carried out before addition and subtraction operation. Again the question arises as to which operation among those with same precedence will be carried out first? The order in which operation within the same precedence group is carried out is called associativity. Most of the operations have associativity from left to right i. e. to solve conflict between *, /, % start the expression from left and perform the operation as the operators are encountered going to right.
The idea can be made clear by evaluating the previous expression stepwise.

```
j = 2* 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8;
j = 6 / 4 + 4 / 4 + 8 – 2 + 5 / 8;        operation *
j = 1 + 4 / 4 + 8 – 2 + 5 / 8 ;           operation /
j = 1 + 1 + 8 – 2 + 5 / 8;                operation /
j = 1 + 1 + 8 – 2 + 0                     operation/ (0 – since it is an integer division.)
j = 2 + 8 – 2;                            operation +
j = 10 – 2;                               operation +
j = 8;                                    operation –
```

Multiplication and division are performed before addition and subtraction.

Escape Sequence	Purpose	Remarks
\n	Newline	Takes cursor to beginning of next line.
\b	Backspace	Moves cursor one position to left of current position
\f	Formfeed	Advances the computer stationary attached to the printer to the top of next page.
\’	Single quote	
\\	Backslash	
\t	Tab	Moves cursor to next tabstop*

\r	Carriage return	Takes cursor to line in which it is placed.
\a	Alert	Alerts user by sounding speaker inside computer.
\”	Double quote	
\?	Question mark	

Above is a table of escape sequences. Escape sequences are written as backward slash followed by a character. The backward slash is called the escape character because it makes the character following it to escape from its original meaning and gives a special meaning to it. Though an escape sequence may appear as two characters (a backslash and another character), it is actually a single character. Certain nonprinting characters as well as double quote (“), apostrophe (‘), question mark (?) and backslash (\ ) can be printed using escape sequences..
Tabstop: An 80-column screen has 10 tabstop i. e. screen is divided into 10 zones of 8 columns each. Printing a tab takes cursor to beginning of next printing zone.

Library Function

Large number of library function, that carry out commonly used calculation and operations accompanies ‘C’. These library function are not part of the language but are provided with the compiler.

Few library functions are given below.

Function	Purpose
abs(j)	return absolute value of j.
sin(d)	return since of d.
getchar()	Enter a character from standard i/p device

In order to use these function we need to declare them. This information is stored in special files called the header files. Each header file contains information about group of related library functions. The required information can be obtained by accessing these header files in the program. This can be done using the preprocessor directive #include<file name>. This would be studied in detailed in later chapters.

Input Output

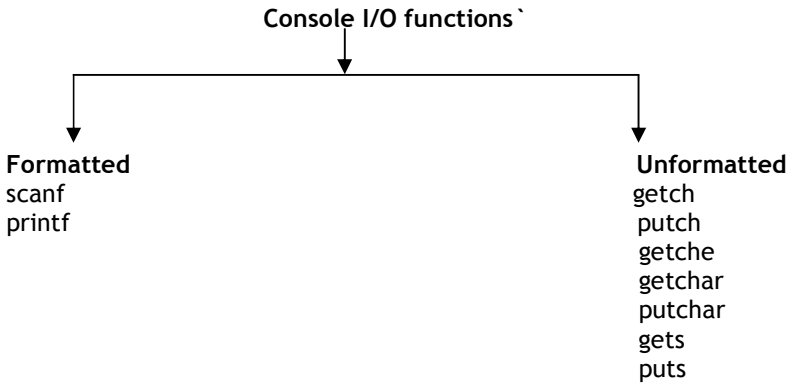
- Input output functions.
- Conversion specifier.

Input and output functions

- printf
- scanf
- getch
- getchar
- gets and puts

Library functions are available for transferring of information between computer and standard input and output devices. E. g. keyboard (I/p device) and monitor (o/p device). These are called console I/O functions.

The declaration of these function and related symbolic constants (more in preprocessor chapter) are available in header file stdio.h. Thus in order to use console I/O function in a program, we only need to access this file in the program. The following figure shows the classification.



Formatted functions allow us to control where o/p would appear on screen, number of places after decimal point, spaces between two values etc.

printf()

This function is used to display output on screen. I.e. it moves data from computer memory to output device. The format of the function is

```
printf ("format string", arg1, arg2...) ;
```

Format string contains the formatting information. Which consist of general characters which can be displayed as they are, conversion specifications, and escape sequences. arg1, arg2… are individual o/p data items. They can be constants, variables (of any data type), array names or even constant expressions.
E. g.

```
# include<stdio.h>
void main()
{
int avg  =   345 ;
float per  = 69.2f ;
char grade = 'B' ;
printf ("Average =  %d \n percentage = %f  \n Grade  = %c", avg, per, grade) ;
}
```

The printf function starts examining the format string from left to right. It displays the characters as they are encountered directly on the screen, till it comes across % or \. When it comes across first conversion specifier it take the first argument and prints it in given format. When it comes across an escape sequence it takes the related action. It continues further in the same way.

Here printf prints "Average =" first. When %d is encountered value of avg is printed. \n causes cursor to move to the beginning of newline. Then "percentage = "is printed. When %f is encountered value of next argument i. e. per is printed and so on. Number of conversion specifiers should match the number of arguments.

scanf()

Data can be entered into the computer from an input device (like keyboard) using the function scanf. The format of the function is

```
scanf ("format string",   &arg1, &arg2 …) ;
```

It returns number of data items returned successfully.

The format strings consist of the conversion specifier. The arguments can be variables or array name and should represent address of variables, where the data item is to be stored. I.e. each variable must be preceded by an ampersand (&). However array names should not begin with an ampersand. For the time being it is sufficient to know that if we have a variable i, its address is &i. We will study the address concept later.
E . g.

```
int   avg ;
float  per ;
char   grade ;
scanf ("%d       %£     %c" , &avg,    &per ,&grade) ;
```

The scanf function is opposite to the printf function. It reads input, interprets them using the conversion specifier and stores them in the given variable. The conversion specifier for scanf are same as that used for printf, with […] being an additional conversion specifier. Using the conversion specifier %s, we cannot input string with blank spaces. This can be achieved by using […]. How do we use it? If we say

```
scanf ("% [A B C ' ' D] "  line) ;
```

Then scanf will read characters from input as long as they match characters specified in square brackets and will terminate on a mismatch. The order of entering characters is not important.
If we precede the characters in square brackets with a circumflex (^), it has the opposite effect. Scanf will read characters from input as long as no character from the square bracket is encountered. The scanf function requires an enter key to be pressed after supplying the input, in order to accept the input.
The following are the unformatted function used to input and output characters.

getch()

It is used to input a single character. It will instantly read the character and does not require enter key to be pressed. It returns the character typed but does not echo it on the screen.

```
int getch (void) ;
ch = getch();
```

ch is assigned the character returned by getch.

putch()

It is the counterpart of getch. i. e. it displays a single character on the screen. It return the character displayed

```
int putch (int) ;
putch(ch) ;
```

ch => the character to be printed.

getchar()

It is used to input a single character. It requires enter key to be passed following the character that you typed. It echoes the character that you entered.

```
ch   = getchar ;
```

putchar()

It is other side of getchar. It displays a single character on the screen.

```
putchar(ch) ;
```

The above functions are alternative to scanf and printf used for inputting and outputting characters. These character I/O function can be used to input and output string by reading or writing one character at a time inside a multipass loop. (We will study looping later).
There are functions available for inputting and outputting string directly. Strings are sequence of character that is terminated with '\0' character, which is called NULL character.

gets() and puts()

They facilitate transfer of string between computer and standard input-output devices. They accept single argument. Argument must be data item that represent a string. The string may include white space characters. When using gets, enter key has to be pressed to end the string. The gets and puts functions offer simple alternative to use of scanf and printf for reading and displaying string as illustrated.

E. g.

```
#include<stdio.h>
main ( )
{
char line [80] ;
gets (line) ;
puts (line) ;
}
```

Conversion Specifiers

%d, %f, %c are called conversion specifiers. They tell how the arguments are to be displayed. That is avg to be displayed as an integer, per as float and grade as character. Field width specifiers can accompany conversion specifiers. Also whether the value is to be left justified or right justified can be mentioned. Following is the table, which gives all the conversion specifiers.

Conversion character	Meaning
c	Data item is a single character
d	Data item is a decimal integer
e	Data item is a floating-point value
f	Data item is a floating-point value
g	Data item is a floating-point value
h	Data item is a short integer
i	Data item is a decimal, hexadecimal or octal integer

o	Data item is an octal integer
s	Data item is a string followed by a white space character (the null character \0 will automatically be added at the end)
u	Data item is an unsigned decimal integer
x	Data item is a hexadecimal integer
[…]	Data item is a string, which may include white space characters

Writing a 'C' program

- Statements.
- First 'C' program.
- Comments.

Now let's put above knowledge to use, by writing our first 'C' program.

Statements

Each instruction of a 'C' program is called a statement. Each statement has to have a ';' (semicolon) at the end. Semi colon is the program terminator.
More than one statement enclosed in brace brackets is called compound statement.
Compound statements are often referred as block of statements. If it has to be a compound statement, brace brackets around all the statements in the compound statement is compulsory. Brace bracket enclosing single statement is optional.

```
if (k < 7)
{
        printf ("\nit is less than 7" ) ;
        k = k + 1 ;                              compound statement
        j = k ;
}
if     (k > 7)
{
       /* Single statement with braces */
         printf ("\n it is greater than 7") ;
}
else
         /* Single statement without braces */
         printf ("it is less than 7") ;
```

(we will study if-else statements in next chapter)

We have acquired sufficient knowledge so as to write a simple 'C' program. The structure of any 'C' program is as follows.

```
Preprocessor directives
External variable declaration
main ( )
{
statement 1 ;
statement 2 ;
.
.
statement n ;
}
```

Let us write a small 'C' program to calculate simple interest.

```
/* program to calculate simple interest */
# include<stdio.h>
void main()
{
/* program to calculate simple interest */
int p, n ;
float r, si ;
p = 1000  ;
n =  3 ;
r =  8.5 ;
/* formula for simple interest */
si = p * n * r / 100 ;
printf("\n Simple interest = Rs %f",si) ;
}
```

You will type this program into an editor. Since we will use the Turbo C compiler, we will write the program in the Turbo 'C' editor using the 'New' submenu in the 'File' menu. Every "C" program should begin with main (); which is a function. Function 'main' should be followed by (). All the statements are enclosed in brace brackets. The statements between opening and closing braces are called the body of the program.
We have defined two variables 'p' and 'n' as integers and two variables 'r' and 'si' as float. We have assigned a value of 1000 to p', 3 to 'n' and 8.5 to 'r' using the assignment operator '='.
si = p * n * r /100 is an arithmetic expression. Here expression on right of '=' is solved, according to precedence and associativity and type conversion rules and resulting value is converted to float and assign to 's', since s is a floating point variable. In the end we have used printf function to display the value of 'si'.
The #include<stdio.h> statement is called the preprocessor directive. All preprocessor directives are written before 'main'. The file 'stdio.h' contains all information for using library function 'printf. Since we are using 'printf' in our program we will need to include it in the program. This is done by using the preprocessor directive #include'.

Comments

Comments are lines written by programmer to give additional information or logic of the program. A line enclosed within /* and */ is a comment. Any number of comment can be placed anywhere in the program. A comment can be split over more than one line.

```
/*  This is
a jazzy
comment */
```

Comment cannot be nested.

```
/* cal  of si / This is not allowed */*/
```

It should be kept in mind that comments are needed since if some other programmer has to work on our program or if we ourselves have to work on it after a long time, they will be helpful. Over commenting should be avoided.

Review Questions

1. What is the correct value to return to the operating system upon the successful completion of a program?

A. -1
B. 1
C. 0
D. Programs do not return a value.

2. What is the only function all C programs must contain?

A. start()
B. system()
C. main()
D. program()

3. What punctuation is used to signal the beginning and end of code blocks?

A. { }
B. -> and <-
C. BEGIN and END
D. (and)

4. What punctuation ends most lines of C code?

A. . (Dot)
B. ; (Semicolon)
C. : (Colon)
D. ' (Single quote)

5. Which of the following is a correct comment?

A. */ Comments */
B. ** Comment **
C. /* Comment */
D. { Comment }

Summary

- We have learnt the basic building blocks of 'C' programming language.
- There are basically four data-types in 'C': integer, character, float and double.
- Constants are values that do not change.
- Variables are values that change. We have seen how to declare, initialize and use variables in 'C' statements.
- Keywords are words whose meaning is known to the compiler.
- We have studied the various arithmetic operators, unary operators and assignment operators and their use.
- We have seen in what order the operators (in expression with different operators) be solved using the precedence and associativity rules.
- We have seen input-output function used for transfer of data, to and from computer

Programming Concepts using C

Chapter 2: Control Structures

Objectives

- Recognize the need for decision-making.
- Use logical expressions.
- Use conditional expressions.
- Write programs using if, if-else statement.
- Write programs using nested if-else statement.
- Write program using switch-case statement.

Introduction

In the last chapter we wrote a program, where all the statement was executed in the order in which they appeared i. e. serially, one after another. There was no conditional execution nor was there any selection to be made.
In this chapter we will see how we can use the control statement to accomplish selective execution using 'if', 'if – else' and switch-case statements. We will also introduce the 'break' statement.

Why Decision Control Statements?

In day-to-day life, most of the time we think according to changing condition. We take action based on some decision, which depend on certain conditions.
E.g. If I work hard, I will get promotion else I will lose my job. Thus decision is taken depending upon the condition.
In order to solve problem that are more realistic, much broader and more interesting, we need the facility of conditional and selective execution. Many programs require that a logical test be carried out at some particular point within the program. An action will then be carried out whose nature depends upon the outcome of logical test. This is known as conditional expression.
A special kind of conditional execution is one in which one group of statement is selected from several available groups. This is known as selection.

Logical Expressions

Before going to the actual decision control statements, let us study the basic logical expression which should be formed. Depending upon this logical expression, i. e. whether its result is true or false, further action has to be taken.

There are two logical operators

&&	And
||	Or

A unary negation operator

!	Not

Four relational operators

<	Less than
<=	Less than or equal to
>	Greater than
>=	Greater than or equal to

Two equality operators

==	Equal to
!=	Not equal to

Remember the difference

= = is used for comparison of two quantities and = is used for assignment

These operators may be used to form logical expressions. The operands may themselves be other types of expressions. The numeric value of logical expression is 1 if the relation is true and 0 if the relation is false.

Expression using relational and equality operators are shown below

```
count <= 100
sqrt (a + b + c) > 0.0005
answer = = 0
letter ! = 'x'
ch <'T'
```

All relational operators have same precedence followed by equality operators and after that are logical operators. Relational operators have lower precedence than arithmetic operators.
Value of count < = 100 is 1, if count has value which is less than or equal to 100.
If count has a value greater than 100, this expression evaluates to false.
Value of ch <'T' is 1, if ch holds a character whose value is less than 'T' and it evaluates to false if ch has a value greater than 'T'.
Expressions connected with logical operators are evaluated from left to right and evaluation stops as truth or falsehood is known.
The truth tables for logical operators can be given as follows:

For '&&' (AND operator)

Operand 1	Operand 2	Output
False	False	False
True	False	False
False	True	False
True	True	True

AND Operator equals true if both its operands evaluates to true.

For '||' (OR operator)

Operand 1	Operand 2	Output
False	False	False
False	True	True
True	False	True
True	True	True

OR operator evaluates to true if either of its operand evaluates to true, else it equals false.

For ! (Negation operator)

False	True
True	False

The unary negation operator (!) converts a non-zero operand into 1.
E.g.

```
(! valid)
Instead of (valid = = 0), where valid is any type of variable.
The following expression using logical operators
( (count <= 100) && (ch ! =  *) )
is 1, if count is less than or equal to 100 and at the same time ch does not contain a '*'.
```

Types of Decision Control Statements

- if statement
- if-else statement
- switch –case statement

There are basically 3 types of decision control statements.

The 'if' statement

The format for if statement is

```
if (expression)
Statement
```

This is the simplest form of 'if' statement. The expression is to be placed in parenthesis. It can be any logical expression. The statement can be either a simple statement or a compound statement. The statement following the "if clause" constitutes the if-block.
If the expression given in brackets evaluates to true (1), the statement is executed. If it evaluates to false the statement is skipped and execution continues after the "if block".
Consider a simple example showing use of the if statement,

Example (1)

```
/* program to demonstrate use of if-statement
 Body of if-statement is executed only if expression results into true */
#include<stdio.h>
main ( )
{
      int j ;
      printf ("\n Enter a number           ") ;
      scanf ("%d",&j) ;
      printf ("\n The number you entered is %d", j) ;

                              if (j>100)
        printf("\n  Ah! you think too big,");
}
```

Program accepts a number from the user and output the number entered. (j>100) is a logical expression which uses a variable operand j (whose value can be any integer entered by user) and a constant 100 as its second operand. The greater than (>) operator (relational operator) connects the two operands to form a logical expression. It evaluates to true only if number j entered by user is greater than 100. If the number entered is greater than 100, the message is printed; if not then the statement is skipped.

If-else Statement

In the "if" statement seen earlier, we can take some action if expression is true. But if expression is false there is no action. We can include an action for both conditions (i.e. true or false) by using if-else statement. The general form which is

```
if (expression)
        statement 1;
else
        statement 2;
```

If expression evaluates to true, statement 1 is executed, while if expression evaluates to false, statement 2 is executed. Statement 1 and statement 2 can again be single statement or compound statements.

The following program illustrates the use of if-else statement. Example (2)

```
/* program to demonstrate if-else statement
   program accepts a character from user and informs whether it is a vowel or a consonant. */

# include<stdio.h>
main ( )
{
      char ch;
      printf ("\nEnter any character (a to z)     ") ;
      scanf ("%c",&ch) ;
      if    ( (ch == 'a' ) || (ch == 'e') || (ch == 'i') || (ch == 'o ')|| (ch == 'u') )
              printf ("\nThis is a vowel") ;
      else
              printf ("\nThis is a consonant ") ;
}
```

Program accepts a character from user and informs whether it is a vowel or a consonant.
The logical expression compares the entered character (ch) with each vowel to see if it matches with any one of them. It uses the equality operator (==) for comparison and OR (||) operator for checking with all vowels. If ch matches with any vowel, expression is true and 'if' block is executed. If it does not match with any vowel, expression is false and 'else' block is executed.

Nested if-else

It is possible to nest if-else statement within one another. An if-else statement can occur within another if-else statement. The inner if-else is said to be nested in the outer if-else. Nesting can go upto any level. There are several forms that nested if-else statement can take.

Form 1

```
if (e1)
 {
        if (e2)
                s1;
        else
                s2;
  }
  else
  {
         if (e3)
                 s3;
        else
                s4;
   }
```

Here e1, e2, e3 are expressions and s1, s2, s3, s4 represent statements. One complete if-else will be executed if e1 is true and another complete if-else will be executed if e1 is false.

Form 2

```
 if (e1)
        s1;
  else {
  if (e2)
      s2; }
```

Form 3

```
 if e1
       s1;
  else
{
        if e2
              s2;
       else
              s3;
   }
```

Form 4

```
 if e1
 {
    if e2
    else
         s2;
 }
 else
      s3;
```

Form 5

```
if e1
  if e2
       s1;
   else
       s2;
```

In the 5th form, to which 'if' does the 'else' belong to? It belongs to e2. The general rule is, **'else' clause is always associated with the closest, preceding, unmatched (else-less) 'if'**. If we want to associate 'else' with 'if' of e1, we can write.

```
if e1
{
   if e2
 }
else
   s2;
```

The brace brackets change the pair of else. If we stretch Example (2) to include digits as well as space character, newline character, the program can be written as

Example(3)

```
/* program to show nesting of if-else statement */
# include<stdio.h>
main ( )
{
        char ch;
        printf("\nEnter any character (a to z) ") ;
        scanf("%c",&ch) ;
         if ((ch == 'a') || (ch == 'e' ) || (ch == 'I') || (ch == 'o') || (ch == 'u') )
                    printf("\nThis is a vowel") ;
        else
                                        if ((ch - 'o') > 0 && ((ch - 'o') < 9) )
                     printf ("\nThis is a digit") ;
        else
                                         if ( (ch == '  '))
                      printf ("\nThis is  a space") ;
        else
                                  if ( ( ch == '\n'))
                      printf ("\nThis is a newline") ;
        else
                                  if ( (ch == '\t'))
                        printf ("\nThis is a tab");
        else
                        printf ("\nCharacter not present in our set") ;
    }
```

It is of form (3) format. There is no limit on how deeply the 'if's and the 'else's can be nested.

switch - case

Another form of statement available for selective execution is the switch statement. It causes particular group of statements to be selected from several available group. The general format is as follows.

switch (expression)
        Statement;

Where statement consist of one more case statements followed by a colon and group of statements.

```
switch (expression)
{
            case expression 1 :    statement 1;
                                   statement 2;

                                   statement n;
                                   break;

             case expression 2 :   statement 1;
                                    statement 2;

                                    statement n;
                                    break;

               default :
                                    statement 1;
                                    statement 2;

                                    statement n;   }
```

The expression should result in an integer value or character value. First the expression following switch is solved.
The result is then matched with each case expression (i.e. expression1, expression2….expression n).
The statement following the matching case expressions are executed. If no values of case expression match with value of switch expression then statement following default label are executed. Else control is transferred directly to the statement that follows switch statement.
Consider the following program. Default is used to handle errors; cases which may pop up unexpectedly (like user entering value which may not be listed in our case expressions)

The expression should result in an integer value or character value.
First the expression following switch is solved. The result is then matched with each case expression (i.e. expression1, expression2….expression m).
The statement following the matching case expressions are executed. If no value of case expression matches with value of switch expression then statement following default label are executed. Else control is transferred directly to the statement that follows switch statement. Consider the following program. Default is used to handle errors; cases which may pop up unexpectedly (like user entering value which may not be listed in our case expressions)

```
#include<stdio.h>
void main ( )
{
   int j = 2;
     switch ( j )
      {
          case 1 :
                        printf ("\nI am in case 1.");
          case 2:
                        printf ("\nI am in case 2.");
          case 3:
                        printf ("\nI am in case 3.");
          default:
                        printf ("\nI am in default.");
        }
}
```

The output will be
I am in case 2.
I am in case 3.
I am in default.

Since j = 2 and it matches with case 2. We would expected the output to be
I am in case 2.
But it is not the thing. Thus switch executes the case where a match is found and all the subsequent cases and default as well. This is called the fall through. Fall through is helpful if we want the same action for number of values. But if we want switch to exist when action for the match is carried out, we have to use a break statement as follows.

Example(4)

```
/* program to demonstrate use of switch- case and need of break in switch statements */
#include<stdio.h>
main ( )
{
int j = 2;
switch( j )
   {
  case 1:
        printf ("\nI am in case 1.");
        break;
 case 2:
        printf ("\nI am in case 2.");
        break;
 case 3:
        printf ("\nI am in case 3.");
        break;
 default:
        printf ("\nI am in default.");
        break;

  }
}
```

Break causes exit from the switch statement. Hence above program gives output as “I am in case 2”.
Let us rewrite Example (3) using switch statement

Example(5)

```
/* program demonstrates use of switch-case statement. */
/* Using switch- case instead of nested if-else*/
#include<stdio.h>
void main ( )
{
  char ch;
  printf ("\nEnter a character     ");
  scanf ("%c",&ch);
  switch(ch)
     {
      case  'a' :
      case  'e' :
      case   'I' :
      case  'o' :
      case  'u' :  printf ("\nThis is a vowel");
/*we need to print the same sentence for all the vowels */
       break;
       case  '1' :
       case  '2' :
       case  '3' :
       case  '4' :
       case  '5' :
       case  '6' :
       case  '7' :
       case  '8' :
       case  '9' :     printf ("\nThis is a digit");
                        break;
```

```
      case  ' '  :     printf ("\nThis is a space");
                        break;
      case  '\n' :     printf ("\nThis is a newline");
                        break;
      case  '\t'  :    printf ("\nThis is a tab");
                        break;
      default :        printf ("\nCharacter not present in the set");
   }
}
```

Switch can be replacement for if-else statement. But it does not provide facility for logical expressions. i.e. Expression like (count>1) or (count<10) are not allowed as case expressions. The expression should result in a specific value not range of values. Hence we have to replace the expression ((ch –'0') > 0) && ((ch –'0') <9)) by

Case 1: case2: …case 9:
Example (5) shows the use of fall through and also where break should be use.

Conditional Expression

If we write a simple program of finding maximum of two numbers, using if-else, we can write

```
#include<stdio.h>
main ( )
{
    int max, a =10, b=20;
       if (a > b)
           max = a;
      else
            max = b;
    printf("\nMax = %d", max);
}
```

The above program can be replaced by a conditional expression (a > b)? a: b; . The general form of a conditional expression is

Expression1? Expression2: Expression3;

Here Expression1 is evaluated first. If it is true then Expression2 is evaluated and that is the value of the expression. If it is false, Expression3 is evaluated and gives value of conditional expression. Thus in our example, if a is greater than b then value of expression is a, else value of expression is b. Conditional expressions can appear on right hand side of a simple assignment. The resulting value is assigned to the variable on the left.
E.g. max = (a > b)? a: b;

Apart from arithmetic statements, conditional operators can be used in other places as well.
E.g. (j = = 1 ? printf("Anyone"): printf("No one"));
Conditional operators can be nested
E.g. int big, a,b,c;
   big = (a>b ? (a >c ? 3 : 4) : (b > c ? 6 :8));

If operands (i.e. Expression2 and Expression3) differ in type, then the resulting data type of conditional expression will be determined by conversion rules stated in previous chapter.
E.g.

```
(n >0) ? f  : n;
```

If f is a float and n is an integer then the result is of type float. The limitation of conditional operators is that only one statement can appear after '?' or after '.'.

Indentation

Indentation is alignment of statements in 'C' program to help improve readability. Indentation has no effect on program execution. It is for the programmer to read and understand programs easily. For example, the statement in the "if block" is written by leaving a few columns.

If we change Example (1) and write it as

```
# include<stdio.h>
main ( )
{
      int j;
      printf ("\n Enter a number") ;
      scanf ("%d" , &j) ;
      printf("\n The number you entered is %d", j) ;
        if(j > 100)
{
                printf ("you think too big!");
                 j = j * 2;
                printf("%d is double the entered number", j) ;

 }
   }
```

Now if we remove the braces of compound statement & indent them as shown below.

```
If (j  >  100)
   printf   (              …       ) ;
   j  =    j   *  2 ;
   printf    (              …       ) ;
```

'C' will consider if statement as containing only one printf statement. j = j * 2 followed by printf will be executed irrespective of j being greater than 100. Since we have indented all three statements, it does not mean 'C' will interpret them as a compound statement.

Review Questions

1. Which of the following is true?
A. if(1)
B. if(66)
C. if(.1)
D. if(-1)
E. All of the above

2. Which of the following is the Boolean operator for logical-and?
A. &
B. &&
C. |
D. |&

3. Evaluate! (1 && ! (0 || 1)).
A. True
B. False
C. Unevaluatable

4. Which of the following shows the correct syntax for “if” statement?
A. if expression
B. if{ expression}
C. if(expression)
D. expression if

Summary

- In order to solve problem that are more realistic, much broader and more interesting we need the facility of conditional and selective execution.
- There are four types of statement available for the purpose. If, if-else, nested if-else and switch statements.
- Depending upon the value of the logical expression the selection of action is done.
- Expressions connected with logical operators are evaluated from left to right and evaluation stops as truth or falsehood is known.
- Indentation is alignment of statement in 'C' program to help improve readability. Indentation has no effect on program execution.

Programming Concepts using C

Chapter 3: Iteration

Objectives

- Identify the need for using loop control statements.
- Discuss the various method of looping
- while loop
- do-while loop.
- for loop
- Use various looping statements.
- Use continue, break statements.

Introduction

In the programs seen so far, execution was carried out either serially or selection of statements was done depending upon the result of certain conditions. In either case, any statement was executed only once. If we want to perform any action repeatedly, it can be done in 'C' using the loop control statements.

In this chapter we will study different types of loop control structures .The different types of loop are while loop, for loop and do-while loops. We will also introduced the continue statement which is used in relation to loop control statement. We will also have a look at goto-label statement.

Why loop control structures

Using only sequential execution and conditional execution set a limit to the range of problems that can be solved. Even in real life we need to repeat action for a specified number of times .

E.g.
1) We need to recite a poem again and again if we want to learn it by-heart.

2) If we have purchased an item on installment basis, period ranging for one year. We need to pay the installments each month (total 12 times).

Loop control structure helps us achieve the same.

Methods of looping

- While loop
- For loop
- Do-while loop

There are 3 methods using which we can repeat any part of the program. The repetitions may be for a fixed number of times as in example (2) or it may be until a certain condition is reached as in example (1)

While loop

- Using while loop

The general form of the while-statement is

```
while (expression)
Statement
```

The expression following while should be enclosed in round brackets. It is usually a logical expression, which results in true (1) or false (0). The statement following while forms the body of the while loops. Statement can be either a single statement or a compound statement.

The statements are executed as long as the logical expression is true. When the result is false, execution starts at the statement following while loop. There should be a statement in the body of the while loop that will eventually cause expression to become false. When there are no such statements, while loop will be executed forever. This should be avoided as it will cause the program to get stuck.

Using while loop
Example (1)

```
/* program demonstrates the use of while loop */
```

```
/* It accepts the radius from an user and calculates the area of circle as long as user enters 'y'
when asked whether to calculate area */
//assuming value of pi is 3
#define PI 3
#include<stdio.h>
void main ( )
{
char ans;
int r,area;
printf("\n Calculate area of circle? (Enter y or Y)");
scanf("%c", &ans);
     while ((ans == 'y') || (ans == 'Y')) /* while loop continues as long as condition is true */
          {
             printf("\n Enter radius of circle  ");
             scanf("%d", &r);
             area = PI * r * r ;
             printf("\nAREA = %d ", area);
             printf("\n Calculate area of circle? (Enter y or Y)");
             scanf("%c",&ans);
          }
}
```

In this program area of a circle is calculated by getting radius from user, as long as value of variable, ans is 'y' or ' Y '. Once ans has a value other than 'y'or'Y', execution starts from statement after while loop. Expression is evaluated each time to get the result.

For loop

- Using for loop

This is the most commonly used looping statement in 'C'. The general form of the statement is

```
For (Expression1; Expression2; Expression3)
Statement;
```

It contains 3 expressions. Expression1 is used to initialize some parameter called the index or loop counter that controls the looping action. It is an assignment expression.
Expression2 is the condition that must result to true to continue execution in for loop. It is a logical expression.
Expression3 is used to alter value of index. It is a unary or assignment expression.

Using for loop

Following is a simple example of for loop Example (2)

```
/* program to demonstrate use of for loop */
#include<stdio.h>
main ( )
{
        //j is used as index of for loop.
        int j ;
        for ( j  = 0; j <= 10 ; j ++)
            printf ("\n value of j = %d", j) ;
}
```

Expression1 is (j =0) (Initialization)
Expression2 is (j <= 10) (condition)

Expression 3 is (j ++) (Incrementing)

Value of j is printed as j runs from 0 to 9.

When for loop is executed for the first time j is initialized to 0. Condition (j <= 10) is checked. If it is true printf statement is executed.

At the end j is incremented by 1, with j ++.
The other alternative being j = j = 1 and j += 1.
Again condition j <= 10 is checked. If it is true, printf is executed. This repeats until condition j <= 10, result in false. Here j is the index. Index can be incremented, or decremented by any value (1 is not compulsory).
If it is decremented, initialization of index should be done accordingly.
1st and 3rd expression is omitted, if other statements are provided for initializing the index and\or altering the index.
If the second expression is omitted, it will be assumed to have value of 1 and loop will continue indefinitely unless terminated by break or return statement.
Any of the three expressions can be omitted, but the semicolons must remain.

J = 0 ;
for (; j <= 10 ; j ++)
{
 	
}
	for (j = 0; j<= 10 ;)
	{
	j ++;
	
	}
		for (; ;)
		{
		/*infinite
		loop */
		}

The same program using while loop will have the following structure
Expression 1;

```
while (Expression2)
{  statement;
Expression 3;
}
j = 0;
while (j <= 10)
{
printf ("\n%d", j);
j ++;
}
```

do-while

- Using do-while loop

do-while is a form of while loop. The general format is

```
do
Statements
while (expression);
```

In while and for loop conditional expression is evaluated at the beginning of loop, whereas in do-while loop expression is evaluated at the end of the loop. This causes execution of statements in the loop at least once. Once the loop statements are executed expression is evaluated and depending on its result further action is taken (which is the similar to while loop).
Thus do-while loop is used in places where statements are to be executed at least once irrespective of the conditional expression.

Using do-while loop

Example (1) can be rewritten using do-while. This will help to understand the difference between while and do-while clearly. Example(3)

```
/* program to demonstrate use of do-while loop. Here terminating
condition is checked at the end of loop */
#define PI 3
#include<stdio.h>
#include<conio.h>
main ( )
{           char ans;
            int r, area;
            do
             {
                printf ("\n Enter radius of circle");
                scanf ("%d", &r);
                area = PI * r * r;
                printf ("AREA = %d", area);
                fflush (stdin);
                printf ("\n Calculate area of circle? (Enter Y or N)");
                ans = getche();
              }
             while((ans == 'y' ) || (ans =='Y'));
}
```

Here area of circle is calculated at least for one circle.'

*fflush() – The reason for using the function fflush() is that when we use scanf() we accept data from user, the data along with the "enter key" goes into the key board buffer. The scanf() assigns data to the given variable but keeps the 'enter key' in the buffer. When the user is prompted for Y/N, this enter key will be provided as a response since getche() requires only one character. To avoid this, fflush() is used. The fflush() flushes out all the remains in the i/p buffer and accepts a fresh input from the user. The parameter passed to fflush() is the buffer we want to flush out. Here stdin() is the buffer related with standard input device the keyboard.

Use while loop or use for loop

Using while or for loop can be a personal choice. But still there is a point to be noted. While-loop is generally used in places where number of iteration are not known and statements are to be executed until a condition is satisfied, though it can be effectively used when number of iterations are known. For-statement is generally used where it is a simple initialization, testing and incrementing values.

Comma operator

Comma operator is used in for statement. If we want to initialize two or more related indices at the same time and if we want to increment or decrement more than one variable, we can use the comma operator in for loop.

```
for (expression1a, expression1b ; expression 2; expression 3)
```

Here expression1a and expression 1b are two expressions separated by comma operator. They will initialize two separate indices.

```
for (expression1; expression2; expression 3a, expression 3b)
```

Here expression3a and expression 3b are separated by comma operator. They are used to alter two different indices used simultaneously within the loop.

Only one expression is allowed in the test expression.
E.g.

```
for (j = 0, k = 3; j  < k ; j = j + 2, k++)
{
}
```

Nested loops

A loop can occur within another loop. The inner loop is said to be nested in the outer loop. We have seen nesting of decision control statement) i.e. if-else statement). Similarly while and for loops can be nested within one another. Nesting can go upto any level. Each level should be controlled by different index. It is obvious that the index should be relevant to each other.

Example (4)

```
/* program to demonstrate use of nested loops */
/* one for-loop nested within another for-loop */
#include<stdio.h>
main ( )
{
  int r, c, sum;
  for (r = 1; r < 3; r++ )/* outer loop */
   {
     for (c = 1; c  <= 2; c++)    /* inner loop */
          {
                sum = r + c;
                printf ("\n r = %d  c = %d  sum = %d", r,  c, sum);
            }
   }
}
```

Output of above program is
r = 1 c = 1 sum = 2
r = 1 c = 2 sum = 3
r = 2 c = 1 sum = 3
r = 2 c = 2 sum = 4

For each value of r, the inner loop is executed two times with c running from 1 to 2 and terminates when c exceeds 2. Value of r ranges from 1 to 3 and terminates when r exceeds 3.
It is necessary that one loop should be completely embedded within another loop. There should be no loop overlapping.

Continue and break statements

We have seen the break statement in relation to the switch-case statements. Similarly we can use break statements in the while and for loops to exit the loop before the predetermined condition. It can be used to handle error or any exceptional condition.

E g.

```
for (count = 0; count <= n; ++count)
{
    . .
    while ((c = getchar ( ) ) ! = '\n')
    {
                if (c = = '*' ) break;
    }
 }
```

The above program skeleton shows the use of break. '\n' character entered as input is the terminating condition for inner while loop. But if character entered is '*', break statement causes exit from the while loop.

Break causes exit only from the loop in which it is used. i. e. in our program though while is nested in **for, break, exit** only while loop, causing execution to resume from statement following while.

There is another statement known as continue statement, which is used to skip remainder of current run through the loop and continue execution of loop from the beginning with the next value of index.

```
E .g.
for (j = 0; j <= n; j++)
{
     if (a [j] < 0)      /*skip negative elements */
        continue;
 .  .
/*  process positive elements */
 .  .
}
```

The above example illustrates use of continue statement. j runs from 0 to n. if a[2] is less than 0, then continue statement causes remaining body of for loop to be skipped and execution again begins from for statement i.e. j = 3. i.e. all values in the array, which are negative, are skipped.

Review Questions

1. What is the final value of x when the code int x; for(x=0; x<10; x++) {} is run?
A. 10
B. 9
C. 0
D. 1

2. Which is not a loop structure?
A. For
B. Do while
C. While
D. Repeat Until

3. How many times is a do while loop guaranteed to loop?
A. 0
B. Infinitely
C. 1
D. Variable

4. When does the code block following while(x<100) execute?
A. When x is less than one hundred
B. When x is greater than one hundred
C. When x is equal to one hundred
D. While it wishes

Summary

- We can repeat a block of statement by using for, while and do-while loops which allow us to do things repeatedly each in its own way.
- The for and while loops checks if terminating condition has reached at the beginning of the loop.
- The do-while loop checks for terminating condition at the end of the loop.
- We have also seen how break cause exit out of a loop.
- The continue statement helps working within the loop, while skipping current iteration

Programming Concepts using C

Chapter 4: Preprocessor

Objectives

- State preprocessor.
- Discuss the different preprocessor directives.
- Use the preprocessor directives.

Introduction

This topic may seem is a bit offline. But we have already used preprocessor directives without any explanation about what it is. We will study the preprocessor directives in detail here.

A preprocessor is a program that processes our program before it is passed to the compiler for compilation. The commands of a preprocessor are called preprocessor directives. Each preprocessor directive begins with a # symbol.

Preprocessor directives can be placed anywhere in the 'C' program. But are generally placed at the beginning of the program before main(). They can be recognized as 'C' program statements starting with '#'. Preprocessor directives are not terminated with a semicolon.

Preprocessor Directives

- Macro expansion
- File inclusion
- Conditional compilation
- Miscellaneous directives

Preprocessor directives can be categorized into four types. Macro expansion, file inclusion, conditional compilation and miscellaneous directives. We have used the #include directive in our examples. It falls into the category of file inclusion directive.

Macro expansion

- Simple macros.
- Macro with arguments.
- Why use macros?

There can be two types of macros one which has no arguments; they are called as simple macros and one which takes arguments. Arguments are values passed to the macros so that the same macro can be used over a wide range of values. Later on we will also discuss how macros are useful.

Simple macro

The definition has the following form

```
#define name replacement text
```

name = it is called the macro template.
replacement text = it is called the macro expansion.
The rules for giving a macro name are same as for variable name. Generally macro name is written with capital letters.E.g.

Macro name

```
#include<stdio.h>
#define UPPER 25
main ( )
{
int  j;
for (j = 1; j <= UPPER; j++ )
 {
      printf ("\n%d", j) ;
  }
}
```

The program is first passed to the preprocessor where it sees if macros are present. It replaces all occurrences of UPPER in the program with 25. Substitutions are only done for separate whole names, not names within double quotes or if name is part of another name. i.e. replacement won't take place for "UPPER" or UPPERCASE.

Macro with arguments

Macros can also be used with argument so that we can obtain different replacement text for different values of arguments.
E.g.

```
#define AREA(x)  (3 * x * x)
#include<stdio.h>
void main ( )
{
    int r1 =6, r2=2,a;
    a=AREA(r1);
        printf("\n Area of circle = %d",a);
    a = AREA(r2);
         printf("\n Area of circle = %d",a);
}
```

Here all occurrences of AREA are replaced by the replacement text.i.e.
a = AREA (r1) is replaced by
a = (3 * r1 * r1), x is replaced by r1 and also
a = AREA (r2) is replaced by
a = (3 * r2 * r2), X is replaced by r2.

No blank should be left between the macro template and its argument. The entire macro expansion should be enclosed in brackets.

Why use macros?

If relevant and suitable names are given then macros are used to increase the readability. Secondly, more important thing is if we decide to change 25 to 40 in our example and if we don't use macro, we will have to change 25 to 40 at all places in the program. When macro is used, change has to be made at only one place.

UPPER is also called as symbolic constant, since value 25 remains constant throughout the program
Other example of simple macros can be
#define directive is used to replace a 'C' statement.
#defines FOREVER for (: :)
#define FOUND printf ("The Yankee Doodle Virus")
#define is used to replace a operator and a condition
#define AND &&
#define ARRANGE (a>25 AND a<50)
The macro expansion can also contain a macro template.

File inclusion

We have used
#include directive in programs written so far
#include filename
The contents of the file given by filename are replaced at the point where the directive is written.
If we have written number of function for specific purpose and if we write programs that use them, then declarations of these functions, macros used, declaration of external variable can all be combined

in the header file. Instead of repeating the above-mentioned statement in each program that uses them, we can include the header file in our programs using the file inclusion directive.

The header file stdio.h supplied with 'C' contains function declarations and all the information regarding input and output. Hence when we use I/O function in our programs we include this header file.

The file inclusion statement can be written in two ways
#include "file-name"
#include <file-name>
If we use the first way, 'C' would search for the file; filename in the current working directory as well as in the specified list of directories.
If we use the second way, the file; filename will be searched only in the specified list of directories.

Conditional compilation

A number of the directives control conditional compilation, which allows certain portions of a program to be selectively compiled or ignored depending upon specified conditions. The directives concerned are: #if, #ifdef, #ifndef together with the preprocessor unary operator defined.

The way that they are used is like this:

```
#ifdef  NAME
/* compile these lines if NAME is defined */
#endif
#ifndef NAME
/* compile these lines if NAME is not defined */
#else
/* compile these lines if NAME is defined */
#endif
```

The #else can be used with #ifdef (and #if or #elif) too. There is no ambiguity about what a given #else binds to, because the use of #endif to delimit the scope of these directives eliminates any possible ambiguity. The Standard specifies that at least eight levels of nesting of conditional directives must be supported, but in practice there is not likely to be any real limit.

These directives are most commonly used to select small fragments of C that are machine specific (when it is not possible to make the whole program completely machine independent), or sometimes to select different algorithms depending on the need to make trade-offs.

The #if and #elif constructs take a single integral constant expression as their arguments. Preprocessor integral constant expressions are the same as other integral constant expressions except that they must not contain cast operators. The token sequence that makes up the constant expression undergoes macro replacement, except that names prefixed by defined are not expanded. In this context, the expression defined NAME or defined (NAME) evaluates to 1 if NAME is currently defined, 0 if it is not. Any other identifiers in the expression including those that are C keywords are replaced with the value 0. Then the expression is evaluated. The replacement even of keywords means that sizeof() can't be used in these expressions to get the result that you would normally expect.

```
if , #elif, #else, #endif
#ifdef, #ifndef
```

We can select whether certain lines of code are to be compiled or not using the conditional compilation directives.

```
#if, #elif, #else, #endif
```

The if directive test the expression following it and depending upon the result of the expression decides whether to compile statements.

```
#if  TEST <= 5
      statement  1;
      statement  2;
#else
       statement  3;
       statement  4;
#endif
```

If we have number of conditions to check, instead of using #else or #if number of times we can use #elif if we want to compile code depending upon the adapter used in our machine.

```
#if   ADAPTER = = MA
     code for monochrome display.
#if   ADAPTER = = CGA
     code for color graphics adapter.
#if    ADAPTER = =EGA
      code for enhanced graphics adapter.
#if    ADAPTER = = VGA
       code for video graphics adapter.
#else
        code for super video graphics adapter.
#endif

#ifdef,  #ifndef
#ifdef, directive checks if a macro is defined.
#ifdef   HDR
    statement  1;
       . .
   #endif
```

Thus certain code can be compiled only if macro HDR is defined. We will require such a macro if for example we are not sure that code has to be deleted or may be required at some later time. Don't define HDR and write the above conditional directive for your code. It won't be compiled. Later if you become sure that you don't need the statements, you delete them. If you need them remove the #ifdef directive or define the macro HDR.

```
#ifndef is exactly opposite to #ifdef
#ifndef HDR
statement  1
 . .
 . .
#endif
```

Compiles statement following it upto #endif if HDR is not defined

Miscellaneous directives

- #undef
- #pragma

We have few more directives, which do not fit in the above categories, hence described separately. #udef is the directive which is used in relation to the #define directive and #pragma is the directive which is just introduced here since it is a specialized directive, used very rarely.

#undef

In order to undefined a defined macro we can use the #undef directive.
#undef macro name
if we have defined
#define UPPER 25
at some place in the program, we can remove it from the system by using
#undef UPPER
E.g.

```
#define   UPPER 25
main ( )
{
. .
. .
#undef UPPER
}
```

#pragma

These are special purpose directives that you can use to turn on or off certain features.
There are different pragmas for different compilers. Turbo C compiler has got a pragma, which allows us to write assembly language statements in 'C' program.

Preprocessor Operators

Preprocessor provides us with an operator, which is called as defined operator.

defined

#if is sometimes used along with defined operator which is used to verify if a symbolic constant is defined or not.
defined(macro name)

returns 1 if macro is defined.
Returns 0 if macro is not defined.

E.g.

```
#if  defined (FOREGRROUND)
     #define BACKGROUND  0
#else
      #define FOREGROUND  0
      #define BACKGROUND  7
#endif
```

Review Questions

1. State True or False
   Preprocessor can be considered as a language, which processes the 'C' statement before they are passed to the compiler.

   A. True

   B. False

1. Will #define text "find mistake" give an error since macro name is not capitalized?

   A. YES

   B. No

2. What is the difference between #include<file-name>and #include "file-name"?

   A. #include<file-name> and # include "file-name" would searched for the file only in the specified list of directories

   B. #include<file-name> and #include"file-name" are same

   C. In case of #include "file-name" 'C' would search for the file; filename in the current working directory as well as in the specified list of directories. In case of #include <file-name>, the file; filename will be searched only in the specified list of directories.

3. What is the output of following code?

```
#define EXMP 25
main ( )
{    int exmp = 35;
     printf ("\nvalue of exmp is  %d",EXMP);

}
```

A. value of exmp is 25
B. value of exmp is 35
C. value of exmp is 2535
D. value of exmp is 3525

Summary

- Preprocessor can be considered as a language, which processes the 'C' statement before they are passed to the compiler.
- The macros cause replacement at the place where they are used, thus they increase size of program but reduce the execution time as compared to functions.(If used in the same place)
- File inclusion directives are used for including header files in the program.
- Directives for conditional compilation are available which help us to conditionally execute statements.

Programming Concepts using C

Chapter 5: Function

Objectives

- Explain basic concepts of functions
- Write functions
- Pass argument to function by using argument passing methods
- Call by value
- Call by reference
- Write recursive functions

Introduction

Until now we wrote programs in which all the code was written in main ( ). If we require to write large programs, it is always easy if the program can be broken down into small modules. This can be done using functions. The main ( ) itself is a function. Designing, writing and maintenance of programs can be done with simplicity and clarity by using different function instead of putting whole code in main function.

We have refered to library function in earlier chapters. While studying goto statements, we have said that they can pass control anywhere within the same function.

In this chapter we will see what functions are, how to define a function, call a function & different methods of passing parameters. We will also learn about recursive functions

Function basics

- What is the function
- Calling a function.
- Types of function.
- Why function is required.
- Function prototype.

A 'C' program can be considered as being a collection of functions. Here we will see what do we mean by a function and basically why do we need to write a function when we are already writing programs without it.

What is a function?

In order to know what a function is, let us write the program of adding two numbers and printing their sum, using functions.
Example(1):

```
/*    program to show what is a function  */
/*    program uses two functions add and display  */
#include<stdio.h>
int add (int x, int y);      /* function declaration* /
void display (int sum);
void main ( )                    / * calling function */
{
          int x, y, sum;
          x = 10;
          y = 20;
          sum = add (x, y);         / * called function * /
          display (sum);
}
/ * function definition * /
return  ---->  int add (int a, int b)
type            {                 ^   ^          argument declaration
                     int s;
                     s = a + b;
returning  ----->  return (s);
from a          }
function.
                void display (int s)
                {
```

```
        printf ("\n %d", s);
    }
```

Here we have broken down the program into two modules.
Adding two numbers.
Printing the result.

We wrote a function for each module, 'sum' and 'display' respectively. Any function definition has the form:

```
return-type    function-name (argument declaration)
{
declaration and statements
}
```

Function definition is the actual code of the function within braces. Whereas function declaration is a statement which tells the compiler that a function (e.g. sum), with these type of argument (e.g. two integers) and these type of return value(e.g. int) is used in this program.

Functions are self-contained block of statements. All variables used in a function are declared at the beginning of the function. In sum variable s is used and hence declared at the beginning of function.

Function definition is the actual code of the function within braces. Whereas function declaration is a statement which tells the compiler that a function (e.g. sum), with these type of argument (e.g. two integers) and these type of return value(e.g. int) is used in this program.

Functions are self-contained block of statements. All variables used in a function are declared at the beginning of the function. In sum variable s is used and hence declared at the beginning of function.

Calling a function

Let us consider the concept of calling a function. When a function calls another function, control passes to the called function and the calling function waits for the control to return to it. In Example (1), main ( ) is the **calling function** and 'sum' and 'display' are the **called functions**.

main( ) calls the function sum(). Control passes to sum and statements in sum are executed. When 'return' statement is encountered, control is returned to function main ( ). Further statement in main() start to execute. The main( ) calls function display().

Any function can call any other function any number of times. Compiler begins program execution with the function main ( ). Communication between function is made possible by passing argument to called function and argument by called functions.
Minimum requirement to recognize a function is a name with opening and closing round brackets.

```
Eg: dummy()
    {
     }
```

Empty round brackets after function name indicate that, no argument is passed to the called function.

Types of functions

There are two types of functions;

- Library functions.
- User-defined functions.

We have mentioned library function in the first chapter in relation to input-output functions. Library functions are built-in functions that are provided with 'C' compiler. There are library functions in 'C' which perform basic needful tasks. I/O functions like printf ( ), scanf ( ), string manipulation functions like strlen ( )and so on. For using library functions programmer need not worry about how it is done. Knowing only what is done is sufficient.

User-defined functions are functions the programmer writes as per his requirement. Programmer has to plan all the ins and outs of the function.

Why function is required ?

Imagine you have to add two integers number of times in your program. If you don't use the function sum(), it would keep repeating the same code that many times. Functions avoid repetition. Since 'sum' is a small function you might not mind repetition, but when you perform complicated operations, lengthy operations, avoiding repetition by using function is a plus point.

Also breaking into different tasks makes things simple then writing a whole single program. Functions once written can be used many times while writing other programs.

Function Prototype

The declaration

int sum(int x, int y)

just before main() in our previous example tells that sum is a function, used in main().
It accepts two integer arguments and returns an integer value. This declaration is called as function prototype.
By default, return value of a function is an integer. A function returning values other than integers (i.e. float, double etc.) must be explicitly mentioned. Functions returning no value have to be declared with return type 'void' as in function display().

Function declaration and definition must match otherwise compiler will give an error. Prototype is placed at the beginning of program, after any preprocessor commands. Parameter names in definition and declarations need not be same. Hence declaration of sum () has names 'x' and 'y' while definition has names 'a' and 'b' Rather, names are optional in prototype.

Communication between functions

- Parameter passing
- Returning values

Functions are so useful mainly because they can communicate with each other. They accept values from functions calling them and return values to the calling functions

Parameter passing

- Passing parameter by value
- Passing address by value

Communication between calling and called function can be made possible by using the argument mechanism. There are two terms related in this context. They are **'formal arguments'** and **'actual arguments'.**

In Example (1) main () calls sum() by passing two integer values to it. These values are called arguments. There names are 'x' and 'y'. Variable used in the statement of a function call are called actual arguments, x and y are actual arguments.

In the definition of sum we have used the names of arguments to be 'a' and 'b'. 'a' and 'b' are called formal arguments. Instead if using different variable names a and b if we used same names x and y compiler would still treat them as different variable because they are in different functions.
In 'C' parameter are passed by values. But you can either pass

1) Contents of variables by value.

OR

2) Address of variable by value.

Passing parameter by value

What happens when function is called by passing arguments? A temporary copy of the actual arguments is made to the formal arguments. The called function works on this copy of variables. The changes are made to this copy of variables. Hence original values of these variables remain intact with the calling function.

The change made by the called function is not seen by the calling function. This is called 'Call by value'. The following example clears the concept.

Example(2)

```
/* Program to demonstrate call by value
/* Value of arguments passed to function
/* Function exchgv, is called from main  .
          It exchange the values of a and b.*/
#include<stdio.h>
void exchgv (int x, int y);
main ( )
       {
         int a, b;
         a = 10;
         b = 20;
         printf ("\n In main before calling exchgv");
         printf ("\n a = %d b = %d", a, b);
         exchgv (a , b);
         printf ("\n In main after returning from exchgv");
         printf ("\na = %d  b = %d", a, b);
         }
        void exchgv (int x, int y)
          {
              int t;
              t = x;
              x = y;
              y = t;
             printf ("\n In exchgv");
             printf ("nx = %d   y = %d", x, y);
          }
```

The output of the above program would be

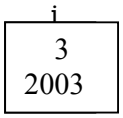
```
In main before calling exchgv
a = 10  b = 20
In exchgv
x = 20  y = 10
```

In main after returning from exchgv
a = 10 b = 20

The values of 'a' and 'b' remain unchanged even after values of 'x' and 'y' are exchanged. When a and b are passed to exchgv, value of a i.e. 10 is copied to x and value of b i.e. 20 is copied to y. Now the action of exchanging the values is done on x and y and not on a and b. When function returns value, a and b in main() are not changed.

Passing address by value

Before going further, let us make the concept of address and pointer clear. When a variable is defined, a memory location is set aside for the variable. This location has a location number which is called the 'address'. The symbol '&' (ampersand) is used to represent address. E.g. If we have a variable i, defined and initialized as follows
int i = 3;



location name
value of variable
address of an integer (location number)

A pointer is a variable which stores address of any data type. Symbol '*' is used to represent pointers.

E.g.

```
int  * j;
j  =  &i;
```

Means,
j is a pointer variable.
j stores the address of an integer.
Value of that integer is 3.
i.e. j = 2003. (i and *j are both equal to 3)
We will study pointers in detail in later chapter.
Let us rewrite Example(2) by passing address instead of values

Example(3)

```
/* program to demonstrate passing of address by value
 Address of arguments passed to function
/ * Function exchgr, is called from main
It exchange the address of a and b.
The change is seen in main */
#include<stdio.h>
void exchgr (int *, int *);
main ( )
        {
        int a, b;
        a = 10;
        b = 20;
        printf ("\nIn main before calling exchgr");
        printf ("\na = %d  b = %d", a, b);
        exchgr (&a, &b);
        printf ("\nIn main after returning from exchgr");
        printf ("\na  = %d  b = %d", a, b);
       }
    void exchgr (int *x,int *y)
        {
          int t;
```

```
        t =  *x;
        *x = *y;
        *y = t;
        printf("\n In exchgr - pass address by value");
        printf("\nx  = %d   y = %d", *x, *y);
}
```

The output of the above program would be

```
In main before calling exchgr
a = 10      b = 20
In exchgr - pass address of variable by value
 a = 20      b = 10
In main after returning from exchgr
 a = 20      b = 10
```

As can be seen from the output of the program when we pass address of variable the value of actual arguments can be altered and this alteration can be seen by the calling function. One more point to be noticed is that by passing address of variable we can make a function return more than one value at a time, indirectly. Exchgr has changed value of a and b both. These changes are available to main() hence main() has got two values from exchgr.

Returning values

- Calling function passes values to called function whereas called function returns value to a calling function.
- It is not necessary that a function should return a value.
- A function with no return statement causes control to the calling function when the ending right brace is encountered, but no value is returned as in function display() of Example(1)
- If the called function does not return a value, it should be specified with the keyword 'void' in the declaration and definition of function.
- A calling function can ignore the value returned by a function.
- There is no restriction on the number of return statement that may be present in a function.
- A function can return only one value at a time. Hence return(2,3) is an invalid return statement.

Recursion

- Writing a recursive function.
- Execution of recursive functions.

A function calling itself is called a recursive function i.e. some statement in a function body gives a call to the same function.

Writing a recursive function

The concept of recursion and recursive function will be made clear by studying the example of factorial. Let us write a function to get the factorial of a number. The factorial of a number is the product of all the integers between 1 and that number.
E.g. Factorial of 3, i.e. 3! = 3*2*1 = 6

Example(4)

```
/* program to find factorial of a number using recursion */
#include<stdio.h>
main( )
        {
```

```
        int a,fact(int), factor;
        printf ("\n Enter number :");
        scanf ("%d", &a);
        factor = fact(a);
        printf ("Factorial value = %d", factor);
      }
int fact(int  x)
 {
            int  f;
            if (x == 1)
                  return (1);
            else
                   f = x * fact (x - 1);
          return (f);
      }
```

The statement
f = x * fact (x – 1) in the function fact is a recursive call. i.e. fact calls fact itself. Only the value of arguments passes each time varies.

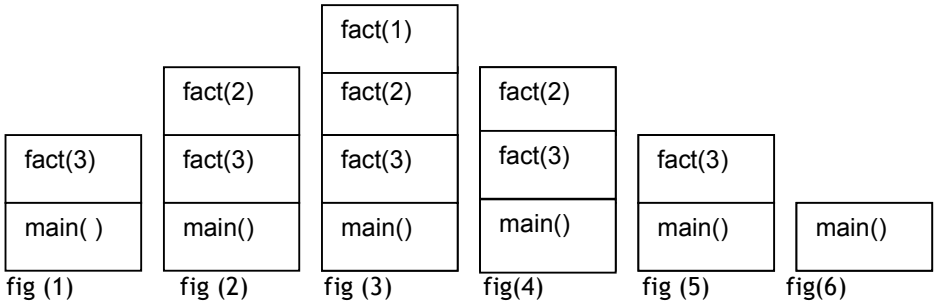
The function calls itself repeatedly. This has to end somewhere. When writing recursive functions you must have a terminating condition in the function, to return, in the absence of which the program will go into infinite loop of calling itself.
In the above example
if (x = =1) return(1);
is the terminating condition. When the function fact is called by passing value '1' as its argument it returns 1.

Execution of recursive functions

Let us see what happens on running this program
If the input given is 1:
The value of a = 1 is copied to x. The condition (x = = 1) is true hence '1' is returned to the function main and printed there.

'C' needs to keep track of where to continue execution after return statement is encountered. This is achieved by using a 'stack' which holds the functions and the location from where execution is to resume. Let us consider the above program by giving input value of 3. The figure showing flow of the program and the stack contents will help understand the program.

Stacks are memory locations. Stack works on LIFO (last in first out) technique. i.e. Contents entered last are removed first.
If the input given is 3.



When function main() is executing main is put in the stack. Fact is called from main with a value of 3. fact(3) is added to stack. Fig(1)

In function fact(3) value of x is equal to 3. (x = = 1) is false hence f = 3 * fact(3 – 1)is executed i.e. fact(2) is called from fact(3). Fact(2) gets added to stack. Fig(2)

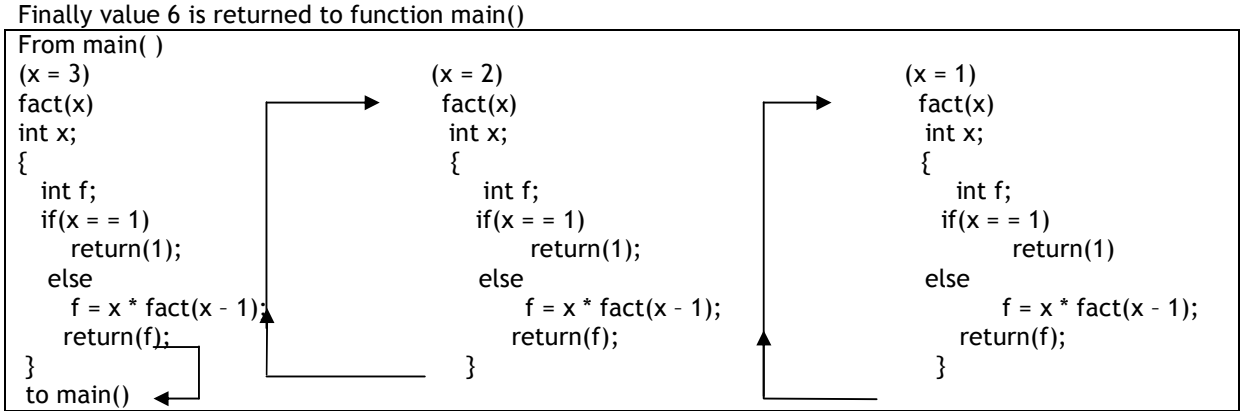
In function fact(2) value of x is equal to 2. (x = =1) is false hence f = 2 * fact(2 – 1) is executed i.e. fact(1) is called from fact(2). Fact(1) gets added to stack. Fig(3)

In function fact(1) value of x is equal to 1. (x = = 1) is true hence fact(1) returns value of 1. fact(1) is removed from stack. fig(4)

Now topmost function in stack is fact(2), so 'C' knows that it has to return to function fact(2) at the place it left i.e. at the statement f = 2 * fact(1). Here fact(1) is replaced by 1. Therefore f = 2. Next statements return(f) return value of 2. fact(2) is removed from stack. Fig(5)

Now topmost function in stack is fact(3). So 'C' returns to function fact(3) at the place it left i.e. fact(3)
at f = 3 * fact(2). fact(2) is replaced by 2. Hence f = 3 * 2 = 6. return(f) return value of 6. fact(3) is removed from stack. Fig(6)

Finally value 6 is returned to function main()



Review Questions

1. Which is not a proper prototype?
A. int funct(char x, char y);
B. double funct(char x)
C. void funct();
D. char x();

2. What is the return type of the function with prototype: "int func(char x, float v, double t);"
A. char
B. int
C. float
D. double

3. Which of the following is a valid function call (assuming the function exists)?
A. funct;
B. funct x, y;
C. funct();
D. int funct();

4. Which of the following is a complete function?
A. int funct();
B. int funct(int x) {return x=x+1;}
C. void funct(int) { printf("Hello");
D. void funct(x) { printf("Hello"); }

Summary

- Functions are self-contained block of statements.
- Functions avoid repetition.
- Communication between calling and called function can be made possible by using the argument mechanism.
- Arguments are of two types 'formal arguments' and 'actual arguments'
- Calling function passes values to called function, whereas called function returns value to a calling function.
- A function can return only one value at a time.
- A function calling itself is called a recursive function.

Programming Concepts using C

Chapter 6: Data types and Storage Classes

Objectives

- Discuss the datatypes in details.
- List the range of values each datatypes can accommodate
- Discuss the different storage classes

Introduction

In all the variables defined till now, definition included two parts, type of variable and name of variable. We did not consider the third part i.e. the storage class. Storage class, type of variable and name of variable together define a variable. Storage class states how and where the variable will be stored in memory and what would be the life of variable.

In this chapter we will study the different types of storage classes like Automatic, External, Static and Register. We will see the different data-types in detail.

Data-types

- character
- integer
- float
- double

There are basically four data types in 'C'

char	Character data-type
int	Integer data-type (fixed point data-type)
float	Single precision floating point data-type
double	Double precision floating point datatypes

Using data types helps compiler to set aside space in memory for the variable.

character

'Character' can be signed characters or unsigned characters. Unsigned characters are 8 bits i.e. value of unsigned character can be between 0to 2^8 i.e. 0 to 255. For signed characters the left-most bit is used to represent the sign. Hence the remaining 7 bits are used for value. Hence value can be between -2^7 to $+2^7$ i.e. -128 to +127.

integer

'Integers' can be short integers or long integers. These two provide different lengths of integers. Short is generally 16 bits and long is 32 bits. Integers can be either 16 or 32 bits. The general rule is short <= int <= long. The integers (short as well as long) can be signed or unsigned. Integers are used for fixed-point numbers. Which type of number is required can be decided before hand depending upon the size of data to be stored in the variable.

float

'Float' are used for floating point numbers. Float is a single precision floating point. Float occupies 4 bytes in memory.

double

'Double' which gives double precision floating point occupies 8 bytes. Even if 8 bytes are not sufficient, 'long double' which occupies 8 bytes can be used.

When we say
double a, b;
8 bytes for a and 8 bytes for b are set aside in memory and the range of values available also get fixed.

Table of datatypes

The following table gives a summary of all the datatypes, number of bytes assigned to each one and the range of value the variable of that type can take up.(AS per 32 bit machine)

Name	Description	Size*	Range*
char	Character or small integer.	1byte	signed: -128 to 127 unsigned: 0 to 255
short int (short)	Short Integer.	2bytes	signed: -32768 to 32767 unsigned: 0 to 65535
int	Integer.	4bytes	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
long int (long)	Long integer.	4bytes	signed: -2147483648 to 2147483647 unsigned: 0 to 4294967295
float	Floating point number.	4bytes	3.4e +/- 38 (7 digits)
double	Double precision floating point number.	8bytes	1.7e +/- 308 (15 digits)
long double	Long double precision floating point number.	8bytes	1.7e +/- 308 (15 digits)

Storage classes

- Automatic storage class.
- Static storage class.
- External storage class.
- Register storage class.

Every variable has a storage class, which may be specified explicitly or it may be implied by the place at which the variable is defined in the source code.

Storage class tells us:

1. Where the variable is stored? In memory? or In register?
2. What is the scope of the variable i.e. how far the variable can be accessed, what function can use it and so on.
3. The life of the variable. i.e. upto what extent a variable can retain its value.

The above points will be clear when we consider each storage class in detail.

Automatic storage class

They are called automatic or the local variables. Local variables are stored in memory. If a local variable is not initialized it will contain garbage value. Automatic variables are always declared within a function. The scope of an automatic variable is confined to the function in which they are declared or to the block of statement in which they are declared. The value of the variable is lost after control goes out of the block in which it is defined and is re-initialized when the block or function is re-entered.

Example(1)

```
/* program show the automatic storage class * /
/ * the output  helps us understand what is the scope of variable
k when defined in different places with automatic storage class
In function main  */
#include<stdio.h>
void main ( )
{
        auto int k = 1,j;
            for(j =0;j<1;j++)
        {
            auto int k = 2;
            printf ("\nInside for loop ");
            printf ("\nk = %d", k);
        }
        printf ("\nOutside for loop");
        printf ("\nk = %d", k);

}
```

The scope of variable 'k = 1', defined in main() outside for loop is the entire function main().

```
Inside for loop
k = 2
Outside for loop
k = 1
```

External storage class

External variable are defined outside of any function. The scope of external variable extends from the point at which it is defined to the end of the program. Scope of external variable is said to be global as against automatic variables. Even if external variable is not defined, its initial value is taken to be '0' by default. External variable is recognized by location of variable definition within the program.

Example(2):

```
/* program to show
external variable declaration and scope */
#include<stdio.h>
    int a = 2;      /*      External var definition.*/
    main( )
     {
          extern b;
```

```
        void corn(),play();
        printf("\nIn main b = %d",b);
        corn( );
        printf("\n In main");
        printf("\na = %d", a);
        play( );
    }
    int b;
    void corn( )
    {                                   /* storage class specifier*/
       extern int a;                        /* external var declaration */
       printf("\nin corn before incrementing a");
       printf("\na = %d", a);
       a++ ;
    }
    void play( )
     {
        int b = 10;
        printf ("\n In play");
        printf ("\nb = %d", b);
    }
```

Output

In main b=0
in corn before incrementing a
a=2
  In main
a=3
In play
b=10

Here 'a' is defined to be an external variable. It is defined outside main(), hence its scope is the entire program. Definition of external variable allocates space for the variable. In order to use variable 'a' in function corn(), we have to declare it in corn() as 'extern int a'. Storage specifiers is to be mentioned in extern .Variable declaration at this place in corn is optional because it is defined outside and above corn() in the same source file. But it is a must to declare external variables.

When we are using them before defining them (as for variable b which is defined before corn) and secondly when we want to use an external variable defined in another file.
Any change made to an external variable by any function is recognized within the entire scope of the variable. Using external variable also reduces the number of argument to be passed to the calling function. As in our example, if we want to use 'a' in function corn(), we need not pass it. It is available to corn() since 'a' is an external variable.

The variable 'b' defined in function play() is local to the function. It takes precedence over the external variable defined outside function corn(). i.e. when control comes to function play, b = 10; is effective.

Static storage class

Static variable are stored in memory. If not initialized, its default value is zero. Static variables are always declared within a function like automatic variable but with a storage class specifier 'static'.

The scope of a static variable is confined to the function in which it is declared or to the block of statements in which they are declared.

The basic difference between automatic and static variable is that static variable unlike automatic variable retains its value even if control goes out of the function in which it is defined. When function is re-entered, the variable is not re-initialized but uses last value assigned to it. The example(1) used for automatic variables with slight modifications will help you clear the concept of static variable.
Example(3):

```
/* program to demonstrate use of static storage class
   variable i  retains its value during second call to function exmp */
# include<stdio.h>
void main( )
   {
      void exmp();
                  exmp();
      exmp();
    }

void exmp( )
    {
            static int i=10;
            printf("\nInside exmp");
            printf("\n%d ", i);
            i++;
     }
```

The output of above program will be

```
Inside exmp
10
Inside exmp
11
```

here when main calls exmp() first the value of i = 10 is printed and i is incremented. When main calls exmp() again, this i = 11 is retained (variable is not initialized again).
i = 11 is printed and i is incremented.
In Example(1) where variable i, in function exmp is automatic variable, first call prints i=10 and the incremented value is lost when function returns. When main calls exmp again i is reinitialized to 10 and again printed as i = 10

Register storage class

The register storage class specifier indicates to the compiler that the value of the object should reside in a machine register. The compiler is not required to honor this request. Because of the limited size and number of registers available on most systems, few variables can actually be put in registers. If the compiler does not allocate a machine register for a register object, the object is treated as having the storage class specifier auto. A register storage class specifier indicates that the object, such as a loop control variable, is heavily used and that the programmer hopes to enhance performance by minimizing access time.

Normally when operation are carried out, it is done by transferring information from the computers memory to the registers, performing operation and then transferring the results back to the computers memory. When a variable is required to be used many times, considerable execution time is required in transferring from memory to register and back. In such cases register storage class is used. When the

variable is stored in register itself, rather than memory, value can be accessed faster. The definition is as follows

```
register int i;
```

There is no default value assigned to register variables and will contain garbage value if not initialized. There is lot of similarity between automatic variables and register variables. The scope of register variable is confined to the block or function in which it is defined. The register looses its value when control is out of the block in which it is defined and is re-initialized on re-entering the block.

E.g.

```
#include<stdio.h>
main( )
{
        register int i ;
        for(i = 1; i<= 10;i++)
        printf("\n%d",i);
  }
```

Here storage class of i is register. When we define storage class of a variable to be register, it does not necessarily mean that a register is for sure assigned, because the numbers of CPU (central processing unit) registers are limited. When register is not assigned, the variable is assigned automatic storage class.

Register storage class cannot be used for all types of variables. They are mainly used for integer type of variables. This is because of the size of register in C.P.U., which are generally 16 bits and cannot store float, double and long values which require more space.

Review Questions

1. What is the output of

```
main ( )
 {
     int h = 40;
     printf ("\nh = %c", h);
  }
```

A. h= <
B. h=>
C. This is not a valid syntax

2. State True or False
   Even if external variable is not defined its initial value is taken to be '0'

A. False
B. True

C. The scope of register variable is confined to the block or function in which it is defined.

A. False
B. True

Summary

- Storage class, type of variable and name of variable together define a variable
- There are basically 4 data-types : integer , character , float and double. The basic types can be further extended by applying different qualifiers.
- 'Characters' can be signed characters or unsigned characters.
- 'Integers' can be short integers or long integers.
- 'Float' are used for floating point numbers.
- 'Double' gives double precision floating point.
- There are four types of storage classes – Automatic, External, Static, Register.
- Automatic variables are always declared within a function. The scope of an automatic variable is confined to the function in which they are declared or to the block of statement in which they are declared. It is the default storage class.
- External variable are defined outside of any function. The scope of external variable extends from the point at which it is defined to the end of the program.
- The basic difference between automatic and static variable is that static variable unlike automatic variables retains its value even if control goes out of the function in which it is defined.
- In register storage class the variable is stored in register itself, rather than memory.
- They are mainly used for integer type of variable

Programming Concepts using C

Chapter 7: Arrays

Objectives

- Discuss arrays
- Define and declare arrays
- Use arrays to store values and retrieve values
- Initialize arrays
- Demonstrate passing an array as a parameter to function.
    - Entire array
    - Individual elements of the array.
- Use 2-Dimensional arrays

Introduction

Until now we used variable which could store only one value at a time. Many times we need to work with a group of data. For e.g. - if we want to input 100 number and perform various operation on them. Using the knowledge obtained till now, we can define 100 variable and store the values of 100 numbers in them. The alternative is to use an array.

In this topic, we will see what is an array. How do we declare, initialize and use an array. How to pass arrays as parameters to functions. We will also study 2- dimensional arrays in brief.

What is an Array?

In order to work with 100 number of same type, instead of defining 100 variable we can use an array. An array can be called as a collection of data items. All the data items should necessarily be of the same type.

All the items in an array have the same name. Then how are they identified would be the question. They are identified with subscript enclosed in square brackets. The subscript of an array generally starts with zero.

E. g. If we want to use 100 numbers

Using variables:

int var1 = 1, var2 = 10 var3 = 14 ,……., var100 = 9;

Using arrays:

int var[100] = {1,10,14,……..,9};

Here 'var' is the name of the array, which stores 100 integers. i.e. the capacity of the array is 100. All the elements in the array have the same name 'var'. Each element is referred to as

var[0] = 1 ← 1st element
var[1] = 10 ← 2nd element
var[2] = 14

.

.

array subscript

Array name → var[99] = 9 ← 100th element

Each element in array is identified with subscripts 0, 1, 2,……by the position in the array. Position of first element is 0, position of last element is 99. (It is one less than number of elements in the array). The need of array may have become clear now. It is easier to refer to array elements and use them instead of defining number of variables for the same purpose.

Let use write a program to find average marks obtained by 30 students in a test.
Example(1):

```
/*Program to find average marks obtained by 30 students in a test
  Demonstrates use of array */
 #include<stdio.h>
void main ( )
{
            float avg, sum = 0;
            int i;
            int marks[30];            /* array definition */
                                                                                    /* store data in
array  */
            for(i=0;i<=29;i++)
                                    {
               printf("\n Enter marks :");
               scanf("%d", &marks[i]);
              }
             /* Get data from array */
             for(i=0;i<=29;i++)
                                    {
               sum = sum + marks [i];
               avg  = sum/30;
               printf("\n Average marks = %f ", avg);
              }
   }
```

We will study the program in parts.

Array definition and declaration

An array is defined like any other variable, except here we have a size – specifier which gives the size i.e. the number of elements in the array. The format is

Storage-class data-type array-name[size-specifier (expression)]
E.g.

Data-type → int marks[30] size – specifier

Array name

Storage-class : same rule about storage class apply to array as ordinary variables. Storage class is optional.**Default storage class is automatic for arrays** defined inside a function and **external for array defined outside function**.

Data-type : All array elements should have the same data-type. Hence we say array of integers, array of floats etc. we cannot have array of 10 elements out of which 5 are integers and remaining 5 are double.

Array-name: It is the name of the array.

Size-specifier expression: It is also called the dimension of an array. It gives how many elements are present in an array. The size specifier must be a positive integer.

The square bracket is an identification that we are dealing with an array. In Example(1) we have defined int marks[30], marks (name of array) is an array of 30 integers. When we define int marks[30], a memory of 30 * 2 = 60 bytes will be allocated to the array; since there are 30 integers and each integer occupies 2 bytes. Array elements occupy continuous memory locations.

If we define an array in one file and use it in other file in a program, we will require to declare the array in the file we are using it. In array declaration, size of array is optional. If specified it must match with array definition. Initial values cannot be present in array declaration.

```
File 1                          File 2
   main ( )                        main ( )
   {                               {
int arr [10];                   extern arr [];          Array s declaration
   ............                 ...............
   ............                 ...............
    }                               }
```

Array definition

Using arrays

Now let us see how to access array elements. The array subscript is used to access individual elements of an array. The following loop in example (1) stores data in the array

```
for (i = 0;  i  <= 29; i + +)
 {
        printf ("\n Enter marks :");
        scanf ("%d", &marks [i] );
 }
```

Variable 'i' is used for the subscript. 'i' is made to run from 0 to 29 (i.e. 30 array elements). Thus marks[i] refer to individual element of an array (i from 0 to 29). Data is requested from the user and stored in the array by accessing each element one by one in for loop. Since scanf requires address of variable, we use &marks[i] in the statement. '&' is the address operator as explained in the chapter on functions. Thus data is entered in the array.
The following block reads each individual data element of the array again by using the subscript of the array.

```
for (i = 0; i <=29; i++)
sum = sum + marks [i];
```

If we want to operate on two array for e.g. if we want to compare two arrays or perform assignment operation, taking entire array as one operand is not allowed. We have to perform operations on element by element basis. Later on we will study string where entire array can be operated on taking it as a single entity.

Initializing arrays

In example(1) we have assigned values to array elements in the program using the for-loop. But arrays can also be initialized as follows

```
int  num [5] = {1, 2, 3, 4, 5};
```

When we initialize an array it is not necessary to specify the size of the array.

```
int  num [] =  {1, 2, 3, 4, 5};
```

The size of the array i.e. 5 will be calculated by the compiler. The individual array elements are assigned values in the order in which they appear.

```
num[0] = 1
num[1] = 2
num[2] = 3
num[3] = 4
num[4] = 5
```

If not initialized, static and external array elements contain a value of '0'. Automatic array elements contain garbage value.

Passing array as parameters to function

- Passing individual elements of array.
- Passing entire array.

Array can be passed to functions, either the whole array or the individual elements of the array.

Passing individual of array

Individual elements are passed to functions and returned from them in the same way as normal variables are passed and returned.
The following example shows how individual elements of array num are passed to the function display().
Example(2):

```
/* program demonstrate passing of individual array elements to function */
#include<stdio.h>
main( )
{
        int i;
        void display(int);
        int num[5] = {1, 2, 3, 4, 5};
        for(i=0;i< 5;i++)
        display(num [i]);
}
    void display(int x)
     {
        printf("\n%d ",x);
     }
```

When we pass individual values of array to function, it can be done in similar manner to ordinary variables i.e. they can be passed by value or by reference.

Passing entire array

But when passing the entire array as a parameter there is a lot of difference. While passing an array to function, as an actual argument only the name must be passed; without the square brackets and the subscripts.
The corresponding formal argument is written in the same way but it must be declare inside the function. The formal declaration of the array should not contain the array dimension. In the declaration of function using arrays as arguments, a pair of empty square brackets must follow the data type of array argument (or name of array if given).

The following example shows us how entire array is passed to function. The example program is just a skeleton.

Example(3)

```
#include<stdio.h.>
main ( )
{
            int n;
            float avg;
            float list [100];              / *array definition * /
 / * declaration of function using array argument * /
          float average (int n, float list [] );
          avg = average (n      ,  list)
                                                actual argument (array name 'list' not followed
by
                                                size of array)
  }
}
/ * definition of function * /
float average (int a , float x[])

{                                    formal argument declaration
                . .
                . .
}
```

In the chapter on function we have seen that argument can be passed by reference or by value. Which method is used while passing array? We have seen a brief explanation of address and pointers earlier.

Let us see how array elements are stored on memory. As said previously array elements are stored in contagious memory locations. The name of the array by itself represents the address of the first element of the array. Taking example of the array 'num' which an array of 5 integers. The name 'num' is the address of the first element, which is '1'. num[0] is the first element. Using the ampersand notation &num[0]. Hence 'num' and &num[0] both address the first array element.

&num[0] &num[1] ..... &num[4]
num num + 1, ... num + 4

1	2	3	4	5

represent array addresses

num[0] num[1] ..... num[4] ← represents array values

Adding the subscript of the array element to array name gives the address of the next element, i.e. address of second element is (num + 1) and is same as &num[1].

In short, 'num' is the address of the first element of the array and (num + i) is the address of the ith element (which is same as &num[i]). num[i] is the ith element.

Going back to previous example; when passing entire array to function, we are passing the name of the array ('list'). Thus passing array is by reference. Any change in any element of the array will be seen throughout the entire scope of the array definition.

2 - Dimensional Arrays

The previous arrays were like a list of elements. Consider the following table

2	4	6
10	12	13
12	14	15
18	20	24

It has 4 rows and 3 columns. It can be represented as a 2 – dimensional array. Row is one dimension and column is second dimension. Each element of the array is referred to by using two subscripts. If 'tda' is the name of the array, then array is defined as

```
Storage-class data – type array-name [no. of rows] [no. of columns]
```

E.g.

```
int tda [4] [3];
```

tda[1] gives the first element of the array, tda[1][1] the second element and so on. Generally 2- D array is referred to row-by-row. Hence 2-D array can be considered one-dimensional arrays placed one below the other .2-D array can be referred to column-by-column. 2-D array is also called a matrix.

The following program shows how the elements of a two dimensional array are accessed.
Example(4):

/ * Program to demonstrate 2-dimensional arrays * /

```
#include<stdio.h>
main ( )
{
        int tda [4] [3];
        int i, j;
        for(i=0;i<4;i++)
                for(j=0;j<3;j++)
                        scanf("%d", &tda[i][j] );
}
```

The program puts data elements in the 2-D array row-by-row. In for loop 'i' which represents row runs from 0 to 3 and j which represent the column runs from 0 to 2. It means, for the 0th row all the column are filled. When i = 1, again j runs from 0 to 2 i.e. for the first row all columns are filled and so on.

If we want to obtain element from the array, the following for – loop can be added to the end of the program.

```
for (i = 0; i< 3; i + +)
          for (j = 0 < 2; j + +)
             printf ("%d", tda [i] [j] );
```

2-D array can be initialized as follows.

```
tda [4] [3] = {  { 2, 4, 6},
                 {10, 12, 15},
                 {18, 20, 24}  };
```

OR

```
tda [4] [3] = {2, 4, 6, 10, 12, 13, 12, 14, 15, 18, 20, 34};
```

If there are too few values within a pair of braces, the remaining elements of that row will be assigned zeros. The number of values within each pair of braces cannot exceed the defined row size. While initializing a 2-D array, the first dimension is optional whereas the second dimension is compulsory.

```
int tda [] [3] = {          }   is ok
```

2-D arrays can be passed to functions but care must be taken to ensure that the declaration of formal array argument must contain explicit size for second dimension and must match with the definition of array in calling function.

Review Questions

1. Which of the following correctly declares an array?
A. int anarray[10];
B. int anarray;
C. anarray{10};
D. array anarray[10];

2. What is the index number of the last element of an array with 29 elements?
A. 29
B. 28
C. 0
D. Programmer-defined

3. Which of the following is a two-dimensional array?
A. array anarray[20][20];
B. int anarray[20][20];
C. int array[20, 20];
D. char array[20];

4. Which of the following correctly accesses the seventh element stored in foo, an array with 100 elements?
A. foo[6];
B. foo[7];
C. foo(7);
D. foo;

5. Which of the following gives the memory address of the first element in array foo, an array with 100 elements?
A. foo(0];
B. foo;
C. &foo;
D. foo[1];

Summary

- An array can be called as a collection of data items.
- All the data items should necessarily be of the same type.
- Each element in an array is identified with subscript 0,1,1,……. by the position in the array.
- An array is defined like any other variable, except here we have a size-specifier which gives the size.
- When we initialize an array it is not necessary to specify the size of the array. The size of the array will be calculated by the compiler.
- Arrays can be passed to functions, either the whole array or the individual elements of the array.
- When passing entire array to function, we are passing the name of the array. Thus passing array is by reference.
- When we pass individual values of array to function they can be passed by value or by reference.
- Generally 2-D array is referred to row-by-row. Hence 2-D array can be considered one-dimensional arrays placed one below the other.

Programming Concepts using C

Chapter 8: String

Objectives

- Describe what strings are.
- Discuss how to access strings.
- Introduce pointers.
- Use string manipulation functions.

Introduction

We saw what arrays are and how to use them. All our example were related to array of integers, array of floats etc. Here we will see a special type of array called strings. So much of the data we handle in our day –to-day life consist of strings. Our name, our address, college, school name etc. Thus string storage and manipulation becomes important in itself.
We shall study what are string and how do we access them. What is the exact difference between string and other arrays? We will also study string manipulation functions, which make use of string easier. Function to compare strings(strcmp), function to find length of string (strln), function to concatenate two string(strcat) etc.

What are strings?

Group of characters is a character array. **A one-dimensional array of characters is called a string**. The difference between any other type of array and string is that a character array ends with a '\0' character.

E.g. char str_name[] = {'S', 'E', 'T' , '\0'};

char array str_name, is initialized as we had previously initialized integer array, but here we have added a '\0' at the end. String can also be initialized as

```
char str_name = "SET";
```

'\0' is automatically added in this case.
It is important to know the difference between a character and a string. String is a group of characters. Every element of a string is a character. A string has to end with a '\0'. Each character occupies 1 byte in memory and the characters of a string are stored in continuous memory locations. 'X' is a character whereas "X" is a string consisting of one character.

It is not necessary to mention the size of the array while initializing, but if you specify the size, you should not count '\0' as character. 'C' compiler will take care of it. In our example, size of name is 3 and not 4. '\0' being the terminating character of a string indicates end of string.

Accessing strings

The following program shows how data is entered into a string, character by character and how data is obtained from a string character by character.

Example(1):

```
/* Program demonstrate how string are accessed
   Data entered in a string character by character and displayed character by character */
#include<stdio.h>
main( )
{
    char poem[10];
    int j;
    for(j=0;j<10;j++)
                                        {
        printf ("\n Enter  character  %d ",j);
        fflush(stdin);                          /*   Block1   */
        scanf("%c",&poem[j]);
       }

 for (j = 0;j<10;j++)
        printf ("%c",poem[j] );                 /*   Block 2 */

}
```

This program is similar to that used for accessing integer arrays. Here '\0' is automatically added to str_name after end of Block 1.

Strings have a wide use in text manipulation. Hence a number of other methods are provided for easy access of strings since we know that '\0' is the terminating character of any string.
We can simplify block 2.

```
    while (str_name [j] ! = '\0' )
    printf ("%c", str_name [j] );
```

'C' provides a special format specifier, "%s" for inputting a string and displaying it, with the help of which entire string can be accessed instead of going character by character. Block 1 can be change to scanf ("%s", str_name);
Block 2 can be changed to printf ("%s", str_name); counter 'j' is not required in this case.

The drawback of scanf and printf is that they can handle only one word at a time. The function gets and puts, discussed earlier can be used as an alternative, they can handle multiword strings.

Introduction to pointers

Before going on further with strings let us revise the concept of pointers of which we have had a bit of feel in the chapter on functions.

There we said that pointers are variables which store address of other variables. Also address of any variable can be obtained by using ampersand (&) operator.

If we have int j = 3;
Address of j is &j.

A pointer variable can be defined as int *k; where k is a pointer to an integer variable. Hence address of j can be assigned to k as k = &j. Contents of j can be obtained from k using '*' operator.

*k gives 3

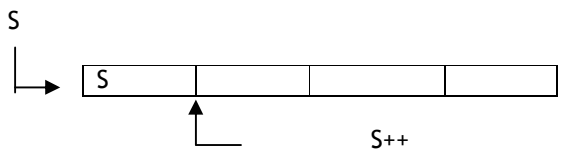
i.e. '*' operator gives contents of address pointer by k.

On a similar basis, what is the address of an array? The name of the array itself is the address of the array. If we have char poem[10]; 'poem' gives bases address of this array. If we define character pointer as

char * ptr;
ptr = poem;
causes ptr to point to the beginning of the array poem. Contents of array can be obtained as

*(ptr +0), *(ptr +1),…..
Instead of
poem[1]…. Both are same.
And an address of array elements as
ptr, (ptr +1),……
Instead of
&poem [0], &poem[1],……Both are same.
Or instead of using an array of characters of given size and setting a pointer to this array, we can use pointer to character and allocate sufficient memory to it, it will be equivalent.

According to pointer logic, if 'S' is a pointer to character. (*S) are the contents of the address at which 'S' points (i.e. value). And ++S pointer to the next character (according to pointer arithmetic, which we will study later.)



Initially s pointer to 'S' (of "SET"). (*s) = 'S', after s++, s points to 'E' Now (*s) = 'E'

Point to be noted is that pointer is a variable but name of array is not a variable. We can say ptr++ but not poem++
This much information is sufficient for understanding the string related built in functions. Pointers and relation between arrays and pointers will be considered in detail in chapter on functions.

String functions

- strlen
- strcmp
- strcpy
- strcat

C provides a number of in-built string handling functions which can be used directly on strings for getting length of strings, comparing, concatenating string and so on.

Following table lists some of these library functions and their purpose.

strlen	Finds length of a string
strlwr	Converts a string to lower case
strupr	Converts a string to uppercase
strcat	Appends one string at the end of another
strncat	Appends first n character of a string at the end of another
strcpy	Copies a string into another
strncpy	Copies first n character of one string into another
strcmp	Compares two strings
strncmp	Compares first n characters of two strings

Let us see how strlen ( ), strcmp ( ), strcpy ( ), strcat ( ) can be implemented using arrays as well as pointers. Also how they are used will be seen with an example of each. The name of the string functions suggests their purpose in most cases. The declaration of all these functions is in the header file "string.h". Hence it is a must to include this header file when we use them.

Note: functions implemented using array are appended with an 'a' at the beginning of the function name. functions implemented using pointers are appended with a 'p' at the beginning of the function name.

E.g. for strlen()

Name of function using arrays is astrlen ( )

Name of function using pointers is pstrlen ( )

strlen()

- Implementing strlen.
- Using strlen.

As name suggests, this function is used to get the length of a string i.e. the number of character in a string, excluding the terminating character '\0'.

```
Int strlen (const char * s);
```

It accepts address of a string as an argument and returns an integer i.e. length of a string. Can you imagine how this function would have been written? It is simple. Let us implement it using array of characters as well as pointer to character.

Implementing strlen ( )

- Array subscript version.
- Pointer version.

Function strlen is already provided and its prototype is written in the header file string.h. We write function astrlen and pstrlen which imitate strlen. Function astrlen uses array to accommodate the string while function pstrlen uses a pointer equivalent to point to the string.

Array subscript version by using astrlen function

```
int astrlen (char *s)
{
        int len = 0, j = 0;
        while (s[j] ! = '\0')
         {
           len + +;
           s [j + +];
          }
           returns (len);
}
```

Pointer version by using pstrlen function

```
int  pstrlen (char * s)
{
           int len = 0;
           while ( ( * s) ! = '\0')
         {
            len + +;
            s + +;
          }
         return (len);
}
```

All that the function does is, it counts the number of character till string terminating character '\0' is encountered.

Using strlen()

The following program shows use of function strlen to calculate length of character array arr.

```
#include<stdio.h>
#include<string.h>
main ( )
{
      int length;
      char arr [] = "L&T Infotech";
      length = strlen (arr);
      printf ("\n Length of string %s is  %d", arr, length);
}
```

strcmp()

Implementing strcmp.
Using strcmp
This function compares two strings.

```
 int strcmp (const char * s, const char * t);
```

It accepts base address of two strings. It returns negative, zero or positive if 's' is less than, equal to or greater than 't'.

Implementing strcmp

- Array subscript version.
- Pointer version.

Function strcmp is already provided and its prototype is written in the header file string.h. We will write function astrcmp which imitate strcmp. Function astrcmp uses array to accommodate the string while function pstrcmp uses a pointer equivalent to point to the string.

Array subscript version

```
/ * strcmp   : will return
< 0 if s < t
= 0 if s = = t
> 0 if s > t   * /

int astrcmp (char * s, char * t)
{
    int j;
    for (j = 0; s [j] = = t [j]; j + +)
    {
          if (s [j] = = '\0')
          return  0;
    }
    return     (s [j]   - t [j]);
}
```

Pointer version

```
int pstrcmp (char * s, char * t)
{
    int j;
    for (; *s = = *t;   s++, t + +)
        {
            if (*s = = '\0')
            return 0;
        }
        return (* s  - * t);
}
```

Strings 's' and 't' are compared character by character and the position where 's' and 't' do not match, the characters are subtracted which gives numeric difference, between ASCII values of non – matching characters, to get the return value. If end of string 's' occurs before a mismatch, 0 is returned.

Using strcmp ()

The following program shows use of function strcmp to compare two strings.

```
#include<stdio.h>
#include<string.h>
main ( )
 {
          char s[10], t[10];
          int v;
          printf("\n Enter 2 strings:    ");
          scanf("%s %s",s,t);
          v = strcmp(s,t);
          if (v>0)
          printf("\n First string is greater than second string ");
    else  if(v<0)
         printf("\nFirst string  is less than second string ");
    else
         printf("\nFirst string  is equal to second string ");
}
```

Output
Enter 2 Strings Susan
Susan
First string is equal to second string

strcpy ()

- Implementing strcpy
- Using strcpy.

This function copies the content of one string to another.

```
char * strcpy (char *t, const char *s);
```

Base address of source string and target string are to be supplied to this function as arguments. It returns the target string.

Implementing strcpy()

- Array index version.
- Pointer version.

Function strcpy is already provided and its prototype is written in the header file string.h. We write functions astrcpy which imitate strcpy. Function astrcpy uses array to accommodate the string while function pstrcpy uses ‘s’ pointer equivalent to point to the string

Array index version

```
astrcpy (char *t, char *s)
{
    int j = 0;
    while ( (t [j] = s [j])  ! = ‘\0’)
              j + +;
}
```

Pointer version

```
pstrcpy (char *t, char *s)
{
      while ( (*t = *s)   ! = '\0')
    {
              s + +;
              t + +;
     }
}
```

Strcpy copies the characters in source string into the target string, one by one. Care is to be taken to see that target string is large enough to hold the entire source string.

Using strcpy()

The following program shows use if function strcpy to copy source string to target string

```
#include<stdio.h>
#include<string.h>
main ( )
{
      char source [] = "Sayonara";
      char target [20];
      strcpy (target, source);
      printf ("\n Source string = %s", source);
      printf ("\n Target string = %s", target);
}
```

Output
Source string = Sayonara
Target string = Sayonara

strcat()

- Implementing strcat.
- Using strcat.

This function concatenates the source string at the end of the target string. Concatenating is appending or adding.
E.g. "Good" and "work' on concatenation gives "Good work".

```
char  *strcat (char *t, const char *s);
```

It accepts base address of two strings to be concatenated as arguments and returns the target string.

Implementing strcat()

- Array index version.
- Pointer version.

Function strcat is already provided and its prototype is written in the header file string.h. We write function astrcat and pstrcat which imitate strcat. Function astrcat uses array to accommodate the string while function pstrcat uses a pointer equivalent to point to the string.

Array index version

```
astrcat (char *t, char *s)
{       int j = 0, k = 0;
        while (t [j]   ! = '\0')
         t [j + +];
        while ( (t [j] = s [k] )  != '\0')
      {
                s [k + +];
                 t [j + +];
       }
}
```

Pointer version

```
pstrcat (char *t, char *s)
{
            while (*t   ! = '\0')
            t + +;
            while( (*t = *s)   ! = '\0')
             {
                  s + +;
                  t + +;
             }
}
```

The target string has to be large enough to hold the resulting string.

Using strcat()

The following program shows use of function strcat to concatenate source string to target string.

```
#include<stdio.h>
#include<string.h>
main ( )
{
                    char source [] = "Hello";
                    char target [30] = "  world!";
                    strcat (target, source);
                    printf ("\nSource string = %s", source);
                    printf ("\nTarget string = %s", target);
}
```

Output
Source string =Hello
Target string =   world!

2-Dimensional array of characters

We have seen 2-dimensional arrays, in chapter on arrays. On the same basis we can have 2-Dimensional array of characters. Imagine them to be string stored one after another.

```
char names [5] [10] = {    "Akshay", "Parag","Raman", "Srinivas", "Gopal"};
```

Above is a 2-dimensional array of characters. The first dimension gives the number of strings in the array and the 2nd dimension gives the length of each array.

Let us write a program of exchanging names in the above array.

Example(2)

```
/* Program demonstrating use of two dimensional arrays
   program exchanges two names */
#include<stdio.h>
main ( )
{
        char names [] [10] =  {"Akshay", "Parag","Raman", "Srinivas", "Gopal"};
        int k;
        char  t;
        printf ("\n Original:  %s\t%s", &names [2][0], &names [3] [0]);
        for (k = 0; k <= 9;  k++)
        {
            t = names [2] [k];
            names [2] [k] = names [3] [k];
            names [3] [k] = t;
         }
          printf ("\n New     :  %s\t%s  ", &names [2] [0], &names [3] [0] );
}
```

Output
Original: Raman Srinivas
New: : Srinivas Raman

To exchange the names we are required to exchange corresponding characters of two names. Also, since 10 characters are not used for all names, the remaining space is wasted in memory. We will see how both these drawbacks can be eliminated by using array of pointers to strings, which we will come to in chapter on pointers.

Review Questions

1. Which of the following is a static string?
A. Static String
B. "Static String"
C. 'Static String'
D. char string[100];

2. What character ends all strings?
A. '.'
B. ' '
C. '\0'
D. '\n'

3. Which of the following reads in a string named x with one hundred characters?
A fgets(x, 101, stdin);
B. fgets(x, 100, stdin);
C. readline(x, 100, '\n');
D. read(x);

4. Which of the following functions compares two strings?
A. compare();
B. stringcompare();
C. cmp();
D. strcmp();

5. Which of the following adds one string to the end of another?
A. append();
B. stringadd();
C. strcat();
D. stradd();

Summary

- A one-dimensional array of characters is called a string.
- The difference between any other type of array and string is that a character array ends with a '\0' character.
- Each character occupies 1 byte in memory and the characters of a string are stored in continuous memory locations.
- 'C' provides a special format specifier, "%s"for inputting a string and displaying it
- The functions gets() and puts() can handle multiword strings.
- C provides a number of In-built string handling functions which can be used directly on strings. The strlen(), is used to get the length of a string. The strcmp() compares two strings. The strcpy() copies the contents of one string to another. The strcat() concatenates two strings

Programming Concepts using C

Chapter 9: Pointers

Objectives

- Discuss the concept of pointers.
- Use pointers instead of arrays.
- Perform operation on arrays.
- Use pointer to pointers.
- Use array of pointers.
- Use pointer to function

Introduction

Until now we have studied variables, array and functions. In this chapter we will study pointers which are nothing but variables storing addresses. Address of other variables, array and also functions. In chapter on functions and strings, we have taken a slight introduction on pointers and seen how values can be passed by reference and more than one value can be returned from functions using pointers.

In this chapter we will study pointers in detail like relationship between arrays and pointers, what operations can be performed on pointer & how pointers are operated. We will also introduce the concept of pointers to function.

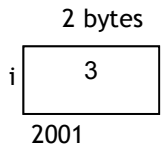
Though pointers seem a bit complicated at the outset there is nothing as interesting and useful as pointers.

What are pointers?

In order to understand what we mean by pointers, let us first understand how variables are stored in memory. When we define a variable; memory space depending upon the data-type of the variable is assigned to the variable.
E.g.

If variable is integer then 2 bytes are assigned to the variable.
If variable is character then 1 byte is assigned.

When we defined int i = 3; two bytes are allocated to variable 'i'.



We can consider the analogy of variable being a house whose name is 'i' . The value 3 stays in it having house number as 2001. This 2001 is called the address of a variable.

House	Variable
Name	Variable name (i)
Who stays	Value of variable (3)
House no.	Address (2001)

Any variable stored in memory has a memory address.

A pointer is a variable itself, which stores the address of another variable of some specific type. So what is the difference between a pointer and other variables? The contents of pointer are address of another int, float, char etc. whereas contents of other variable are an int, float, char etc.
Address of a variable is given by ampersand notation (&), which is called the unary operator or address operator that evaluates address of its operand.
Hence address of int i is &i. In pointer variable declaration, variable name must be preceded by an asterisk (*)

Data-type *ptvar;

Data-type of pointer object

Name of pointer variable

We define a pointer variable as int *j where j is a pointer variable which contains the address of an integer variable. The '*' notation is used with pointer variables to give the value of the variable pointed to by a pointer variable.

j = &i; says that j contains the address of integer variable i
i.e. j = 2001

j (2 bytes , address)	i(2 bytes , int)
2001	3
4008	2001
House	Pointer variable
House name	j
Who stays	Value of j (2001)
House number	Address of j (4008)

Collecting the statement together

```
int i = 3;
int *j;
j = &I;
```

i is an integer variable containing 3.
j points to i.
j gives the contents of the address it contains.

Example(1)

```
/* Program demonstrating the concept of pointers
The use of * and & operators is shown */
#include<stdio.h>
main ( )
{
int i = 3;
int *j;
j = &i;
printf ("\n%d  %u  %u  %d\n", i,  &i, j, *j);
}
```

Above program prints the values of i, &i, j and *j respectively. A point to be noted and remembered is that a pointer points to a particular kind of object i.e. every pointer point to a specific data type.

If j point to integer i, then *j can occur anywhere where i could.
If we say i = i + 1; i++; ++i.
Is same as j = j +1; (*j)++; ++ (*j)

Pointers and arrays

Pointers and array are closely related to each other. Pointers can be used in all places where arrays are used.

If we have an array int n[5].
And a pointer to an integer int *pt.

Then pt = &n[0] or pt = n (since name of array is synonym for address of first element). ,sets 'pt' to point to the 0th element of n. i.e. 'pt' to contains the address of n[0].

pt + 1 points to the next element of array i.e. n[1] and
pt+i points to the ith element of the array
if pt points to n[0]
*(pt + 1) are contents of n[1] and
*(pt + i) are contents of n[i]

The following table gives a clear picture.

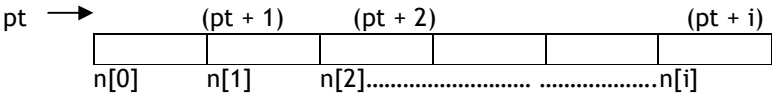
→ n Array of integers
→ pt pointer to integer

Address		Contents	
&n[0]	pt	n[0]	*pt
&n[1]	pt + 1	n[1]	*(pt + 1)
&n[2]	pt + 2	n[2]	*(pt + 2)

Example (2) prints the value and address of each element using pointer as well as array to show the relation between the two.

```
/* Program prints the value and address of elements in an array
using array as well as pointer to show relation between the two */
#include<stdio.h>
void main ( )
{
        int i;
        int n[5] = {1,2,3,4,5};
        int * pt;
        pt = n;
        for (i = 0; i <= 4; i++)
    {
        printf ("\n Using Arrays: ");
        printf ("\n Address : %u  \t  value :  %d",  &n[i], n [i] );
        printf ("\n Using  Pointers : ");
        printf ("\n Address  : %u\t  value : %d", pt + i , *(pt + i) );
    }
}
```

The combination of array name with its subscript is equivalent to the combination of pointer with its offset.



Now let us write example(1) in array topic using pointers.

```
/* example (1) in array chapter rewritten using pointers
to show the relation between arrays and pointers */
#include<stdio.h>
void main ( )
{
        float avg, sum = 0;
        int i;
        int marks [10];
        int * pt;
        pt = marks;
        for (i = 0;  i  <= 9; i ++)
        {
            printf("\n Enter marks : ");
            scanf ("%d",pt  +  i);
        }
        for (i  = 0; i  <= 9;  i ++)
        sum = sum + *(pt + i);
        avg = sum / 10;
        printf ("\n Avg marks = %f", avg);
}
```

Here pt is a pointer to an integer. pt = marks, assign pt to point to the first element of array 'marks' .Since scanf requires address of variable, as i changes from 0 to 9, (pt + i) gives the address of corresponding array element. *(pt + i) gives the value at the address (pt + i) points to.

It is to keep in mind that a pointer is a variable but array name is not a variable, hence assignment, increment, decrement operations can be done on pointers but not on array names.

Operations on pointers

- Increment-decrement.
- Adding a number to pointer.
- Subtracting a number from a pointer.
- Subtraction of one pointer from another.
- Comparison of two pointer variables.
- Number of operation can be performed on pointers since they are variables.

Operations like incrementing pointers, decrementing pointers, subtracting pointers from each other, comparing them and subtracting a number from pointer are possible. But some operations seem absurd and would give unpredictable results if performed.

Increment - decrement

int * pt;
pt ++ increment pt to point to next location of same type. What do we mean by same type? If pt is a pointer to integer, integers occupy 2 bytes. So if pt = 4001, pt++ gives pt = 4002.

pt-- decrements pt to point to previous element of the same type.
This will be clear if we consider pointers with reference to arrays whose elements are placed in consecutive memory location.

Adding a number to a pointer

A number can be added to pointer

```
E.g. int i  = 3, *j;
         j  = &i;
         j  = j + 1;
         j  = j + 9;
/* j points to 9 integer locations after current location */
```

Subtracting a number from a pointer

A number can be subtracted from a pointer

```
E.g. int   i  = 3, *j;
      j  = &i;
      j  = j -2;
      j  = j -6;
/* j point to 6 integer location after current location */
```

Subtraction of one pointer from another

One pointer variable can be subtracted from another, provided both point to elements of same array. The result is the number of elements between corresponding array elements.

```
#include<stdio.h>
main ( )
{
          int arr [] = {10,  20,  30,  45,  67,  56,  74};
          int  *i, *j;
          i  = &arr [1];
          j  = &arr [5];
          printf (“%d %d”, j  - i  , *j  -  *i);
}
```

Output
(j – i) gives the difference between address
(*j - *i) gives the difference between values contained in address at j and i

Comparison of two pointer variables

Comparison of two pointer variables is possible provided both point to one array (i.e. both pointers point to objects of same data type). Pointers can also be compared with zero(NULL).

```
#include<stdio.h>
main ( )
{
        int arr [] = {10,  20,  36,  72,  45,  36};
        int *j, *k;
        j  = &arr [4];
        k = (arr + 4);
        if (j == k)
        printf ("\nThe two pointers point to same location. ");
           else
        printf ("\n The two pointers do not point to same location. ");
}
```

Output
The two pointers point to same location.

The operation of adding two pointers, multiplication of a pointer with a constant, division of a pointer with a constant should not be performed.

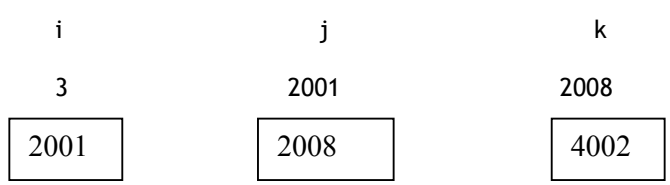
Pointer to pointer

A pointer to a variable is a variable that stores address of another variable of a specific type. A pointer to pointer is a variable that stores address of a pointer variable of specific type.

```
/* Program demonstrates  pointer to pointer */

#include<stdio.h>
main ( )
{
        int i  = 3, *j, **k;
        j  = &i;
        k  = &j;
        printf ("\nAddress of i  = %u", &i);
        printf ("\nAddress of i  = %u", j);
        printf ("\nAddress of i  = %u",*k);
        printf ("\nAddress of j  = %u", &j);
        printf ("\nAddress of j  = %u", k);
        printf ("\nAddress of k  = %u", &k);
        printf ("\nValue of j   = %u", j);
        printf ("\nValue of j  = %u", *k);
        printf ("\nValue of k = %u", k);
        printf ("\nValue of i = %d", i);
        printf ("\nValue of i = %d", * (&i));
        printf ("\nValue of i = %d", * j);
        printf ("\nValue of i = %d", **k);
}
```

In the above example
i is an ordinary integer variable
j is a pointer variable which contains address of i
k is a pointer variable which contains address of j. j is a pointer hence k is a pointer to pointer.

The following figure can make things clear



Variable	Value	Address
i	3	2001 (&I, j, *k)
j	2001 (j, *k)	2008 (&j, k)
k	2008 (k)	4002 (&k)

The address given here are for explanation purpose. You may get different addresses on your computer. Thus address of i can be obtained also in terms of j and k as given in example program. Also value of i can be obtained in terms of j and k. value of i as *(&i) is same as *j because &i = j.

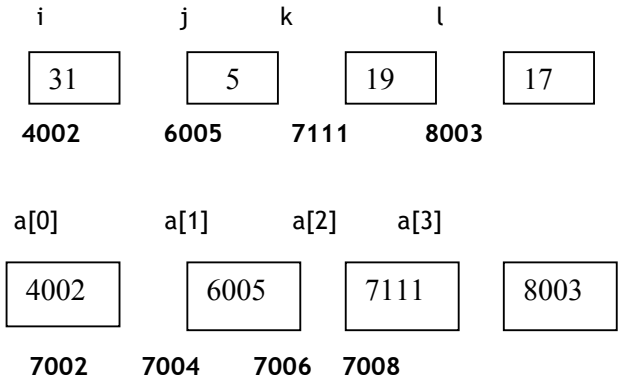
Array of pointers

We have seen array of integers, array of characters. Similarly we can also have array of pointers. Since pointers are nothing but variables which store addresses. Array of pointers is a collection of addresses.

The format is
Data-type * array-name [expression]
The square bracket of array takes precedence over the '*' operator. It is array of pointer to specific data-type. The address can be address to individual variable or address of array elements. The following program illustrates the basic concept of array of pointers.

```
//Array of pointers
# include<stdio.h>
main ( )
{
   int  *a [4];       /* array of pointers to integers */
   int i = 31, j = 5, k = 19, l = 17, m;
       a [0] = &i;
       a [1] = &j;
       a [2] = &k;
       a [3] = &l;
      for (m = 0; m <= 3; m++)
      printf ("%d  ", * (a [m] ) );
}
```

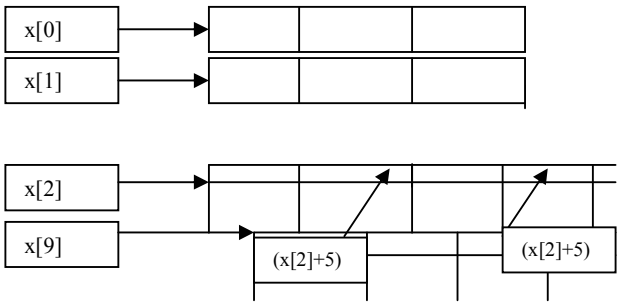
int *a[4] declares an array a as an array of pointers (which is indicated by the *). It is pointers to integers. i.e. 'a' is a collection of addresses which contains addresses of integers.
The next line defines integers i, j, k, l. The addresses of these integers are assigned to each array elements. We know &i is address of i and so on.
In for loop, m run from 0 to 3. a[m] gives an address of integer. '*' gives contents of address i.e. *a[m] gives contents of address a[m].
The difference between ordinary variable and pointer variable is that ordinary variable stores value of specific type, while pointer stores address of specific type. Same difference exists between array and array of pointers. Array is a collection of values. **Array of pointers is a collection of address.**



We have said, address can also be addresses to array or array elements. This is a very interesting and important feature. We have studied multidimensional arrays. The name of multidimensional array is a pointer to group of arrays. A multidimensional array can be expressed in terms of array of pointers rather than as a pointer to group of continuous arrays.
We can use

data-type * array-name [expression 1]
instead of
data-type * array-name[expression 1][expression 2]

The last expression is omitted when defining array of pointers
With the following example, let us see relation between multidimensional array and array of pointers.
We have a two-dimensional integer array with 10 rows and 20 columns.



We can define X as a two dimensional
Array, int x[10][20].
Or as a 1 dimensional array of pointers, int *x[10].

x[0] points to beginning of first row.
x[1] points to beginning of second Row and so on.
Here number of element in each row is not given

If we want to access x[2][5] it can be done by
*(x[2] + 5)

x[2] gives address of third row.

(x[2] + 5) gives address of fifth element in third row.

*(x[2] + 5) gives contents

Array of pointers to strings are widely used. The n-element array can point to n different strings. The two dimensional array of characters 'nam' can be stored in array of pointers.

```
char * nam [] = {   "Akshay",
                    "Parag",
                    "Raman",
                    "Srinivas",
                    "Gopal"
                 };
```

The need to use pointer instead of multidimensional arrays can be explained at this place. In two-dimensional arrays 'nam', a memory space of 5 * 10 = 50 bytes is allocated straight away. While in pointer array, (number of character in all rows added) = 29 bytes and 10 bytes for address i.e. 29 + 10 = 39 bytes are used. Thus there is a lot of saving in space. Secondly, string can be manipulated with ease using pointer arrays.

```
/* Rewriting example (2) in strings chapter, Exchanging strings using pointers instead of arrays
*/
#include<stdio.h>
main ( )
    {
        char *name[]   = {"akshay","parag","raman","srinivas","gopal"};
        char* temp;
        printf ("\n Original Names : %s %s",name[2],name[3]);
        temp = name[2];
        name[2] = name[3];
        name[3] = temp;
        printf ("\n New Names : %s  %s",name[2],name[3]);
    }
```

Output

Original Names : raman srinivas
New Names : srinivas raman

A temporary pointer to character is defined and only the addresses of names are exchanged instead of the names themselves.

Pointer to function

The idea may seem strange but it exists. As we can have address of integer variable, character variable, array, we can also get address of a function. Thus we have a pointer to function. So we can invoke functions using their pointers instead of the usual way.

```
#include<stdio.h>
main ( )
    {
        void display( );
        printf ("\n Address of function display is %u ", display);
        display( );
      }
  void display( )
      {
        puts ("\n Example of pointer to function");
      }
```

Output
Address of function display is 419872
Example of pointer to function

Above is a simple program, which gives a normal call to function 'display'. Now if we want to invoke function, using pointer we will need the address of the function. How do we get the address of function? The name of the function by itself without the round brackets after it, gives the address of function.

The program given below invokes function display using its address

```
/* program demonstrating pointer to function it invokes function
display using its address  */
#include<stdio.h>
main ( )
{
        int display ( );
        int (*func_ptr) ( );
        func_ptr = display;
/* assign address of function */
      printf ("\n Address of function display is %u", func_ptr);
        (*func_ptr) ( );
/* invokes function display */
}
int display( )
{
   puts ("\n Into the world of pointer to function");
            return(1);
}
```

Let us study the program statement by statement. The statement in main, int display(), declares that function returning integer is used in main. func_ptr is a pointer to function which return an integer. If you have understood how to define pointer to variable and array of pointer, you will understand this definition quickly.

Pointer to integer	int *p;
Array of pointers to integer	int *p[];
Pointer to function returning integer	int(*p)();
Function returning pointer to integer	int *p();

We use round brackets around the '*' operator because we want the pointer to precede the function brackets. If we say int *p(), it defines a function returning a pointer to integer. Here function brackets take precedence. Thus func_ptr is a pointer to a function returning an integer. The address of function display is assigned to func_ptr in the third statement. First pointer to function is defined then an address is assigned to it. The basic remains the same no matter pointer points to any object. (*func_ptr)() invokes the function display since func_ptr now points to display.

Dynamic allocation

malloc()

When we say int X[10], the compiler assigns 10 * 2 = 20 bytes of memory to X. Using pointers, dynamic allocation of memory can be done. **Dynamic allocation is allocating memory dynamically i.e. at execution time.**
If we say
int *X;
X is not assigned a memory block when it is defined as a pointer, sufficient amount of memory can be assigned to X by using the memory allocation malloc().
X = malloc(10 * sizeof (int));
The above statement assigns a block of memory to X whose size is equal to 10 * size of integer quantity. The **malloc() returns a pointer to character**. But since X is a pointer to int, to maintain the consistency we use typecasting.

```
X = (int*) malloc(10 * sizeof (int));
```

This is known as dynamic allocation. In dynamic allocation, memory can be allocated and freed as and when required. Hence there is no wastage of space. The only limit being the available space for allocating memory.

calloc()

Another function calloc can be used instead of malloc(), which initializes to **0**, the memory block allocated. While in case of malloc they contain garbage values.

realloc()

It can be used to append memory to existing memory block. If we have assigned 10 bytes to X using malloc(). And later we decide to add 4 bytes to X, again using malloc(), it will assign 4 bytes but previous 10 bytes will be lost. Thus using realloc() adds 4 bytes to existing 10 bytes.

free()

It frees the specified memory block to be used for other purpose.

Review Questions

1. Which of the following is the proper declaration of a pointer?
A. int x;
B. int &x;
C. ptr x;
D. int *x;

2. Which of the following gives the memory address of integer variable a?
A. *a;
B. a;
C. &a;
D. address(a);

3. Which of the following gives the memory address of a pointer a?
A. a;
B. *a;
C. &a;
D. address(a);

4. Which of the following gives the value stored in pointer a?
A. a;
B. val(a);
C. *a;
D. &a;

5. Which of the following is the proper keyword to allocate memory in C?
A. new
B. malloc()
C. create
D. value

6. Which of the following is the proper keyword to deallocate memory?
A. free
B. delete
C. clear
D. remove

Summary

- Pointers are variable which store addresses. (Address of other variables, array and function.)
- Every pointer points to a specific data type.
- Address of a variable is given by ampersand notation (&) . The star '*' notation is used with pointer variable to give the value of the variable pointed to by a pointer variable.
- Pointers can be used in all places where array are used.
- The combination of array name with its subscript is equivalent to the combination of pointer with its offset
- A pointer is a variable but array name is not a variable. Hence assignment, increment, decrement operations can be done on pointers but not an array names.
- A pointer to pointer is a variable that stores address of a pointer variable of specific type.
- Array of pointers is a collection of addresses.
- We can invoke functions using their pointers instead of the usual way.
- Dynamic allocation is allocating memory dynamically i.e. at execution time.

Programming Concepts using C

Chapter 10: Structures

Objectives

- Discuss what is a structure and why it is used.
- Discuss basics of structure
- Discuss typedef
- Use nested structures.
- Use array of structures.
- Use pointer to structure.
- Pass structures as parameters to functions.
- Discuss Unions and how to use them.
- Discuss bitfields and how to use bitfields.

Introduction

We have studied arrays which is a data structure containing elements of same type. Now we will study structures whose individual elements are of different type, which resembles the real world application closely.

In this chapter we will see how structures are defined and how their individual members are accessed and used within a program. We will see how pointers, array and functions are related to structures. We will also study unions, which are slightly different from structures and how they are used. We will introduce the concept of bit-fields which is basically a space-saver.

What is a structure and why structures are used?

Structure is a collection of data items of different data-types. We have seen that a variable can hold a single data item. Array can hold a collection of data items but all must be of the same type. Only arrays are insufficient, because in real life if we consider any object, entities related to that object are of various types. For e.g. if we want to keep a record of all the employees in a company, an employee number of integer type and salary type float. This can be done by using one array. One integer array for employee number and a third array of floating point numbers for salary.

The following code snippet shows how a record of two employees is created using arrays.
Example(1)

```
#include<stdio.h>
void main()

{

        int emp_no[4];
        float salary[4];
        int j=0;
        printf("enter details of employee\n");

        for(j=0;j<=2;j++)
        {
                scanf("%d %f",&emp_no[j],&salary[j]);
                        printf("\n");
        }

        for(j=0;j<=2;j++)
        {
                printf("\n%d\t%f",emp_no[j],salary[j]);
        }



}
```

Since we have considered only 2 entities emp_no and salary we require only 2 arrays. The number of arrays will go on increasing as more and more items are added. It will then become infeasible.

Structures make things simpler. Also using structures it is clear that name, emp_no and salary are all related to employee.

Defining a structure

The general format of structure definition is

```
struct tag {
                    member 1;
                    member 2;
                    ……….
                    member 3;
          };
```

Remember the semicolon after the closing brace brackets. 'struct' is the keyword which is required, 'tag' is the name of the structure and members are the elements of the structure. Once type of structure, i.e. its composition is defined, individual variables can be declared as storage-class.struct tag var1, var2, ……varn where var1, var2….varn are structure variables of type tag. The structure composition and variable declaration can be combined

```
struct tag {
                    member 1;
                    member 2;
                    ……….
                    member n;
                  }var1, var2,…. varn;
```

Let us write the previous program using structures.

```
struct emp
                {
                  char name [15];
                  int emp_no;
                  float salary;
                };
```

The name of structure is 'emp'. It contains members, an array of characters, an integer, and a float variable. An individual member of a structure can be of our usual data-type(int, char..) or array or pointer or other structures. The names of all members within a structure should be distinct. They can be same as any other variable outside the structure. The declaration of structure as above doesn't reserve any space but just gives the format of structure i.e. its type.

Like in an array, each member of a structure is stored beside each other

Name	Emp_no	Salary

Now let us define variables which will use struct emp
struct emp e1, e2, e3;
e1, e2, e3 are variables of type emp. i.e. each will use format of struct emp. Also this statement assigns space in memory for each variable. Memory assigned to a structure variable is equal to storage space required for all members added. i.e. for each e1, e2, e3

Space assigned = 15 + 2 + 4 (15 for char array + 2 for integer + 4 for float)
= 21 bytes

The format of structure and variable definition combined

```
Struct emp
        {
            char name [15];
            int emp_no;
            float salary;
        } e1, e2, e3;
```

The drawback of eliminating structure name is that later on in the program we cannot define more variables of same format using name of structure. All needed variables are to be defined with format itself at the beginning.

typedef

At this point let us introduce the feature of typedef. Using typedef we can assign a new name to an existing data-type or a complex data-type. It will be said to be user defined data type. Variables can be defined using this user defined data type; once it is established.
Its general form is
Typedef type new-type;

E.g. if we say **typedef int tool;**

Tool is the new user defined data type and can be used in place of int. and we can use
tool x, y, z; in place of int x, y, z;

It is to be noted that new data-type is not established but a new name is given to existing data-type. Typedef feature is particularly convenient for structures since its use eliminates the repeated use of struct tag while defining variables. Also the new name often suggests the purpose of the structure.
We can write

```
typedef struct emp
        {
            char name [15];
            int emp_no;
            float salary;
        } record;
```

And define variables of struct emp using
record e1, e2, e3;

Initializing Structures

We cannot initialize individual structure members at the time of stating its format.

```
i.e.    struct emp
        { char name [15] = "Tom";
            int emp_no = 5;
          float salary = 11800.56;
        };
```

Is not allowed.

Structure can be initialized as follows:

struct emp e1 = {"John", 5, 11800.56};
struct emp e2 = {"Jack", 6, 10000.00};

Just as we initialize arrays by enclosing the values separated by commas in brace brackets.
During initialization, the values should be given in the order in which the members are set.

Accessing members of structures

Let us see how we can use individual members of structure. A structure member can be accessed by writing **structure variable. member**

The dot ('.') is called the structure member operator which connects the structure variable name and the member.

If individual member is an array then
variable.member[expression]
expression is non-negative integer representing array elements.

Members of structures can be operated as individual separate entities. '.' Operator falls into the highest precedence group and its associativity is left to right.

For e1 employee structure, the number of employee can be obtained as **e1.emp_no**. While the employees salary can be obtained as **e1. salary.** If we want to know the 5th character of employees name it can be done by saying **e1.name[5]**.

Once you know how individual members of structure are accessed they can be used like any other ordinary variables. The operations performed on entire structure are: address of structure can be obtained, one structure can be assigned directly to other structure provided format of both structures are same.
Using all the explanation given till now let us modify example(1) using structures
Example(2):

```
/*Example 1 rewritten using structures*/
/* Data related to employees  employee number, salary and name of employee is stored and
retrieved using a structure */

#include<stdio.h>
main ( )
{
            typedef  struct emp
                    {
                       char name [15];
                       int emp_no;
                       float salary;
                     } record;
            record e1, e2;
            printf("\nEnter employee name, employee number and salary of 2 employees :");
            scanf("%s       %d      %f ", e1. name, &e1.emp_no, &e1.salary);
            printf("\n");
            scanf("%s       %d      %f", e2. name, &e2.emp_no, &e2.salary);
            printf("\n");
            printf("\n%s     %d     %f", e1. name, e1.emp_no, e1. salary);
            printf("\n%s     %d     %f", e2. name, e2.emp_no, e2.salary);
}
```

Structure format is declared variable e1 and e2 are defined. Values of employee no, name and salary for both variable e1 and e2 are taken from user and printed.

Using structures

- Structure in structure.
- Array of structures.
- Pointer to structure.

- Passing structure as parameter.

We have seen all the basics of structures. Now we will see how to use structures for the following.

- how to use in relation to pointers and functions
- how array of structures can be formed and used
- how structures can be embedded in each other.

Structure in structure

Member of a structure can be a structure itself. When does this need arise? When a member of a structure has to be further broken down into entities that member can be declared as structure. Suppose we want to add address of the employee, in the employee records, address will consist of phone, city and pin. Hence a structure can be defined as

```
struct address
    {
            char phone [15];
            char city [15];
            int pin;
     }
struct emp
    {       char name [15];
            int emp_no;
            float salary;
            struct address a;
     }e1;
```

Thus member 'a' is itself a structure. In such cases declaration of embedded structure must appear before the declaration of the outer structure.

The struct emp can now be initialized as
struct emp e1 = ("John", 5, 18844.70, "022-5334716", "Mumbai", 411421};

i.e. just append the member of embedded structure to the main structure.
The member of embedded structure can be obtained with an additional dot operator. If we want to obtain the city from employees address it is written as **e1. a. city;**

Array of structures

Since we found only arrays insufficient to solve real life problems, we studied structures. But, if we have a large number of employees say 100, we will need to define 100 structure variables. Arrays again come to our rescue. We can define array of structures just like we define array of variables of any other data-type.

The array of structures is defined and operation on this array is performed just like our usual array of other data types. When we define array of integers, we say

int num[10];

The general format for array definition is
Data-type array-name[size of array];

The definition below groups four characteristics of a rectangle into a single struct called rectangle.

```
struct rectangle
 {
   float l; /* length */
   float w; /* width */
   float A; /* Area */
   float P; /* Perimeter */
 };
```

What this definition does not do is get any memory allocated -- what it does do is create a new programmer-defined data type. Consequently, no data can be stored in rectangle. To get memory allocated, we need to declare a data structure of this type, such as is done below.

```
struct rectangle rect;
```

This structure consists of four members which can be accessed by means of the dot operator, which is also known as the structure member operator. The following example assigns the value 13.2 to the width member.

```
rect.w = 13.2;
```

Obviously, there needs to be a method for using a pointer to this structure. This is needed, for example, for calls to scanf() and in functions where you wish to change values stored in the structure. The scanf() call below uses the address-of operator to store data in the structure member for the width.

```
scanf("%f", &rect.w);
```

The following declares r to be a pointer to a structure of type rectangle and then assigns to it the address of the rect data structure.

```
struct rectangle *r;

r = &rect;
```

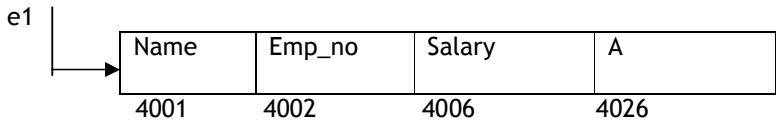
Once the address of the structure has been assigned to the pointer variable, the arrow operator (also known as the structure pointer operator) is used to access individual members. It gets is name from the fact that -> looks like an arrow. Here the product of the length and width members is assigned to the area member.

```
r->A = r->l * r->w;
```

```
#include <stdio.h>
struct rectangle
  {
   float l; /* length */
   float w; /* width */
   float A; /* Area */
   float P; /* Perimeter */
  };

void calcrect(struct rectangle *);

void main()
{
   struct rectangle rect;
   /* -------------------Introduction------------------ */
   printf("This program calculates the area and perimeter\n");
   printf("of a rectangle after you enter its dimensions.\n");
   printf("When asked to, type in a dimension and hit the ENTER key.\n");
   printf("\n");
   /* -------------------Inputting Data------------------*/
   printf("What is the length of the rectangle? ");
   scanf("%f", &rect.l);
   printf("What is the rectangle's width? ");
   scanf("%f", &rect.w);
   /* -------------------Calculation------------------ */
   calcrect(&rect);
   /* -------------------Printing Results------------------ */
   printf("\n");
   printf("The area of a rectangle with a length of %.2f units and\n", rect.l);
   printf("a width of %.2f units is %.2f square units.\n", rect.w, rect.A);
   printf("Its perimeter is %.2f units.\n", rect.P);
}

void calcrect(struct rectangle *r)
{
   r->A = r->l * r->w;
   r->P = 2 * (r->l + r->w);
}
```

Pointer to structure

Just as we have pointer to integer, pointer to array etc. We can also get pointer to a structure using the '&' operator. Again taking the struct emp, we define a variable e1 of type struct emp
struct emp e1;
&e1 gives the starting address of the structure. The starting address of structure is depicted below.



If we have a variable
Data-type var
then

data-type *ptr_var gives pointer to the same type.
ptr_var = &var assigns address of 'var' to pointer variable.

Similarly
struct emp e1;
defines a variable e1 of struct emp type
struct emp *ptr_emp defines a pointer to struct emp type.
ptr_emp = &e1 makes ptr_emp to point to e1

Variable and pointer variable declaration can be combined together with structure format definition as

```
struct emp {
        .
        } e1, *ptr_emp;
        ptr_emp = &e1;
```

How do we access members of structure if we have a pointer to structure?
ptvar -> member

where ptvar is a structure type pointer variable and the operator -> is comparable to (.)period operator.
'.' operator requires a structure variable, while -> operator requires a structure pointer on its left side.

Let us see how we can access individual members of structure using the structure pointer. The following table gives how we can access different types of members of a structure. i.e. how we access the member if it is an ordinary variable, array or a structure itself.

SrNo.	Member	Structure variable	Structure pointer
1.	Ordinary Variable	var. member e1.emp_no	ptvar->member (ptr_emp ->emp_no) **(*ptr_emp).emp_no**
2.	Array	var.member[expression] e1.name	ptvar->member[expression] ptr_emp->name
3.	Structure	var.member.submember e1.a.pin	ptvar->member.submember ptr_emp->a.pin (*ptr_emp).a.pin

Since **ptr_emp is a pointer to structure, (*ptr_emp). Salary gives contents of member salary.**
The parenthesis are necessary because precedence of structure member operator '.' Is higher than '*'.
The expression *ptr_emp.emp_no is illegal as emp_no is not a pointer

Passing structure as parameter

- Individual structure members.
- Structure as a whole.
- Pointer to structure.

Structure can be passed to a function as parameter and can also be returned from functions using three approaches. Either individual members of structure, whole structure or pointer to structure can be passed.

Individual structure members

Since individual structure members can be accessed and act as individual entity, they can be passed and returned from functions.
Following example illustrates passing and return of individual member of structure.

Program demonstrating passing and return of individual member of structure

```
#include<stdio.h>
main ( )
{
float arriers (char *s, int n, float m);
typedef struct emp
    {
            char name[15];
            int emp_no;
            float salary;
    }record;
record e1 = {"smith",2,20000.25};
    e1.salary = arriers(e1.name,e1.emp_no,e1.salary);
}
float arriers(char *s, int n, float m)
  {
        m = m + 2000;
        printf("\n%s    %d      %f   ",s, n, m);
        return  m;
}
```

Output
smith 2 22000.250000

In the declaration of structures, 'name' is declared as an array. Since we are passing the base address of name, the value is stored in emp_no and salary. it is a combination of call by reference as well as call by value. The salary is incremented by 2000 and returned to function main. In main it is assigned to e1. salary.

Thus it is seen that passing individual members to and from functions is analogous to ordinary variables.

Structures as a whole

When number of members of a structure is more and we need to pass all of them, individual member approach becomes tedious and lengthy. Instead we can pass structure as a whole. The following program illustrates passing entire structure to function and what is the effect of it.

```
/* program demonstrating passing entire structure to function.
structure not returned from function */
typedef struct  {
                char name [15];
                int emp_no;
                float salary;
                }record ;
#include<stdio.h>
# include<string.h>
main( )
{
                void change (record);
                record e1 =  {"smith", 2, 20000.25};
                printf("\n%s     %d       %f",e1.name,e1.emp_no,e1.salary);
                change(e1);
                printf("\n%s     %d       %f",e1.name,e1.emp_no,e1.salary);
}
```

```
void change (record e2)
{
                strcpy(e2.name,"Jones");
                e2.emp_no = 16;
                e2.salary = 9999;
                return;
}
```

Output
smith 2 200000.250000
smith 2 200000.250000

The function 'change' accepts structure as an argument and assigns different values to each member of structure. Function returns nothing. In main, structure is initialized to some value. Value of each member is printed before calling function change and after calling function change. There is no difference between values before and after call to main. It thus shows that transfer is by value.

One more thing to be noted is that here structure format is defined outside main. If it is defined in main like previous example, function 'change' would not know what is struct emp. Hence its definition is made global. Thus while passing structure as a whole, it is necessary to make the structure definition known to the called as well as calling program.

Since structure can be returned from a function, (as against arrays which cannot be returned from a function) we can modify the program so that altered structure is returned from 'change' to main.

```
/* Modified to return structure from called function */
typedef struct
{
                char name [15];
                int emp_no;
                float salary;
} record;
#include<stdio.h.>
#include<string.h>
void main ( )
{
                record change (record);
                record e1 = {"Smith", 2, 20000.25};
                printf ("\nBefore Change %s      %d      %f ",e1.name,e1.emp_no,e1.salary);
                e1 = change(e1);
                printf ("\nAfter Change %s      %d      %f ",e1.name,e1.emp_no,e1.salary);
}
record change (record e2)
{
                strcpy (e2.name, "Jones");
                e2.emp_no = 16;
                e2.salary = 9999;
                return e2;
}
```

Output
Smith 2 20000.25
Jones 16 9999.99

Thus the alterations made in function change are now available to main.

Passing pointer to structure

When large structures are to be passed to functions it is better to pass pointers to structures. Also passing pointer to structure makes altered values available to calling program. So we don't need to return structure as in previous example.

If we write the previous program by passing pointer to struct emp

```
/* Rewritten using pointer to structure, pointer to structure is passed instead of whole
structure  */
typedef struct emp
{
            char name [15];
            int emp_no;
            float salary;
            }record;

#include<stdio.h>
# include<string.h>
main ( )

{
                void change(record *);
                record e1 = {"Smith", 52, 8250.50};
                record *e2;
                e2 = &e1;
                printf ("\nBefore Change %s      %d          %f",e1.name,e1.emp_no,e1.salary);
                change (e2);
                printf ("\nAfter  Change %s      %d          %f",e2->name,e2->emp_no,e2->salary);
}
void change(record *pt)
{
               strcpy (pt->name, "Jones");
               pt->emp_no = 16;
               pt->salary = 9999;
               return;
}
```

We can also return pointer to a structure from a function. This is mainly useful when several structures are passed to function but only one is returned.

Exercise

Ask for emp_no from user and display entire record for that employee. Store employee record as array of structures and pass it to function and return matched record as pointer to structure.

Unions

- What is a union
- Using unions

Let us now turn our attention to unions which are very much similar to structures where both are used to group number of different variable elements. We will study what are unions how they are different from structures and how they are used.

What is Union

The composition of union may be defined as

```
union  tag  {
                member 1;
                member 2;
                 ............
                member 3;
                 };
```

This is similar to structure. The only difference being the keyword 'union' instead of 'struct'. Individual union variables can be declared as

```
storage-class union tag var1, var2, .........varn;
```

The combination gives

```
storage-class Union tag
 {
                member 1;
                member 2;
                 ............
                member n;
 }var1, var2,      .........varn;
```

The tag is optional here
The members of union are accessed in the same way as structure members are accessed, using the '.' operator or ->operator
union-name.member
union-name->member

The difference between structure and union lies in the functionality and hence the purpose of using them. If we have a structure

```
struct    id   {
                  char color [12];
                   int size;
                   }shirt;
```

A total of 12 bytes for character array and 2 bytes for integer =14 bytes would be allocated to above structure. Any member of the structure can be accessed at any time. A union is used for application involving multiple members where values of only one member need to be used at any one time. If we need a value of either size or color at a time but never both at a time, unions are used. Eg A shirt is to be identified by either size or color.

```
union      id     {
                     char color[12];
                      int size;
                    } shirt;
```

The space allocated to any union is large enough to hold largest of all types. Here 12 bytes, since char array is largest of the two given types. The same 12 bytes are used by the char array as well as integer, thus saving space. Well the programmer has to keep track of which union member is currently being used at any step.

Using unions

A simple program using unions follows;

```
#include<stdio.h>
main( )
{
        union id
                {
                    float cost;
                    int size;
                };
        struct  sid
                {
                     char manufacturer [20];
                     union  id description;
                } shirt;

printf("\nspace assigned to union =  %d  bytes ", sizeof(union id) );

  shirt.description.cost = 500;

printf("\ncost=  %f  \t  size =  %d", shirt.description.cost,shirt.description.size);

 shirt.description.size = 12;

printf ("\ncost = %f  \t  size =  %d", shirt.description.cost,shirt.description.size);
 }
```

output :
space assigned to union = 4 bytes.
cost = 500.799 size = 26711
cost = 00.0000 size = 12

We have defined a union and a structure, one of whose member is union id. First we have found out the memory space allocated to the union using the size function. It is seen from the output that 4 bytes are allocated to union, since cost variable is of 4 bytes. Then value is assigned to cost, member of union. From the output it is seen that at that time 'size' contains no reasonable value.

Just like nested structures, we can have nested unions and union in structures (as in previous example). Also arrays of unions and pointer to union can be used. Pointer to union can be passed to and from functions. Unions can be initialized but with only one value. The operations performed on unions are same as on structures like accessing union members, taking address, assignment of entire union variable to other union variable.

Bitfields

- What are bitfields?
- Using bitfields.

When the question of saving memory is at the priority, we can group variables requiring a few bits as a single word (i.e. single integer), instead of defining each variable as an integer or character.
This can be done using bitfields

What are bitfields?

One-bit can give values (true/false). Value of 2-bits can range from 0 to 4. Value of 3-bits can range from 0 through 7. Several such data items can be combined into an individual word of memory. Such data items are called bit-fields are defined as members of structure.

In general

```
struct tag
{
        member 1 :  size;
        member 2 :  size;
          ……….
        member m :  size;
};
```

Each member name is followed by a semicolon and an unsigned integer indicating field size.
Variable definition and accessing each member is same as ordinary structure.

Using bitfields

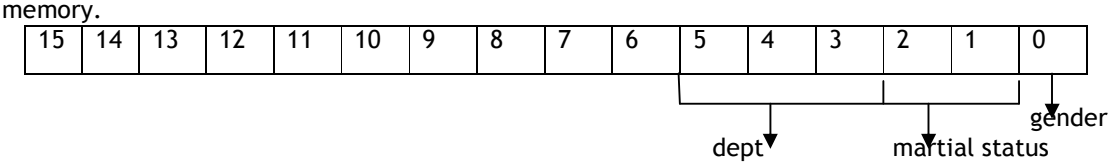
If we have to store the information of an employee like:

- He/she can be male or female. One-bit will be sufficient for the purpose.
- Can be single, married, divorced or widowed. two-bits are sufficient.
- Can belong one of 8 different departments. three bits will do

Thus we declare the structure as

```
Struct  emp
            {
            unsigned gender : 1;
            unsigned mar_status : 2;
            unsigned dept : 3;
            };
```

The number after the colon specifies the size of each bit-field. It is defined as a structure, which is subdivided into 3 bit fields. They have width of 1, 2 and 3. Hence they occupy 6 bits within a word of memory.



Let us write a small program using the given bitfield structure

```
//program demonstrating use of bitfields
#include<stdio.h>
#define MALE 0
#define FEMALE 1
#define MARRIED 2
#define DIVORCED 1
#define WIDOWED 2
#define RND 0
main( )
{
        struct emp
        {
        unsigned gender : 1;
        unsigned mar_status : 2;
        unsigned dept : 3;
        };
        struct emp e;
        e. gender = MALE;
        e. mar_status = DIVORCED;
        e.dept = RND;
        printf ("\nGender   = %d  \t  Marital Status = %d  \t  Department = %d",
        e.gender,  e.mar_status,  e.dept);
        printf ("\n requires : %d bytes", sizeof(e) );
}
```

Output
Gender =0 Marital Status =1 Department =0
Requires: 4 bytes

Values are assigned to bitfields using #defines. Bitfields can also be initialized. Bitfields can appear in arithmetic expressions.
Arrays of bitfields are not allowed. The (&) address operator cannot be applied to a bitfield, pointer cannot access a bitfield and a function cannot return a bitfield.

A bitfield without a name can be used for padding.

```
struct xyz {
        unsigned a :  5;
        unsigned b :  5;
        unsigned    :  6;
        unsigned c :  5;
            };
```

0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
c											b					a

A bitfield can be forced to start at a beginning of a new word by specifying unnamed bitfield of width 0.

```
Struct xyz {
        Unsigned a  :  1;
        Unsigned b  :  2;
        Unsigned    :  0;
        Unsigned c  :  3;
            };
```

0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
C															b	a

Review Questions

1. Which of the following accesses a variable in structure b?
A. b->var;
B. b.var;
C. b-var;
D. b>var;

2. Which of the following accesses a variable in structure *b?
A. b->var;
B. b.var;
C. b-var;
D. b>var;

3. Which of the following is a properly defined struct?
A. struct {int a;}
B. struct a_struct {int a;}
C. struct a_struct int a;
D. struct a_struct {int a;};

4. Which properly declares a variable of struct foo?
A. struct foo;
B. struct foo var;
C. foo;
D. int foo;

Summary

- Structure is a collection of data items of different data-types.
- The individual members of a structure can be our usual data-type(int, char..) or arrays or pointers or other structures.
- The names of all members within a structure should be distinct. They can be same as any other variable outside the structure.
- Each member of a structure is stored beside each other.
- The arrays of structure are defined and operations on this array are performed just like our usual array of other data types. Structure can be passed to a function as parameter and can also be returned from functions. When large structures are to be passed to functions it is better to pass pointer to structures.
- Using typedef we can assign a new name to an existing data-type or a complex data type.
- A union is used for application involving multiple members, where values of only one member need to be used at any one time.
- We can group variable requiring a few bits as a single word (i.e. single integer), instead of defining each variable as an integer or character.

Programming Concepts using C

Chapter 11: Files

Objectives

- Discuss the file basics.
- Use file opening and closing functions.
- Use command line parameters.
- Use character as well as string based functions.
- Use command line parameters.
- Use formatted I/O functions.
- Compare binary and text files.

Introduction

So far we have taken input from the user and displayed output on the screen. I.e. we have considered console I/O. Many applications require that information be stored permanently and it should be possible to access and alter the information whenever necessary. In such cases information can be stored in files. 'C' provides a number of functions for using files.

In this chapter we will study types of files i.e. text files and binary files. How files are opened and closed and how they are operated on & different ways in which information is stored and retrieved from files. We will study, how parameter can be passed to program on command line. We will also see how we can access record from files at random.

File handling in 'C'

- File basics.
- 'C' language support for files.

Right at the outset, let us see, what exactly is meant by files and how 'C' language facilitates file handling. We will also discuss about buffer and its use in relation to a file.

File basics

Files are divided basically into two types:

- Stream- oriented (standard or high-level)
- System- oriented (or low level)

Stream oriented data files are easier to work with than system-oriented data-files and thus more commonly used.

Numbers of library functions are used to operate on files which are broadly classified as

- High level file I/O functions
- Low level file I/O functions

High level file I/O functions do their own buffer management whereas in low level file I/O functions buffer management has to be done by the programmer.
Now, what is a buffer? A buffer is a block of memory where the data to be stored in file is placed temporarily.

E.g.

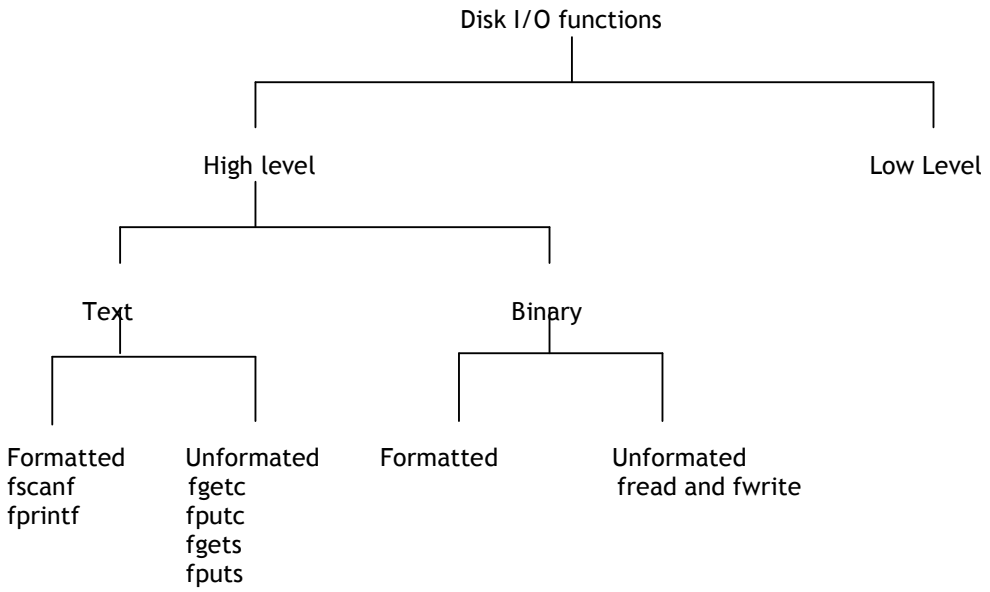
If an user enters characters to be written to a file, through the keyboard, these characters are not directly written to the file but are temporarily stored in buffer. When buffer is full, disk is accessed and contents transferred to the file on disk. Or when end of file is reached buffer contents are transferred to disk file.

What will happen if data is directly send to disk file instead of using buffer? Since accessing disk involves mechanical movements, if disk has to be accessed for each character, it will take a lot of time. Thus buffer allows information to be read from and written to data files more rapidly.

High level file I/O functions are further categorized into text and binary, which are in turn categorized into formatted and unformatted.

C-language support for files

'C' language does not give any facility for input and output. Each operating system has its own facility for inputting and outputting data from and to files and devices. The system programmer writes small programs that link the 'C' compiler for particular operating systems I/O facility.
There are several standard I/O functions available in libraries. The following figure gives a clear picture.



Text files

- Opening and closing a file.
- Character based I/O functions.
- Using command line parameters.
- String based I/O functions.
- Formatted I/O functions.

Let us now study text files in detail, we will see how text files are opened and closed. We will see function for reading and writing characters and strings in a file. We will also see formatted I/O functions and learn the command line parameters.

Opening and closing a file

- File is not opening.

Before opening a file we need to establish a file pointer.

```
FILE *fptr;
```

FILE is a structure defined in the header file "stdio.h". This header files stores all the information regarding the file i.e. name of the file, in what mode it is opened, starting buffer address, a character pointer which points to the character being read. The above statement establishes a buffer area. Each file opened has its own file structure. Before processing any file, it must be opened.
On opening a file link is established between the program and operating system through the FILE structure, since operating system returns a pointer to the structure FILE. An opening a file associates the file name with buffer area. The file opening function is written as

```
fptr = fopen(file-name, file-type);
```

file-name = Name of data file to be opened.
File-type = Mode or manner in which data file will be used
(i.e. whether for reading, writing etc).

The following table gives the different modes in which a file can be opened.

File-type	Meaning
" r"	Open an existing file for reading only.
"w"	Open a new file for writing only. If a file with the specified file-name currently exists, it will be destroyed and a new file created in its place.
"a"	Open an existing file for appending (i.e. for adding new information at the end of the file.) A new file will be created if the file with the specified file-name does not exist.
"r+"	Open an existing file for both reading and writing.
"w+"	Open a new file for both reading and writing. If a file with the specified file-name currently exists, it will be destroyed and a new file created in its place.
"a+"	Open an existing file for both reading and appending. A new file will be created if the file with the specified file-name does not

When we have finished processing the file we need to close it explicitly. The syntax is fclose(fptr);
If fclose is not present in the program, compiler automatically closes it at the end of the program execution.
The following program skeleton shows opening and closing file statements. The in between file processing statements are not included.

```
#include<stdio.h>
main ( )
{
FILE * fptr;
        fptr = fopen ("sample.dat", "r");
         ............
         ............
        fclose (fptr);
}
```

The header file "stdio.h", must be included for it makes available the FILE structure. FILE * fptr defines fptr, as a pointer which points to structure of type FILE. The file opening statement searches for the file 'sample.dat' on disk. If it is present, it loads the file from the disk into memory in the read mode and returns a pointer to FILE and assigns it to fptr. It sets up a character pointer, which points to first character of the chunk of memory where the file has been loaded.
The last statement closes the data file. On closing a file the links between the file and the associated buffer are freed so that they can be used by other files.
It is seen that once the file is opened, all the file processing and file closing functions use the file pointer and not the name of the file.

File is not opening

There may be problem that file does not get opened. It can be for various reasons. If we need to open a file for reading, the file may not exist. If we want to open a file for writing, disk space may be insufficient.
In any case, if file opening function fails it returns a NULL which is defined in "stdio.h" as #define NULL 0

It is necessary to check whether file is successfully opened. It can be done as given below.

```
fptr = fopen ("sample.dat", "r");
if (fptr = = NULL)
{
    puts ("\nCannot open file");
    exit ( );
}
```

Character based I/O functions

Before using any file it is necessary to create the file. It can be created by

- using any text editor directly
- writing a program, to write into the file.

Program will consist of accepting character input from user and writing to the file using file processing function 'fputc'.
Any existing file or file created in the above manner can be read in any of the following way

- Directly by using operating system command such as print or type.
- Using the text editor or word processor.
- A program can be written to read the file.

In the program function fgetc reads characters from data file and putchar will display them on the screen. Individual data characters can also be processed as they are read.
Thus fputc is a function used to write data character in a file.

int fputc(int ch, FILE *ptvar);
ch = Character to be written.
ptvar = File pointer.
fgetc is the counterpart of the function fputc and it is used to read a character from the file. It returns the character read to the calling program.

int fgetc(FILE * ptvar);
Let us write a program that copies the contents of one text file into another text file, character by character.

Example(1)

```
#include<stdio.h>
#include<process.h>
void main ( )
{
        FILE *ft,*fs;
        char  ch;                                   /*step 1*/
        fs = fopen("c:\\lnt\\files\\pr1.txt","r");  /*  step 2*/
        if(fs==NULL)
        { puts ("\nCannot open source fill");
          exit(1);
        }
         ft = fopen("c:\\lnt\\files\\pr2.txt","w");   /* step 3 */
         if(ft==NULL)
         { puts("\nCannot open target file");
         fclose(fs);
         exit(1);
         }
           while(1)
            {   ch = fgetc(fs);                     /* step 4 */
                if(ch==EOF)
                break;
                else
               fputc(ch,ft);
             }
fclose(fs);
fclose(ft);
}
```

Let us study the program.
First line include the header file stdio.h in the program

Step 1 : defines two file pointers.

Step2 : opens the file pr1.c in the read mode and assigns the file pointer returned, to fs. Thus fs now points to the source file structure i.e. file which is to be read. The next if-block checks if fopen is successful, if not the program exist using the exit() statement.

Step 3 : opens the file pr2.c in the write mode and assigns the returned file pointer to ft. ft thus points to the target file.

Next if-block checks if fopen is successful, if not the source file is closed (since it has been opened previously) and the program is terminated using exit()
The main action is in the while loop. while(1) runs the while loop forever.

Step 4 : reads character from source file one by one and assigns to ch. If ch has a value of EOF, break causes control to be passed out of the while loop else fputc writes the character ch to the target file. At the end both files are closed with the function fclose.

What is 'EOF'? When end of file is reached, fgetc() encounters a special character, which is returned as EOF. EOF is a macro defined in file stdio.h. EOF is given by a special character, whose ASCII value is 26. When file is created this character is inserted beyond the last character in the file. It can be retrieved from the keyboard by typing ctrl Z.

Using command line parameters

We have seen in the topic on function that main itself is a function. Till now we have always kept the brackets after main as empty. Yes, argument can also be passed to main, as we pass argument to other functions. The parenthesis of main may contain special arguments that allow parameters to be passed to main from the operating system.

There are two arguments

argc = It is an integer which contains the count of, number of parameters passed to a program.
argv = It is an array of pointers to string, which contain all the parameters passed. The name of program occupies the place of argv[0] followed by the parameters.

```
main (int argc, char *argv)
{
. . .

}
```

Consider the previous program of copying one file to another. If the name of source code is filecopy.c then name of executable file is filecopy.exe. The source file and target files are either kept fixed within the program or they are accepted from the user while the program is running. But using command line parameters, we can pass the source file and the target file names as command line parameters. Since we pass the argument to main at the command prompt, they are called command line parameters.
Eg.
program can be run from the command prompt as
C:> FILECOPY pr1.c pr2.c

All the string at the command line are stored in memory and argv points to the first string which is name of the program.

argv[0] = filecopy
argv[1] = abc.c
argv[1] = abc.c
and argc is set to number of string. Here argc =3.

Using command line parameters the program in example (1) can be written as

Example(2)

```
#include<stdio.h>
main (int argc, char *argv)
{
FILE *fs,*ft;
char  ch;

          if(argc!=3)
           {
               puts("\n Insufficient  arguments");

            }
            fs=fopen("E:\\abc.txt","r");
            if(fs==NULL)
```

```
            {
               puts ("\nCannot open source file");

            }
            ft=fopen("E:\\abc.txt","w");
            if  (ft==NULL)
             {
                puts("\nCannot  open target file");
                fclose(fs);

              }
              while(1)
              {
               ch = fgetc (fs);
               if (ch == EOF)
                   break;
               else
               fputc(ch,ft);
               }
fclose(fs);
fclose(ft);
}
```

Since we require source file and target file names compulsorily, it is checked whether all arguments are passed to the program. In (argc != 3) structure, if number of arguments are insufficient program is exited. Rest of the program remains the same except the names of source file and target file are replaced by argv[1] and argv[2].

If program name is followed by n parameters on the command line, argc will be set to (n + 1) and argv will have (n + 1) entries, index ranging from 0 to n. argc and argv are not assigned values explicitly as they are automatically initialized. If no parameter is passed to program, argc = 1 and argv[0] will contain name of program.(exe file). argv should have NULL as the ending element.

String based I/O

In some applications accessing files character by character is sufficient. Some files can be created and read more easily with programs that utilize string oriented library functions.
'C' provides two string functions for reading and writing to files.

fgets = It is used for reading a string from a file (including newline). Its general format is
char *fgets(char *line, int maxline, FILE *fp);

line = Address of array of characters where string is stored.

maxline = Maximum length of string. This argument prevent array from overflowing. At the most (maxline – 1) characters will be read.
fp = It is the file pointer.
fgets return a line from the file on end of file or error it returns NULL.

fputs()

It is used for writing a string to a file. fputs does not automatically add a newline character to the end of the string. Its general format is

int fputs (char * line, FILE *fp);
line = It is the string which is to be written to the disk.
fp = It is the file pointer.

It returns EOF if error occurs otherwise last character is written.

fgets and fputs are analogous to gets and puts.

C File I/O and Binary File I/O

When accessing files through C, the first necessity is to have a way to access the files. For C File I/O you need to use a FILE pointer, which will let the program keep track of the file being accessed. (You can think of it as the memory address of the file or the location of the file).

For example:

FILE *fp;
To open a file you need to use the fopen function, which returns a FILE pointer. Once you've opened a file, you can use the FILE pointer to let the compiler perform input and output functions on the file.

FILE *fopen(const char *filename, const char *mode);
In the filename, if you use a string literal as the argument, you need to remember to use double backslashes rather than a single backslash as you otherwise risk an escape character such as \t. Using double backslashes \\ escapes the \ key, so the string works as it is expected.

The modes are as follows:

r - open for reading
w - open for writing (file need not exist)
a - open for appending (file need not exist)
r+ - open for reading and writing, start at beginning
w+ - open for reading and writing (overwrite file)
a+ - open for reading and writing (append if file exists)

Note that it's possible for fopen to fail even if your program is perfectly correct: you might try to open a file specified by the user, and that file might not exist (or it might be write-protected). In those cases, fopen will return 0, the NULL pointer.

Here's a simple example of using fopen:

```
FILE *fp;
fp=fopen("c:\\test.txt", "r");
```

This code will open test.txt for reading in text mode. To open a file in a binary mode you must add a b to the end of the mode string; for example, "rb" (for the reading and writing modes, you can add the b either after the plus sign - "r+b" - or before - "rb+")

To close a function you can use the function

int fclose(FILE *a_file);
fclose returns zero if the file is closed successfully.

An example of fclose is
fclose(fp);

To work with text input and output, you use fprintf and fscanf, both of which are similar to their friends printf and scanf except that you must pass the FILE pointer as first argument. For example:

```
FILE *fp;
fp=fopen("c:\\test.txt", "w");
fprintf(fp, "Testing...\n");
```

It is also possible to read (or write) a single character at a time--this can be useful if you wish to perform character-by-character input (for instance, if you need to keep track of every piece of punctuation in a file it would make more sense to read in a single character than to read in a string at a time.) The fgetc function, which takes a file pointer, and returns an int, will let you read a single character from a file:

```
int fgetc (FILE *fp);
```

Notice that fgetc returns an int. What this actually means is that when it reads a normal character in the file, it will return a value suitable for storing in an unsigned char (basically, a number in the range 0 to 255). On the other hand, when you're at the very end of the file, you can't get a character value--in this case, fgetc will return "EOF", which is a constant that indicates that you've reached the end of the file. To see a full example using fgetc in practice, take a look at the example here.

The fputc function allows you to write a character at a time--you might find this useful if you wanted to copy a file character by character. It looks like this:

```
int fputc( int c, FILE *fp );
```

Note that the first argument should be in the range of an unsigned char so that it is a valid character. The second argument is the file to write to. On success, fputc will return the value c, and on failure, it will return EOF.

Binary files

- Comparison(Text Vs Binary)
- Block I/O using binary mode

Now will will study binary files and see in what way text files differ from binary files also we will see how to transfer data blocks to and from files as a whole.

Comparison (Text Vs Binary)

- How they handle newline characters.
- How do they indicate end of file.
- How they store numbers on disk.

They are mainly three differences between text mode and binary mode files; we will discuss each in detail.

For binary File I/O you use fread and fwrite.

The declarations for each are similar:

```
size_t fread(void *ptr, size_t size_of_elements, size_t number_of_elements, FILE *a_file);

size_t fwrite(const void *ptr, size_t size_of_elements, size_t number_of_elements, FILE *a_file);
```

Both of these functions deal with blocks of memories - usually arrays. Because they accept pointers, you can also use these functions with other data structures; you can even write structs to a file or a read struct into memory.

Let's look at one function to see how the notation works.

fread takes four arguments. Don't by confused by the declaration of a void *ptr; void means that it is a pointer that can be used for any type variable. The first argument is the name of the array or the address of the structure you want to write to the file. The second argument is the size of each element of the array; it is in bytes. For example, if you have an array of characters, you would want to read it

in one byte chunks, so size_of_elements is one. You can use the sizeof operator to get the size of the various datatypes; for example, if you have a variable int x; you can get the size of x with sizeof(x);. This usage works even for structs or arrays. Eg, if you have a variable of a struct type with the name a_struct, you can use sizeof(a_struct) to find out how much memory it is taking up.

e.g.,

sizeof(int);

The third argument is simply how many elements you want to read or write; for example, if you pass a 100 element array, you want to read no more than 100 elements, so you pass in 100.

The final argument is simply the file pointer we've been using. When fread is used, after being passed an array, fread will read from the file until it has filled the array, and it will return the number of elements actually read. If the file, for example, is only 30 bytes, but you try to read 100 bytes, it will return that it read 30 bytes. To check to ensure the end of file was reached, use the feof function, which accepts a FILE pointer and returns true if the end of the file has been reached.

fwrite() is similar in usage, except instead of reading into the memory you write from memory into a file.

For example,
```
FILE *fp;
fp=fopen("c:\\test.bin", "wb");
char x[10]="ABCDEFGHIJ";
fwrite(x, sizeof(x[0]), sizeof(x)/sizeof(x[0]), fp);
```

How they handle newline characters

'C' considers newline as a single character '\n'. On the other hand, DOS converts it into two characters linefeed(\r) and newline(\n).
Thus when a file is opened in text mode and a 'C' program writes a 'C' text file to disk DOS converts all newline into carriage return – linefeed combination. And when a 'C" program reads in a text file, the carriage return linefeed combination are converted to a single newline character.
When number of character in a file are counted, then the count will be less than the number of characters (i.e. filesize) counted by DOS.
Such a discrepancy does not arise when file is opened in binary mode, because these conversions will not take place. And the file size will go hand in hand with the DOS operating system.
Till now we have seen how to open file in text mode. For opening a file in binary mode, only a 'b' is to be appended to the file open mode.
```
FILE *fp;
Fp = fopen("xyz.dat","rb");
```
For text mode we just write "r" or "rt", since text mode is the default mode.

How do they indicate end of file ?

In text mode, a special character whose ASCII value is 26 is automatically inserted at the end, to indicate end of file. One can read a file till this mark is obtained to read the entire file.
But in case of binary mode, there is no end marker. The directory is used to get number of characters present in the file and this count is used.
One important thing to be noted is that, if a file is written by opening in binary mode and we store a number 26(Hex 1A) in it and then we text mode to read the file, file will be terminated before hand since it will encounter its special character 26 ahead of actual end of file. Thus it is seen that binary and text modes are not compatible. A file written in binary mode should be read in binary mode while a file written in text mode should be read in text mode.

How they store numbers on disk

How are characters and number stored in memory?
A character occupies 1 byte
An integer occupies 2 bytes.
A float occupies 4 bytes. And so on.
When we write numbers using text mode (using fprintf), they are stored as string of characters.
i.e. If we stores an integer 1234, in memory, it will occupy 2 bytes. While in a file it would occupy 4 bytes i.e. one byte per character.
Similarly if we have a floating point number 1234.56, in memory, it will occupy 4 bytes.
Written to file it will occupy 7 bytes.
Thus number of bytes are equal to number of digits in the number. And as number of digits increase, space required increases.
When we use binary mode, fwrite function stores the numbers in binary format. Binary format means each number occupies same number of bytes on disk as it occupies in memory. i.e. 1234 will occupy 2 bytes on disk while 1234.56 will occupy 4 bytes on disk with binary mode.

Random access

Till now we have read from a file and written to a file. But all operations were performed serially. Apart from just reading the records and writing them, we may need to process the records. It may be desirable to add new records to the file (either at the end or in between somewhere), delete existing records, and modify contents of existing records or to rearrange the record. The operations can be performed with ease if we can access the file at random.

We know that
fread, reads the record where the pointer is currently placed and advances pointer by one record.
fwrite, writes the record where the pointer is currently placed.

We have a few functions which help us access the file records at random.
rewind (fpt) :-This function places the pointer to the beginning of the file irrespective of where it is present right now.

void rewind(FILE *fp);

ftell(fpt) = This function tells where the pointer is positioned right now. It returns this pointer as a long integer which is an offset from the beginning of the file. returns -1L on error
long ftell(FILE * fp);
fseek = This function lets us move the pointer from one record to another.
fseek(FILE *fp, long offset, int reference)
The first argument is the file pointer. Second argument is the offset which tells the compiler by how many bytes should the pointer be moved from a particular position. And the third argument is the reference from which the pointer should be offset. It can be

SEEK_END: move the pointer from end of file.
SEEK_CUR: move the pointer w.r.t. current position.
SEEK_SET: move the pointer w.r.t. beginning of file.

fseek return 0 if successful or nonzero on failure.

To move the pointer to previous record from its current position, we can use
fseek(fpt, -recsize, SEEEK_CUR);
(-recsize)moves the pointer back by recsize bytes from the current position

fseek(fpt, -recsize, SEEK_END);

This fseek would place the pointer beyond the last record in the file. 0 is the offset.
fgetpos = Gets current file pointer position.

int fgetpos(FILE * fp, fops_t *pos)
Position stored in *pos can be passed to fsetpos to set the file pointer. Returns 0 on success otherwise nonzero.
fsetpos = Position the file pointer on file.
int fsetpos(FILE *fp, const_t *pos)

New position pointed to by pos is the value obtained by a previous call to fgetpos. On success return 0 on failure returns a nonzero value.

Review Questions

1) Which of the following are valid file opening modes (This question is multiple choice)
   a. r
   b. rb
   c. r+
   d. rt
   e. All from the options

2) Function f open returns FILE * Open a file, write the values of.
   a. True
   b. False

3) Which function is used to write contents to a file field by field?
   a. fputs()
   b. fscanf()
   c. fprintf()
   d. fwrite()

4) Is there a difference between the following two modes?
   "r+" and "w+"
   a. Yes
   b. NO

Summary

- The files can be used to store information permanently which can be accessed and altered whenever necessary.
- Library functions used to operate on files are classified as
- High level file I/O functions
- Low level file I/O functions
- A buffer is a block of memory where the data to be stored in file is placed temporarily.
- Before opening a file we need to establish a file pointer. Each file opened has its own file structure. Before processing any file, it must be opened. When we have finished processing the file we need to close it explicitly. It is necessary to check whether file is successfully opened.
- Arguments can also be passed to main (), using special parameters. Since we pass the arguments to main, at the command prompt, they are called command line parameters.
- fgetc and fputc are functions used to read a character and write a character from and to a file respectively. fgets and fputs are for reading and writing string to files.
- Text mode and binary and mode files differ in the way
    - They handle new line characters.
    - The way they indicate end of file
    - The way they store number on disk
- fread and fwrite are used to read from and write to file in binary mode.

Programming Concepts using C

Chapter 12: Linked List

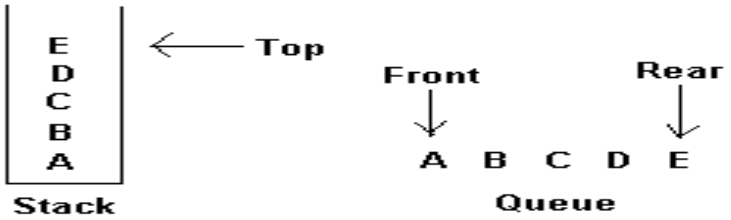
Objectives

- Discuss self-referential structures.
- Discuss linked list.
- Implement list operations.
- Creation.
- Insertion.
- Deletion.

Introduction to stack, Queue and link list

Two of the more common data objects found in computer algorithms are stacks and queues. Both of these objects are special cases of the more general data object, an ordered list.

A stack is an ordered list in which all insertions and deletions are made at one end, called the **top**. A queue is an ordered list in which all insertions take place at one end, the **rear**, while all deletions take place at the other end, the **front**.
Given a stack S=(a[1],a[2],.....a[n])
Then, we say that a1 is the bottommost element and element a[i]) is on top of element a[i-1], 1<i<=n.
When viewed as a queue with a[n] as the rear element one says that a[i+1] is behind a[i], 1<i<=n.



The restrictions on a stack imply that if the elements A,B,C,D,E are added to the stack, in that order, then the first element to be removed/deleted must be E. Equivalently we say that the last element to be inserted into the stack will be the first to be removed. For this reason stacks are sometimes referred to as **Last In First Out (LIFO) lists**. The restrictions on queue imply that the first element which is inserted into the queue will be the first one to be removed. Thus A is the first letter to be removed, and queues are known as **First In First Out (FIFO) lists**.

We have seen arrays and structures as means of storing data. We have known their limitations. Here we will study linked list as data structures. Linked list which is based on structures, self referential pointers and dynamic memory allocation have a lot of applications. Since here we use dynamic memory allocation, the memory available is the limit for storing data whereas in arrays the memory allocated is the limit.
It can be used in line editors to implement polynomials & to implement space matrices. It is used by operating system for memory allocation or to maintain job queues in multi-user system. Variable length string can be easily represented using linked list.
In this chapter we will see what is a **linked list**, how it is represented, how are nodes of a list created, & how are they inserted and how they are deleted.

Self-referential structures

In structures, members can be of any data type. If we include a member in the structure, which is a pointer to the parent structure type, such a structure is called self-referential structure.
In general

```
struct tag
 {
        member 1;
        member 2;
         ............
         ............
        struct tag *name;
 };
```

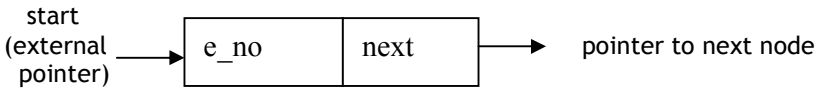
The structure of type tag contains a member, 'name' which is a pointer to structure of type tag .
Thus tag is a self referential structure.

List as a data structure

Self-referential data structures are useful in applications that involve linked data structures such as lists, trees and graphs. One structure will represent one component of the list. Each component of the list is called a node. The self-referential pointer indicates where the next component is located. Thus it links one component of list with another.

```
struct emp
 {
        int e_no;
        struct emp *next;
 } ;
typedef struct emp node;
node *start;
```

struct emp identifies a structure consisting of two members, an integer and a pointer (next) to another structure of same type. 'start' is an external pointer to struct emp. It will indicate the beginning of linked list.



Pointer to next node is assigned a value of NULL until next node is created.

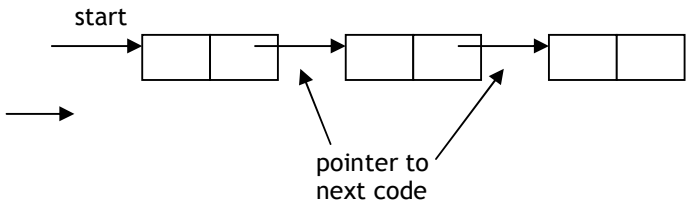
All operations on linked list, i.e. changing order of nodes, adding nodes, deleting nodes etc. can be done easily since only the pointer has to be altered.

There are several different kinds of linked data structures.

Linear linked list : In this components are linked together in sequential manner.
Doubly linked list : Here multiple pointers permit forward and backward traversal within the list.
Circular linked list : In which the last node points to the first node.
Trees : Here components are arranged hierarchically.

Here we will study linear linked lists.

Representation of linked list



Since 3rd node is the last node, NULL value is assigned to next node pointer. Thus NULL detects end of list.

Implementing list operations

- Creating a node.
- Inserting a node.
- Displaying a list.
- Deleting a node.

Whatever be the application which uses linklist, the basic operations are creating a node, insertion of a node, deleting a node and displaying the list. Here we explain the operations and write a function for each of them. At the end we write a program that combines all the operations on a list.

Creating a node

The basic thing is to create a node. It can be done by using the malloc function.

start = (node*) malloc(sizeof(node))

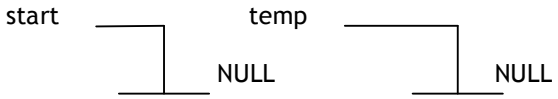
This statement creates the starting node of list. A block of memory whose size is equal to the sizeof(node) is allocated to the node pointer start. Typecasting is used because otherwise malloc will return pointer to character.

code snippet to create a node.

```
node*  create (node *start)
{
    node *temp;
    int n;
    start = temp = NULL;                                  ←———        step 1
/ * Accept e_no, till terminating condition reached.
 -1 entered for e_no can be decide to be terminating condition for loop * /
     printf ("\nEnter employee no (-1 to end) :  ");
     scanf ("%d", &n);
     while (n  ! = -1)
      {
         if (start = = NULL)
         {
              start = (node *) malloc (sizeof (node *) );      ←———        step 2 (a)
              start ->e_no = n;
              start ->next = NULL;
              temp = start;                                   ←———        step 2 (b)
           }
        else
          {
             temp->next = (node *) malloc (sizeof (node*) );  ┐
             temp->next ->e_no = n;                           ├——→        step 3
             temp->next->next = NULL;                         ←———        step 4 (a)
             temp = temp->next;      ←————————————————————————        step 4 (b)

/ * to link next node with previous list * /
          }
            printf ("\nEnter employee no (-1 to end): ");
            scanf ("%d", &n);
           } / * while * /
}
```

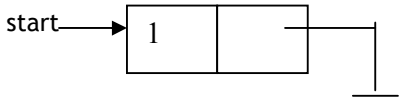
Self-referential structure emp is defined and is named as node. Two pointers, 'start' and 'temp' are defined and initially assigned a NULL value.

step 1

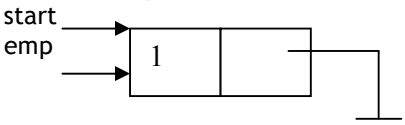


A value for e_no is accepted. e_no entered as -1, is decided to be the terminating condition for creating nodes. It is checked if starting node is to be created or further nodes are to be created. If value of start pointer is NULL, it means starting node does not exist and is to be created. Memory is assigned to start pointer. Value of e_no is assigned to its e_no field and self-referential pointer assigned a value of NULL.

step 2(a)

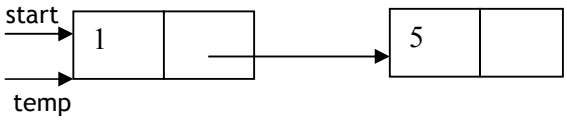


step 2(b)

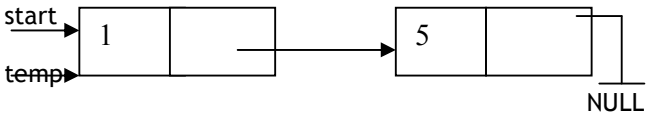


If value of start pointer is not null, it means start pointer exists and further node is to be created which is created in the same manner.

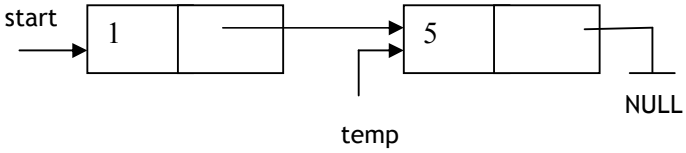
step 3



step 4(a)



step 4(b)



As new nodes are created start pointer is kept fixed while temp pointer is moved.

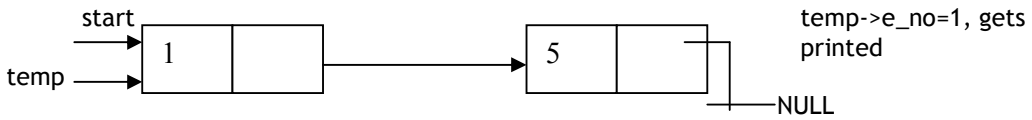
Displaying a list

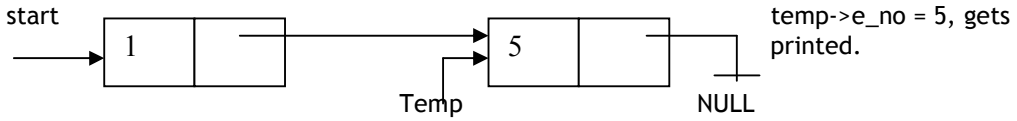
List can be displayed by beginning at the start pointer and traversing the list node till NULL is encountered.
Here also we use a temporary pointer which initially start with first pointer and gets assigned by further links till it becomes NULL.

Function to display the list follows.

code snippet for displaying nodes of link list

```
display (node* start)
{
    node *temp;
    temp = start;
    while(temp ! = NULL)
{
    printf("n%d",temp->e_no);
    temp = temp->next; /* go to next node * /
 }
}
```





Inserting a node

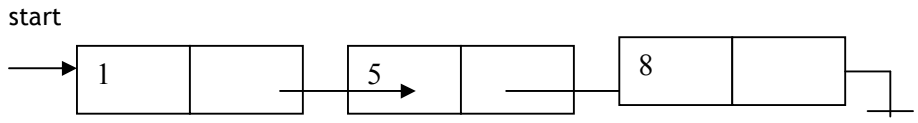
A node can be inserted into the list either before the first node (so that the new node becomes the first node) or in between anywhere in the list. If we write a function to insert, we will need the information as to where the node is to be inserted, by accepting an e_no, before which the node is to be inserted.

code snippet for insertion of node

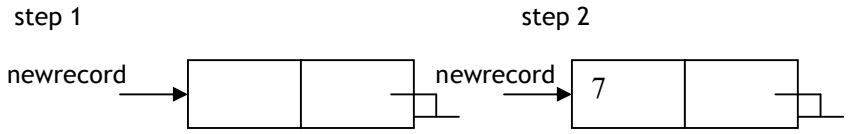
```
node* insert (node *start)
{
        int n, target;
        node *newrecord, *tag;
        printf ("\nNew employee number :");
        scanf ("%d", &n);
        printf ("\nPlace before :");
        scanf ("%d", &target);

        if (start ->e_no = = target)
         {
            / * new node is to be first in list * /
            / * allocate space for new node * /
            newrecord = (node *) malloc (sizeof (node) );          <---  Step 1

            / * assign new e_no to field in newrecord * /
            newrecord ->e_no = n;                                  <---  step 2

            / * assign current pointer to newrecord -> next  * /
            newrecord ->next = start;                              <---  step 3
            / * new pointer becomes the beginning of the list  * /
            start = newrecord ;                                    <---  step 4
         }
    else
        {
```

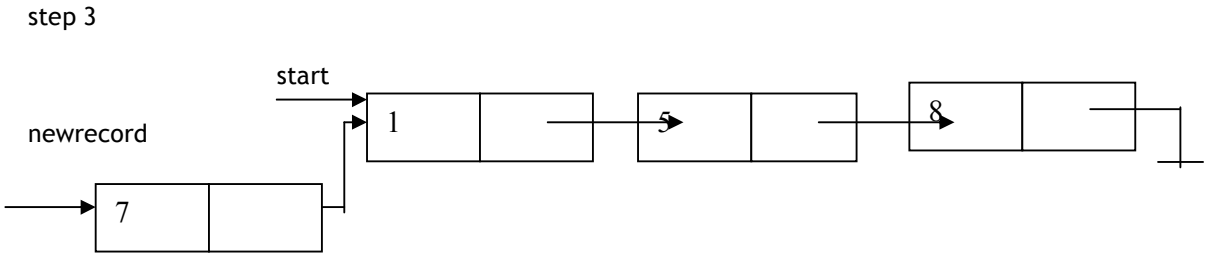
```
/ * insert new node after an existing node  * /
/ * locate node preceding the target node  * /
            tag  = locate (start, target);                    ←————        step a
            if (tag = = NULL)
                    printf ("\nMatch not found");
           else
            {
/ * allocate space for new node * /
            newrecord = (node *) malloc (sizeof (node) );                  step b
            newrecord ->e_no = n;
            newrecod ->next = tag ->next;                     ←————        step c
/ * assign newpointer to tag->next * /
            tag->next = newrecord;                            ←————        step d
            }
         }
            return (start);
}
node* locate (node* ptr, int target)
{
        if (ptr->next->e_no = = target)
             return (ptr);
         else
              if (ptr->next->next = = NULL)
                   return (NULL);
             else
                locate (ptr ->next, target);
}
```

A new pointer defined, newrecord. New employee number is accepted. Target is assigned the value of e_no before which new node is to be inserted.

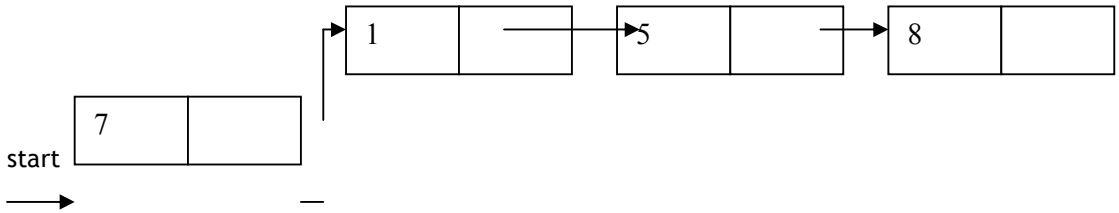


If target is 1 and new e_no = 7, then new node is to be inserted at the beginning of the list as shown diagrammatically.



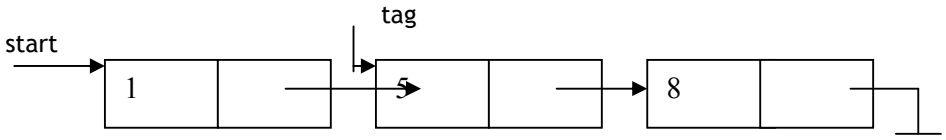


step 4

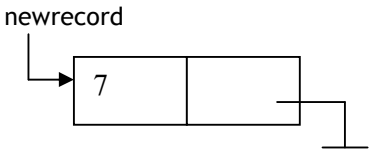


If target is 8 and new e_no = 7, then newrecord node is be inserted before node whose e_ no = 8. The function locate, locates the node before the target node and returns a pointer to it.

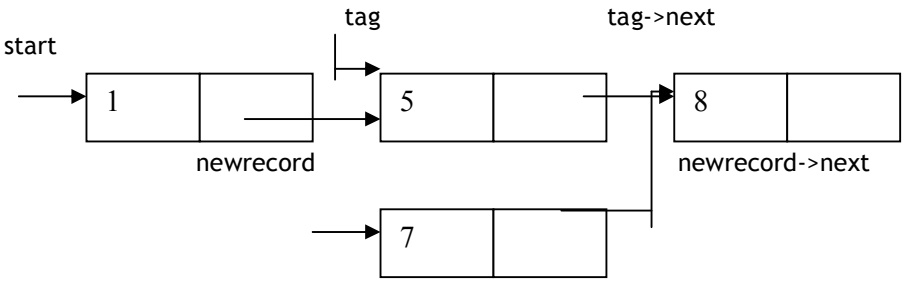
step a



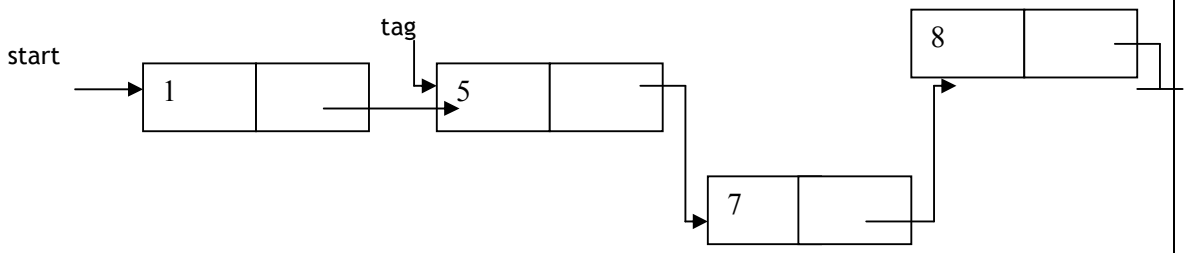
step b



step c



step d



Deleting a node

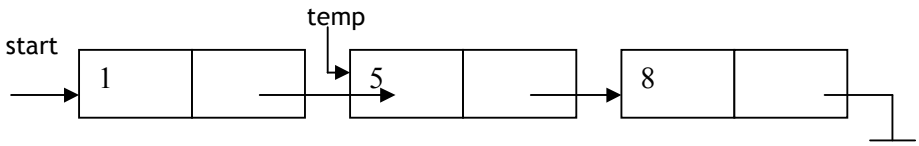
Here, like insertion we need the information as to which node is to be deleted. It can be the first node or any other node. So again there are two different considerations.

code snippet for Deletion of node

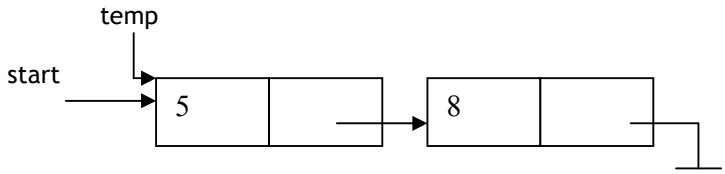
```
node * delete (node *start)
{
        node *tag, *team;
        int n, target;
        printf ("\nEmployee number to be deleted:");
        scanf ("%d", &target);
        if ( ( start ->e_no = = target) = = 0)
           { / * delete first node * /
/ * mark the node following the target node * /
            temp = start ->next;          <------------------------------   step 1
/ * free space of target node * /
            free (start);
/ * adjust pointer to first node */
            first = temp;                 <------------------------------   step 2
        }
      else
        {    / * delete data item other than first * /
             / * locate the node preceding the target node * /
             tag = locate (start, target);  <------------------------------   step a
             if (tag = = NULL)
             printf ("\n Match not found, place try again");
        else
           {
/ * mark the node following the target node * /
                  temp = tag ->next ->next;  <------------------------------   step b
/ * free space for target node * /
                  free (tag ->next);
/ * adjust link to the next node * /
               tag->next = temp;          <------------------------------   step c
             }
        }
    return (start);
}
```

If first node to be deleted

step 1



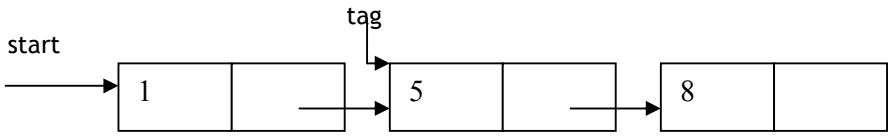
step 2



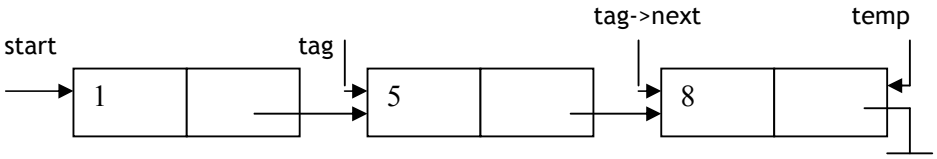
Node is freed using the function free.

If node whose e_no = 8 is to be deleted.

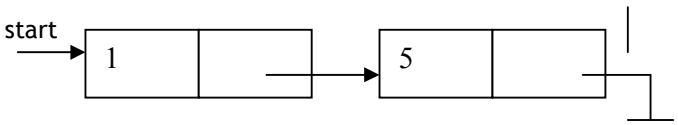
step a



step b



step c



Program performing operations on linklist

The below program is not full fledged, it is just sufficient to understand how linked list operations can be implemented. It can be made more sophisticated if required.

We can create a complete program with the help of following code snippet :

```
#include<stdio.h>
struct emp {
        int e_no;
        struct emp *next;
        };
        typedef struct emp node;

main ( )
{
        node *start;
        node* create (node *pt);
        void display (node *pt);
        node* insert (node *pt);
        node* locate (node *pt, int target);
        node* delete (node *pt);
        printf ("\nCreate a link list…. ");
        start = create (start);
        display (start);
        printf ("\nINSERT a node…");
        start = insert (start);
        display (start);
        printf ("\nDELETE a node…");
        start = delete (start);
        display (start);
        printf ("\nTHANK you !");
}
```

Review Questions

1. What are self referential structures?
2. What is a linklist?
3. Write a program to create a linklist
4. To create a linked list, declaring the number of nodes in the source code, is necessary.
    a. Yes
    b. NO

Summary

- We have seen input-output function used for transfer of data to and from computer.
- If we include a member in the structure, which is a pointer to the parent structure type. Such a structure is called as self-referential structure.
- Self-referential data structures are useful in applications that involve linked data structures such as lists and trees and graphs.
- Each element of a link list is called a node. Nodes of a linklist can be created, deleted, inserted.
- Linklist have a number of applications

Programming Concepts using C

Chapter 13: Miscellaneous

Objectives

- Discuss enumerations and use them.
- Discuss type casting.
- Use bitwise operators.
- One's complement operator.
- Bitwise AND operator.
- Bitwise OR operator.
- Bitwise XOR operator.
- Left shift operator.
- Right shift operator

Introduction

In this chapter we will study the enumeration constants and how they can be used in our programs. We will also see how data of any type can be changed explicitly by using casts. We will also study all the bitwise operators, OR operator, AND operator, XOR operator, left shift operator & right shift operator.

Enumerations

- What are enumerations?
- Why enumerations are used?
- Use of enumerations.
- Weakness of enumerations.

Enumerations are a way of defining constants. Here we will see what are enumerations? What is the need of enumerations? How we use them and the drawbacks of enumerations.

What are enumerations?

Enumerations are data-type like structures and unions.

In general enumerations may be defined as
enum tag {member1, member2,…, member n};
enum is the requird keyword, tag is the name of the enumeration and member1…membern are members (identifiers). Each member is a constant. The constant represent integer values. The member names must differ from each other.

It is similar to structures i.e once an enumeration has been defined, variable of that type may be defined.
enum tag var1, var2……varn:
The definition of enum and variable declaration may be combined together
enum tag {mem1, mem2, ….mem n} var1, var2;

E.g.
enum colors {black, blue, cyan, green, magenta, red, white, yellow);
enum colors background, foreground;

Colors are the **name**s of the enumeration.
Black, blue is the members of the enumeration which are called as **enumeration constants**.

Enumerations constants are automatically assigned integer values internally, starting with 0 for first member with each successive member increasing by 1.
i.e. member 1 is assigned 0
member 2 is assigned 1
. .
member n is assigned n -1

In our example

black	0
blue	1
cyan	2
green	3
magenta	4
red	5
white	6
yellow	7

If we want to have a different integer value assigned to the constants, it can be done by explicitly assigning value to the constant.

enum colors {black = 10, blue = 20, cyan = 30, green = 40, magenta = 50, red = 60, white = 70, yellow = 80};
If we don't assign explicit values to few of them, they will automatically be assigned values which increase successively by 1 from the last explicit assignment.
enum colors {black = -1, blue, cyan, green, magenta, red = 2, white, yellow};

The enumeration constants will represent the following integer values.

black	-1
cyan	0
green	2
magenta	3
red	2
white	3
yellow	4

Values need not be distinct in same enumeration. The enumeration variables are generally assigned enumeration constants rather than the integer values. They can be used like any other integer constants.

Why enumerations are used

Enumeration variables are particularly useful as flags to indicate various options or to identify various conditions; it increases the clarity of the program.
If I want to have color for the background, it is easier to understand with
background = blue;

Use of enumeration

Following example shows how enumerated data types are used. If we need to use all departments in a certain program, the departments are easier to understand if we use assembly, manufacturing, toolroom etc instead of integer values like 0, 1, 2…
Example(1)

```
#include<stdio.h>
# include<string.h>
void main()
{         enum emp_dept   { assembly, manufacturing, accounts, stores};
          struct employee {
                  char name [30];
                  int age;
                  float bs;
                  enum emp_dept dept; };
            struct employee e;
            strcpy (e.name, "Loather Mattheus");
            e .age = 28;
            e .bs = 5575.50;
            e .dept = manufacturing;
            printf ("\nName = %s",e.name);
            printf ("\nAge = %d", e.age);
            printf ("\nBasic salary = %f",e.bs);
            printf ("\nDepartment = %d", e.dept);
          if (e.dept == accounts)
                  printf ("\n%s is an accountant", e.name);
          else
                  printf ("\n%s is not an accountant", e.name);
}
```

Output
Name = Loather Mattheus
Age = 28

Basic salary = 5575.50
Deparement = 1
Loather Mattheus is not an account

Weakness of Enumerations

When we use printf statement to print enumeration variables, the enumeration constant cannot be printed. Only the equivalent integer value can be printed. Ordinary integer variables can be used in place of enumeration variables. Thus enumeration variables do not provide any fundamentally new capabilities.

Type casting

We have seen that compiler causes internal type conversion automatically when it has to handle operands of different type.
There is also provision for programmer to explicitly convert value of an expression into a different data type. To do so, expression must be preceded by name of desire data type enclosed in parenthesis.
(data-type) expression;
This is called type casting done with a unary operator cast.
E.g

```
main ( )
{
        int x = 6, y = 4;
        float a;
        a = x / y;
}
```

The expression x/y will cause integer division (i.e. decimal part will be truncated) giving result 1.
It will then be converted to float (1.00) since it is to be assigned to "a" which is a float.
But if we want that the result should not be truncated and some other factors don't allow us to define x as float then we can use type casting.

```
main ( )
{
      int x = 6, y = 4;
      float a;
      a = (float) x / y;
      printf ("\nX  = %d", x);
}
```

The float(x), converts x into a floating point value before performing the division so that result is floating point. The conversion produces x into proper type. The value of x is not changed permanently as seen from the printf statement that follows.

Another example can be given as a function 'sqrt' which takes double as argument and would give nonsense results if some other type of data is given.

```
int x;
double y;
y = sqrt (double (x) );
x is converted to type double.
```

Bitwise operator

- One's complement operator
- Logical bitwise operator
- Shift operator

Operators can be divided into 3 categories. One's complement operator, logical bitwise operator and shift operators.
These operations can be performed only on integers (short, long) and character whether signed or unsigned.

We have considered byte as the smallest thing that can be operated upon. But each byte is further made up of bits.'C' contains several operators that allow bitwise operations to be carried out easily. Using bitwise operations we can directly interact with the hardware. The collection of bits is nothing but binary representation of numbers.

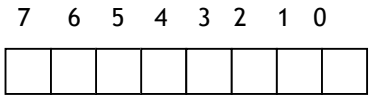
Decimal	Binary
5	00000101
6	00000110
5225	00010100 01101001

byte

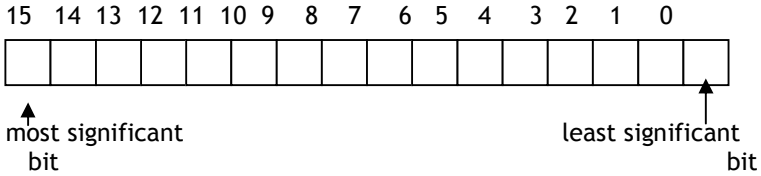
word

Bits are numbered from zero onwards, increasing from right to left.

Character



Integer



When we are using bitwise operators, we are considering the binary representation of numbers

One's complement operator

The ones complement operator, which is a unary operator, causes the bits of its operand to be inverted. i.e. each 1 is converted to 0 and each 0 is converted to 1. The operator precedes the operand.

E.g.
~(5225) = 59286
Here '~' being a bitwise operator we will consider binary equivalent of operand.
Binary equivalent of 5225 is 0001010001101001.
On inversion it will result into 1110101110010110 = 59286.

Decimal 1 is same as binary 0000000000000001. One's complement of one is 1111111111111110. As it is seen one's complement of a number gives an entirely different number. Hence it can be used to encrypt a file.

Logical bitwise operators

- AND operator
- OR operator
- XOR operator

There are three types of logical bitwise operators AND operator (&), bitwise OR operator (|), bitwise XOR operator (^).
These operators use two operands. The operator has to be applied to each pair of bits (One from either operand), independent of other bits within the operand. The least significant bits within the two operands will be considered, then the next least significant bit and so on until operator has been applied to all bits. Both operands must be of the same type.

AND operator

Two bits on AND results into one only if both are one. The AND operation can be better understood by looking at the truth table.

Bit from operand 1	Bit from operand 2	Result
0	0	0
1	0	0
0	1	0
1	1	1

There is a difference between the logical AND operator (&&) and bitwise AND operator (&).
E.g.
If x =1 and y = 2

Then x && y will consider x as one expression which on evaluation gives true (Since it has some value other than 0) and y as other expression which on evaluation gives true. Thus result is true
Therefore (x && y) is 1.

For x & y, binary equivalent of x will be ANDed with binary equivalent of y, bit by bit.

```
x        0000 0001
y        0000 0010
_________________
         0000 0000
```

After AND (refer truth table)
Therefore (x & y) is 0.

The AND operation can be used to

- Check whether particular bits are ON or OFF.
- To turn of any bits.

In either case, the second operand has to be constructed according to the bits to be tested or to be turned off. It is called as mask

Check whether particular bits are ON or OFF

We have a bit pattern 1010 1101 and we want to check if 3rd bit is ON or OFF. We will construct a bit pattern (mask) such that 3rd bit in the pattern is 1. Mask is 0000 1000

AND these two patterns,

```
1010 1101    Original bit pattern
0000 1000     AND Mask
___________
0000 1000
```

We see that the resulting bit pattern has third bit ON. When will the resulting pattern have third bit ON? It is when third bit of both operands are ON. We have constructed a mask such that third bit is ON. Thus third bit in the original bit pattern must be ON.

To turn of any bits.

If we require that fifth bit of given bit pattern 1010 1101 is turned OFF, we will construct a bit pattern such that its fifth bit is OFF and all other bits are ON. So that whatever is the fifth bit of given pattern, result will have fifth bit set to OFF, while all other remain unchanged.

```
1010 1101    Original bit pattern
1101 1111     AND mask
____________
1000 1101    Resulting bit pattern (fifth bit turned OFF)
```

OR operator

Two bits on OR result into 1 if either bit is 1. The truth table for OR operation can be given as follows

Bit from operand 1	Bit from operand 2	Result
0	0	0
0	1	1
1	0	1
1	1	1

Bitwise OR operator is used to turn ON particular bit in a number. The bit to be turned ON in the original pattern should be made 1 in the OR mask. All other bits in the mask kept to 0 & remaining original bits remain unchanged.

```
1101 0000    Original bit pattern
0000 0111    OR mask
____________
1101 0111    Resulting bit pattern
```

XOR operator

It is called as bitwise exclusive OR operator. It excludes the last condition in the OR truth table. i.e. the bitwise exclusive OR operator sets 1 in each bit position where its operands have different bits and 0 where they are same.

Bit from operand one	Bit from operand two	Result
0	0	0
0	1	1
1	0	1
1	1	0

XOR operation can be used toggle (invert) bits. If we want to toggle last 8 bit of a pattern,

```
a = 0110 1101 1011 0111    Original bit pattern
    0000 0000 1111 1111     XOR mask
    ____________________
    0110 1101 0100 1000     Result
```

When each of the rightmost 8 bit in ‘a’ is XOR with corresponding 1 in mask, the resulting bit is opposite of bit originally in ‘a’. On the other hand, when each of the leftmost 8 bit in ‘a’ is XORed with corresponding 0 in mask, resulting bit will be same as bit originally in ‘a’.

Shift operators

Right shift operator
Left shift operator

These operators take two operands. The first operand is the bit pattern to be shifted and the second is an unsigned integer, a number that indicates the number of places the bits are shifted.

Right shift operator

>> is right shift operator.

E.g.

x>>3 shifts all bits in x, three places to the right.
If x contains the bit pattern 1101 0111, ch>>3 would give 0001 1010.

When bits are shifted to the right, the blanks created at the left are filled with zeros if the operand is unsigned. If the operand is *signed, blanks are filled with either zeroes or signbit. It depends on the machine.

If operand is a multiple of 2, then shifting the operand one bit to right causes division by 2.

64 >> 1 = (0100 0000) >> 1 = (0010 0000) which is the binary equivalent of 32.
64 >> 2 = (0100 0000) >> 2 = (0001 0000) which is the binary equivalent of 16.

* Signed quantity: Leftmost bit is used to indicate sign of a signed number. It is called the signbit. Leftmost bit is 1 if number is negative. Leftmost bit is 0 if number is positive.

Left shift operator

<< is the left shift operator.
E.g.
y << 5 shifts all bits in y, five places to the left.

If y contains the bit pattern 0110 1001, y << 5 gives 0010 0000. For each bit shifted to left, a zero is added to the right of the number.
If operand is multiple of two then shifting operand one bit two left causes multiplication by 2.

Summary

- Enumerated data types provide us with enumeration constants. These are used as flags or to indicate certain conditions.
- Typecasting does not permanently change the type of variable. It only temporarily presents the variable in the required data type.
- All the bitwise operators, one's complement, bitwise AND, bitwise OR, bitwise XOR, left shift, right shift are used to perform operations on individual bits in an operand. They take us close to hardware related manipulations.